ackee
blockchain security

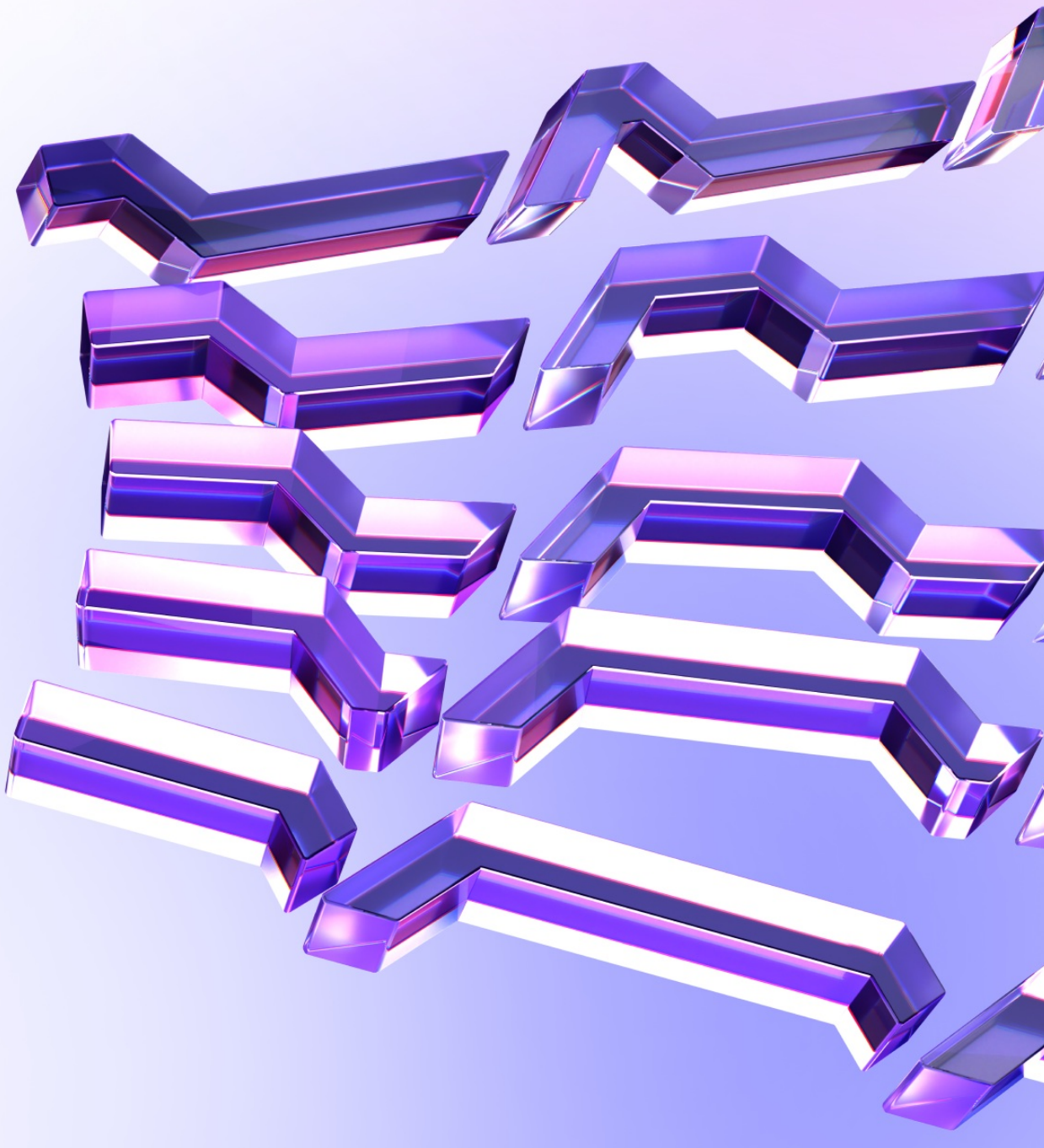
Wake

IPOR-Labs

ipor-fusion

December 11, 2025 18:18 UTC

Wake Arena AI Report



Contents

1. Document Revisions . . . 3
2. Overview . . . 4
2.1. Ackee Blockchain Security . . . 4
2.2. Wake Arena . . . 5
2.3. Disclaimer . . . 5
2.4. Finding Classification . . . 6
3. Executive Summary . . . 7
Revision 1.0 . . . 7
4. Findings Summary . . . 11
Report Revision 1.0 . . . 22
Findings . . . 22
Appendix A: How to cite . . . 282

1. Document Revisions

1.0	AI Scan	Dec 11, 2025 18:18 UTC

2. Overview

This document presents the findings Wake Arena identified using AI vulnerability checks in the reviewed contracts.

2.1. Ackee Blockchain Security

Ackee Blockchain Security is an in-house team of security researchers performing security audits focusing on manual code reviews with extensive fuzz testing for Ethereum and Solana. Ackee is trusted by top-tier organizations in web3, securing protocols including Lido, Safe, and Axelar.

We develop open-source security and developer tooling Wake for Ethereum and Trident for Solana, supported by grants from Coinbase and the Solana Foundation. Wake and Trident help auditors in the manual review process to discover hardly recognizable edge-case vulnerabilities.

Our team teaches about blockchain security at the Czech Technical University in Prague, led by our co-founder and CEO, Josef Gattermayer, Ph.D. As the official educational partners of the Solana Foundation, we run the School of Solana and the Solana Auditors Bootcamp.

Ackee's mission is to build a stronger blockchain community by sharing our knowledge.

Ackee Blockchain a.s.
Rohanske nabrezi 717/4
186 00 Prague, Czech Republic
https://ackee.xyz
hello@ackee.xyz

2.2. Wake Arena

This report was created by Wake Arena, an automated vulnerability analysis tool. It uses Wake Framework with additional detectors for AI and static analysis of Solidity smart contracts.

To find potential vulnerabilities, Wake uses:

- The code structure and patterns
- Control flow graphs
- Data flow graphs
- Common vulnerability patterns
- Contract interactions

Wake Arena aims for precision, not recall. This means the presented findings are optimized for true positives, not for finding all possible issues, and should always be complemented with additional manual review for a complete security assessment.

2.3. Disclaimer

We've put our best effort to find known vulnerabilities in the system, but automated findings shouldn't be considered a complete list of all existing issues. The statements made in this document should not be interpreted as investment or legal advice, nor should its authors be held accountable for decisions made based on them.

2.4. Finding Classification

Each finding is classified by two independent ratings: *Impact* and *Confidence*.

Impact

Measuring the potential consequences of the issue on the system.

- **Critical** - Code that activates the issue directly enables theft of significant assets with minimal preconditions.
- **High** - Code that activates the issue will lead to undefined or catastrophic consequences for the system.
- **Medium** - Code that activates the issue will result in consequences of serious substance.
- **Low** - Code that activates the issue will have outcomes on the system that are either recoverable or don't jeopardize its regular functioning.
- **Warning** - The issue represents a potential security concern in the code structure or logic that could become problematic with code modifications.
- **Info** - The issue relates to code quality practices that may affect security. Examples include insufficient logging for critical operations or inconsistent error handling patterns.

Confidence

Indicating the probability that the identified issue is a valid security concern.

- **High** - The analysis has identified a pattern that strongly indicates the presence of the issue.
- **Medium** - Evidence suggests the issue exists, but manual verification is recommended.
- **Low** - Potential indicators of the issue have been detected, but there is a significant possibility of false positives.

3. Executive Summary

Revision 1.0

Audit Overview

The IPOR-Labs/ipor-fusion codebase at commit `1e25bb68c4f3b33fd0b6ce20f17f70e2d5b18b40` was assessed across the in-scope vault core, protocol integration fuses (Aave V3, Euler V2, Morpho), price feeds, universal token swapper, whitelist wrappers, transient storage utilities, zaps, and reward claimers. The architecture centers on an ERC-4626-compatible `PlasmaVault` that orchestrates actions through pluggable fuses and supporting libraries. A total of 78 AI-detected issues were identified in this snapshot, spanning critical correctness, access control, accounting, and oracle integrity concerns.

Detected issue tally: 2 critical, 16 high, 20 medium, 17 low, 6 warning, 17 info (total 78).

The most severe AI-detected issues concentrate on access control and guardrail correctness at execution boundaries:

- Aave V3 borrowing can be triggered by any caller: `AaveV3BorrowFuse.enter` calls `IPool.borrow(…, address(this))` without caller authorization, enabling unpermissioned debt creation against the fuse/vault and pushing health factor toward liquidation.
- Slippage guard inversion: `UniversalTokenSwapperFuse` computes `SLIPPAGE_REVERSE = 1e18 - slippageReverse_` while inputs are already `1e18 - slippage`, inverting the threshold so `_checkSlippage` accepts severely unfavorable prices (e.g., up to ~95% loss still passes).
- Whitelist enforcement gaps: `WhitelistWrappedPlasmaVault.withdraw` and `redeem` can be routed via allowance to bypass receiver/owner whitelist

checks, undermining access restrictions.

- Permissionless batch execution: `EulerV2BatchFuse.enter` and `enterTransient` are callable by anyone, allowing arbitrary EVC batch operations on fuse-owned subaccounts.
- Accounting DoS via unsafe casts: balance fuses for Aave/Euler cast signed net positions to `uint256` (e.g., via `SafeCast.toUint256`) and revert on negative balances, halting vault-wide balance refresh logic.
- Oracle and pricing correctness: `PtPriceFeed` omits the `SY.exchangeRate` factor when using `getPtToSyRate` and returns the current block timestamp from `latestRoundData`, undermining USD normalization and freshness checks used by consumers.
- Zap and executor surfaces: `ERC4626ZapInWithNativeToken` allows user-controlled refund lists and arbitrary multicall that can sweep ERC20 residues held by the zap; `SwapExecutorRestricted` exhibits stuck ETH and double-transfer edge cases.

Overall, the security posture shows a modular design with clear separation of concerns and extensive use of OpenZeppelin upgradeable primitives, but several fuses expose public entrypoints without sufficient gating, multiple guardrails (slippage, whitelist, limits) are mis-specified or bypassable, and a number of accounting/rounding defects can cause DoS or user value deviations. The combination of 2 critical and 16 high-severity detections across core execution and pricing paths indicates the codebase is not yet ready for mainnet deployment without targeted remediations and additional hardening.

Key Technical Findings

- `AaveV3BorrowFuse.enter` and `enterTransient` are public and perform `IPool.borrow` with `onBehalfOf = address(this)` without caller checks, allowing any EOA to induce new variable debt on the contract and degrade

the Aave position health factor.

- `UniversalTokenSwapperFuse.<constructor>` subtracts `slippageReverse_` again (`SLIPPAGE_REVERSE = 1e18 - slippageReverse_`) despite NatSpec requiring `1e18 - slippage` as input; `_checkSlippage` then accepts `out/in` ratios far below the intended minimum.
- `EulerV2BatchFuse.enter` is permissionless and executes EVC batch operations on fuse-owned subaccounts; `enterTransient` allows attacker-staged transient inputs to be consumed on the next call.
- `WhitelistWrappedPlasmaVault.withdraw` and `redeem` enforce whitelist on `msg.sender` only; allowance flows enable non-whitelisted `owner`/`receiver`, bypassing the intended restrictions.
- Balance calculation fuses cast signed net values to `uint256` (e.g., Aave/Euler balance fuses), reverting when the net market value is negative and causing denial of service in vault balance refresh paths.
- `AssetDistributionProtectionLib.checkLimits` enforces zero-tolerance comparisons; floor rounding lets attackers use dust deltas to trigger limit violations and DoS distributions.
- `PtPriceFeed` USD conversion misses the `SY.exchangeRate` factor when using `getPtToSyRate`, and `latestRoundData` returns `block.timestamp` rather than the source data timestamp, weakening staleness guarantees.
- `RedemptionDelayLib` exposes unauthenticated per-account lock updates and applies a global, cross-vault lock, enabling griefing and cross-vault coupling of redemption delays.
- `ERC4626ZapInWithNativeToken` exposes arbitrary multicall and user-controlled refund sweeping, enabling transfer of any ERC20 balance held by the zap and creating stuck-ETH scenarios for non-payable receivers.
- Transient storage conversion routines (`TransientStorageMapperFuse`) lose sign and precision on signed extractions and perform lossy round-trips without early returns when types/decimals match; packing performs

unchecked narrowing.

- Fuse registry logic (`FuseWhitelist`/`FuseWhitelistLib`) treats `fuseType == 0` as not-found sentinel in update paths, creating DoS for valid type-0 fuses; remove operations emit updates even when the metadata ID is absent.
- ERC-4626 conformance gaps in `PlasmaVault`: `previewWithdraw` divides by the fee rather than `(1e18 - fee)` which inflates required shares; `withdraw` can transfer fewer assets than requested; asymmetric virtual offsets permit share inflation at low TVL.

Conclusion

Given 2 critical and 16 high-severity detections affecting access control, slippage protection, whitelist enforcement, oracle correctness, and core accounting, the current codebase is not ready for deployment. The modular architecture is promising, but execution paths must be gated, guardrails corrected, and accounting/rounding defects fixed. We recommend addressing the listed findings, adding stronger input validation and staleness checks, and performing another full audit and scenario-driven testing pass before mainnet use.

4. Findings Summary

Summary of findings:

Critical	High	Medium	Low	Warning	Info	Total
2	16	20	17	6	17	**78**

Table 1. Findings Count by Impact

Findings in detail:

Finding title	Impact	Reported	Status
C1: Inverted slippage threshold in constructor makes slippage guard ineffective	**Critical**	1.0	Reported
C2: Public borrow entrypoint allows unpermissioned Aave V3 debt creation on behalf of the contract	**Critical**	1.0	Reported
H1: Negative net market value makes balanceOf revert via SafeCast.toUint256, causing delegatecalled balance updates to fail (DoS)	**High**	1.0	Reported
H2: Whitelist bypass in redeem allows non-whitelisted owner/receiver via allowance proxy	**High**	1.0	Reported
H3: Signed extraction clamps negatives to zero, silently losing sign across mappings	**High**	1.0	Reported

Finding title	Impact	Reported	Status
H4: Mispriced PT when using getPtToSyRate: missing SY.exchangeRate factor in USD conversion	**High**	1.0	Reported
H5: Double inversion of slippageReverse_ in constructor severely weakens slippage protection	**High**	1.0	Reported
H6: Unauthenticated redemption-lock updates enable third-party account locking via public manager	**High**	1.0	Reported
H7: Deposit path bypasses deposit fee; previews and mint realize fee but deposit does not	**High**	1.0	Reported
H8: Unsafe cast of signed net balance to uint256 reverts on negative values, causing vault-wide DoS via delegatecall	**High**	1.0	Reported
H9: Whitelist bypass in withdraw allows non-whitelisted owner/receiver via allowance proxy	**High**	1.0	Reported
H10: latestRoundData returns current block timestamp instead of underlying data timestamp, undermining freshness checks	**High**	1.0	Reported

Finding title	Impact	Reported	Status
H11: Missing early-return in _convertValue forces a lossy round-trip even when types and decimals are identical	**High**	1.0	Reported
H12: Permissionless rEUL claim enables forced remainder diversion via allowRemainderLoss	**High**	1.0	Reported
H13: enter is permissionless: anyone can execute EVC batch on fuse-owned Euler subaccounts	**High**	1.0	Reported
H14: User-controlled refund list allows sweeping any ERC20 balances held by the zap contract	**High**	1.0	Reported
H15: enterTransient is permissionless and executes enter with attacker-staged transient inputs	**High**	1.0	Reported
H16: Zero-tolerance limit check in checkLimits enables dust-induced DoS via floor rounding	**High**	1.0	Reported
M1: Public fee realization truncates management fees via pre-mint timestamp advance (griefing)	Medium	1.0	Reported
M2: Unsafe uint256 cast in AaveV3BalanceFuse.balanceOf causes revert on negative net balance, DoSing balance refresh	Medium	1.0	Reported

Finding title	Impact	Reported	Status
M3: Composite price feed lacks staleness and round-integrity validation	Medium	1.0	Reported
M4: Third-party deposits/mints can grief-lock arbitrary receivers for the full redemption delay	Medium	1.0	Reported
M5: Validation accepts onEulerFlashLoan callback to this contract, but the function is not implemented (batch reverts)	Medium	1.0	Reported
M6: updateFuseState treats fuseType==0 as not-found sentinel, causing DoS for type-0 fuses; existence check inconsistent with other update paths	Medium	1.0	Reported
M7: withdraw transfers fewer assets than requested, violating ERC-4626 semantics	Medium	1.0	Reported
M8: previewWithdraw divides by fee instead of 1e18 - fee, inflating required shares	Medium	1.0	Reported
M9: ERC4626.deposit without minSharesOut enables share-slippage from MEV donations and rate drift	Medium	1.0	Reported

Finding title	Impact	Reported	Status
M10: exit misuses maxCollateralAmount as exact value, causing API mismatch and revert-prone withdrawals	Medium	1.0	Reported
M11: DoS in updateFuseState: existence check uses fuseType == 0, blocking valid type-0 fuses	Medium	1.0	Reported
M12: Repayment blocked when borrowing is disabled due to over-restrictive validation	Medium	1.0	Reported
M13: exit returns zero in success path due to missing assignment in _performWithdraw	Medium	1.0	Reported
M14: Asymmetric units across enter and exit: enter returns shares, exit returns assets	Medium	1.0	Reported
M15: Unchecked narrowing in _packNumericValue truncates or two's-complement wraps values (including address truncation)	Medium	1.0	Reported
M16: Unsafe conversion in toBytes32(bytes): unmasked mload on short inputs yields dirty higher-order bytes	Medium	1.0	Reported
M17: Unvalidated vault address can permanently trap user assets (no post-transfer check or rescue)	Medium	1.0	Reported

Finding title	Impact	Reported	Status
M18: Global per-account redemption lock (not vault-scoped) causes cross-vault coupling and DoS	Medium	1.0	Reported
M19: Asymmetric virtual offsets in conversions enable share inflation at low TVL	Medium	1.0	Reported
M20: Whitelist checks only caller; non-whitelisted receivers and owners permitted via receiver/allowance	Medium	1.0	Reported
L1: Dust supply shares can become stuck due to early return when assetsMax == 0 in _exit	Low	1.0	Reported
L2: Caught-withdraw path returns requested amount, misreporting actual withdrawal	Low	1.0	Reported
L3: Native ETH sent to the executor is stuck (no payable receive and no withdrawal path)	Low	1.0	Reported
L4: ETH sent to the contract is unrecoverable (no payable receiver and no withdrawal)	Low	1.0	Reported
L5: removeFuseMetadata does not revert on absent metadataId and still emits update event	Low	1.0	Reported

Finding title	Impact	Reported	Status
L6: Double transfer when tokenIn == tokenOut triggers revert and denies execution	Low	1.0	Reported
L7: Unconditional ETH refund to sender can revert for non-payable contracts, causing denial of service	Low	1.0	Reported
L8: Redundant pre-approval to Morpho before flashLoan widens allowance window and wastes gas	Low	1.0	Reported
L9: Zero-amount supply in enter can revert on Aave and halt execution pipeline	Low	1.0	Reported
L10: latestRoundData leaves metadata unset (roundId/startedAt/answeredInRound = 0), breaking AggregatorV3 consumer assumptions	Low	1.0	Reported
L11: Division by zero in slippage check when USD-in delta rounds to zero	Low	1.0	Reported
L12: Enter event unit mismatch: emits minted shares as supplyAmount documented as assets	Low	1.0	Reported
L13: Arbitrary-call zap can exfiltrate any ERC20 residue via unrestricted refund sweep	Low	1.0	Reported

Finding title	Impact	Reported	Status
L14: Deposit wrapper lacks minSharesOut slippage protection, enabling front-running to reduce minted shares	Low	1.0	Reported
L15: Trusting distributor return value can strand surplus rewards in MorphoClaimFuse	Low	1.0	Reported
L16: Unrestricted external multicall lets anyone transfer tokens owned by the zap contract	Low	1.0	Reported
L17: Division-by-zero in slippage verification for dust-sized swaps (DoS)	Low	1.0	Reported
W1: Missing length equality check between fuse and inputsByFuse enables out-of-bounds panic in enter	Warning	1.0	Reported
W2: Comment-implementation mismatch in _convertValue (no early return for same type and decimals)	Warning	1.0	Reported
W3: Validator blocks `flashLoan`: documentation claims support but `_validateEulerVault` rejects it	Warning	1.0	Reported
W4: NatSpec claims reentrancy protection in `enter`, but no guard is implemented	Warning	1.0	Reported

Finding title	Impact	Reported	Status
W5: Arbitrary approves via multicall persist allowances that enable future drains of newly received tokens	Warning	1.0	Reported
W6: EIP-7201 storage slots contradict comments; constants are contiguous sub-slots from one namespace	Warning	1.0	Reported
I1: ETH can be forcibly sent and permanently stuck; no recovery mechanism	Info	1.0	Reported
I2: Unused on-chain deployment of ERC4626ZapInAllowance and exposed public address	Info	1.0	Reported
I3: Unused public immutable VERSION adds dead code and exposes misleading getter	Info	1.0	Reported
I4: Unused custom error FuseDoesNotHaveMarketId is dead code and misleading	Info	1.0	Reported
I5: Dead immutable ASSET_DECIMALS (never read): bytecode bloat and hidden normalization assumptions	Info	1.0	Reported
I6: Documentation claims per-asset grant validation that the implementation does not perform	Info	1.0	Reported

Finding title	Impact	Reported	Status
I7: Redundant getPoolDataProvider call inside per-asset loop increases gas and failure surface	Info	1.0	Reported
I8: NatSpec/documentation mismatch: references `dexData` instead of `dexsData`	Info	1.0	Reported
I9: Documentation mismatch: updateMarketsBalances is restricted but NatSpec claims public/no restriction	Info	1.0	Reported
I10: `enterTransient` NatSpec output layout does not match implementation	Info	1.0	Reported
I11: Two sources of truth for output decimals (decimals constant vs _decimals helper) risk divergence	Info	1.0	Reported
I12: Unused local variable `coinBalance` in latestRoundData	Info	1.0	Reported
I13: Unused custom errors: `EVCInvalidSelector`, `DuplicateAsset`, `InvalidBatchItem`	Info	1.0	Reported
I14: Unused custom error `CurveStableSwapNG_InvalidBalance` is declared but never used	Info	1.0	Reported
I15: Always-true boolean return in transferApprovedAssets is redundant and misleading	Info	1.0	Reported

Finding title	Impact	Reported	Status
I16: Gas waste from copying large external parameter into memory instead of using calldata	Info	1.0	Reported
I17: Comment claims fixed 8-decimal prices while implementation uses dynamic oracle decimals (documentation mismatch)	Info	1.0	Reported

Table 2. Table of Findings

Report Revision 1.0

Findings

The following section presents the list of findings discovered in this revision.

For the complete list of all findings, Go back to Findings Summary

C1: Inverted slippage threshold in constructor makes slippage guard ineffective

Critical impact finding

Impact:	Critical	Confidence:	High
Target:	./contracts/fuses/universal_token_swapper/UniversalTokenSwapperFuse.sol	Type:	Logic error

Description

The contract documents `SLIPPAGE_REVERSE` as the minimum acceptable output/input ratio in WAD, defined as `1e18 - slippage percentage`. However, the constructor subtracts this value again and sets `SLIPPAGE_REVERSE = 1e18 - slippageReverse_`. This double subtraction inverts the threshold and effectively stores the raw slippage percentage (e.g., `0.05e18` for 5%) instead of the intended minimum ratio (e.g., `0.95e18`).

Consequently, function `_checkSlippage` compares the USD-value ratio `out/in` against the wrong bound and allows swaps with extremely unfavorable prices to pass the guard. With a nominal 5% tolerance, the check accepts `out/in >= 0.05e18` instead of `out/in >= 0.95e18`, permitting up to ~95% loss without reverting.

Affected elements:

- State variable `SLIPPAGE_REVERSE` (immutable)
- Function `<constructor>` where the value is derived
- Function `_checkSlippage` where the value is used
- Function `enter` that relies on `_checkSlippage` as the final safety gate

Listing 1. Code snippet from contracts/fuses/universal_token_swapper/UniversalTokenSwapperFuse.sol

```
70 /// @dev slippageReverse in WAD decimals, 1e18 - slippage;
71 uint256 public immutable SLIPPAGE_REVERSE;
72 uint256 private constant _ONE = 1e18;
```

Listing 2. Code snippet from contracts/fuses/universal_token_swapper/UniversalTokenSwapperFuse.sol

```
81 constructor(uint256 marketId_, address executor_, uint256 slippageReverse_) {
82     if (executor_ == address(0)) {
83         revert UniversalTokenSwapperFuseInvalidExecutorAddress();
84     }
85     if (slippageReverse_ > _ONE) {
86         revert UniversalTokenSwapperFuseSlippageFail();
87     }
88     VERSION = address(this);
89     MARKET_ID = marketId_;
90     EXECUTOR = executor_;
91     SLIPPAGE_REVERSE = _ONE - slippageReverse_;
92 }
```

The threshold is then consumed in `_checkSlippage` as the lower bound for the WAD ratio `amountUsdOutDelta / amountUsdInDelta`:

Listing 3. Code snippet from contracts/fuses/universal_token_swapper/UniversalTokenSwapperFuse.sol

```
196 uint256 amountUsdInDelta = IporMath.convertToWad(
197     tokenInDelta_ * tokenInPrice, IERC20Metadata(tokenIn_).decimals() +
    tokenInPriceDecimals
198 );
199 uint256 amountUsdOutDelta = IporMath.convertToWad(
200     tokenOutDelta_ * tokenOutPrice, IERC20Metadata(tokenOut_).decimals() +
    tokenOutPriceDecimals
201 );
202
203 uint256 quotient = IporMath.division(amountUsdOutDelta * 1e18,
    amountUsdInDelta);
204
```

```
205 if (quotient < SLIPPAGE_REVERSE) {
206     revert UniversalTokenSwapperFuseSlippageFail();
207 }
```

Numerical example (WAD): If governance/config passes `slippageReverse_ = 0.95e18` to represent 5% tolerance as documented, the constructor computes `SLIPPAGE_REVERSE = 0.05e18`. For a swap that returns only 10% of the USD value (`out/in = 0.10e18`), the guard still passes because `0.10e18 >= 0.05e18`.

This is a pure logic bug amplified by documentation mismatch: integrators are encouraged by the NatSpec to pass `1e18 - slippage`, which then gets subtracted once more in the constructor.

Exploit Scenario

A mispriced route or malicious execution can realize immediate and outsized losses because the slippage guard is inverted.

Consider the following attack scenario:

- Alice prepares swap data that routes through a thin-liquidity pool she controls, returning only 10% of fair value;
- Alice triggers the swap via the protocol's execution path that uses this fuse (or compromises the executor/route);
- The contract computes `out_usd / in_usd` and compares it to `SLIPPAGE_REVERSE`, which is erroneously set near `5e16` for a nominal 5% tolerance; and
- Bob's vault observes the check pass (`0.10e18 >= 0.05e18`) and accepts the trade, crystallizing a ~90% loss that should have been rejected.

Recommendation

Fix the threshold definition so that the stored value matches the comparison semantics, then enforce it consistently.

Two safe approaches:

- Treat the constructor parameter as the minimum acceptable ratio in WAD (recommended):
- Rename to `minAcceptableRatioWad` (or keep the existing name but update NatSpec);
- Set `SLIPPAGE_REVERSE = minAcceptableRatioWad;` and validate `minAcceptableRatioWad <= 1e18`;
- Maintain `_checkSlippage` as `if (quotient < SLIPPAGE_REVERSE) revert`.
- Alternatively, treat the constructor parameter as a slippage percentage in WAD and compute the threshold once:
- Rename to `slippagePercentWad` and set `SLIPPAGE_REVERSE = _ONE - slippagePercentWad;` with bounds `slippagePercentWad <= 1e18`;
- Update all NatSpec comments to reflect the new meaning of the parameter/value.

Additionally:

- Add unit tests that assert a 5% tolerance rejects `out/in = 0.94e18` and accepts `out/in = 0.95e18`;
- Include deployment checks that log the final `SLIPPAGE_REVERSE` so operators can verify the configured threshold.

Go back to Findings Summary

C2: Public borrow entrypoint allows unpermissioned Aave V3 debt creation on behalf of the contract

Critical impact finding

Impact:	Critical	Confidence:	High
Target:	./contracts/fuses/aave_v3/AaveV3BorrowFuse.sol	Type:	Access control

Description

A publicly callable borrow entrypoint that lacks caller authorization enables any address to trigger Aave V3 variable debt creation on behalf of this contract. The function sets `onBehalfOf` to `address(this)`, so the debt owner is the contract itself. The only gate is an asset allowlist, which validates the asset but not the caller.

If this contract has borrowing power on Aave (for example, because it already supplied collateral), any external user can call `enter` with an allowed asset and arbitrary amount up to available borrowing power. This unpermissioned borrow will:

- Increase variable debt for the contract;
- Transfer the borrowed tokens to the contract address; and
- Reduce the health factor, potentially to liquidation levels, with ongoing interest accrual.

There is no role-based check, no vault-only guard, and no verification that the call is executed through the restricted orchestrator. As a result, the borrow surface is fully public.

Affected components:

- Functions: `enter`, `enterTransient` (via `enter`), both callable without access control;
- State: `AAVE_V3_POOL_ADDRESSES_PROVIDER`, `MARKET_ID`, and constant `INTEREST_RATE_MODE` participate in the borrow path;
- External call: `IPool.borrow` is invoked with `onBehalfOf = address(this)`.

Concrete evidence from the code shows the lack of access control and the direct call into Aave V3:

Listing 4. Code snippet from contracts/fuses/aave_v3/AaveV3BorrowFuse.sol

```
96 function enter(AaveV3BorrowFuseEnterData memory data_) public returns
   (address asset, uint256 amount) {
97     if (data_.amount == 0) {
98         return (data_.asset, 0);
99     }
100
101     if (!PlasmaVaultConfigLib.isSubstrateAsAssetGranted(MARKET_ID,
   data_.asset)) {
102         revert AaveV3BorrowFuseUnsupportedAsset("enter", data_.asset);
103     }
104
105     IPool(IPoolAddressesProvider(AAVE_V3_POOL_ADDRESSES_PROVIDER).getPool())
106         .borrow(data_.asset, data_.amount, INTEREST_RATE_MODE, 0, address
   (this));
107
108     emit AaveV3BorrowFuseEnter(VERSION, data_.asset, data_.amount,
   INTEREST_RATE_MODE);
109
110     return (data_.asset, data_.amount);
111 }
```

Impact in production:

- Any address can drain the contract's borrowing capacity across all granted assets;
- Health factor can be pushed toward liquidation, exposing the position to

liquidators;

- Continuous interest accrual imposes an operational burden to repay or rebalance;
- Operational disruption if automated strategies assume stable borrowing power.

Exploit Scenario

An external caller can force the contract to take on new Aave V3 variable debt without any authorization, provided the contract has borrowing power.

Consider the following attack scenario:

1. Alice observes that the contract has collateral on Aave and sufficient borrowing capacity for USDC;
2. Alice calls `enter` with `{ asset: USDC, amount: maxBorrowable }` from her EOA;
3. The function passes the asset allowlist check and calls `IPool.borrow(asset, amount, 2, 0, address(this))`;
4. The borrowed USDC is transferred to the contract and the corresponding variable debt is assigned to the contract; and
5. Bob, operating the protocol, now faces degraded health factor and ongoing interest accrual, and must repay or liquidations may occur.

Variations:

- Repeated calls across multiple granted assets to fully exhaust borrowing power;
- Sequencing borrows to leave the health factor just above liquidation, making the position fragile to price moves;
- Front-running legitimate rebalancing to maximize disruption costs.

Recommendation

Restrict who can invoke borrowing and enforce that calls come only from the intended orchestrator context.

Recommended changes:

- Add a guard that blocks direct calls and only allows delegatecall execution from the orchestrator (vault). A minimal, robust pattern in this codebase is to revert when executed as a standalone call by comparing the fixed `VERSION` with `address(this)`:
- Example: `require(address(this) != VERSION, "Only orchestrator (delegatecall) can call");` at the top of `enter` and `enterTransient`.
- Alternatively, make the borrow path role-gated using an allowlist set at deployment, e.g., store the orchestrator address in an immutable and require `msg.sender == ORCHESTRATOR` when not executed via delegatecall.
- Keep the existing asset allowlist, but do not rely on it for caller authorization.
- Consider adding an upper bound on `data_.amount` based on current available borrowing power and internal policy to avoid accidental over-borrow when invoked by trusted components.

If changing the interface is acceptable, tighten the surface further:

- Expose only a single orchestrated entrypoint on the orchestrator (vault) and make fuse functions non-callable from EOAs (e.g., internalizing logic or adding explicit `onlyOrchestrator` modifiers in all public/external fuse functions).

These changes ensure that only the intended privileged flow can create debt and that arbitrary EOAs cannot borrow on behalf of the contract.

Go back to Findings Summary

H1: Negative net market value makes balanceOf revert via SafeCast.toUint256, causing delegatecalled balance updates to fail (DoS)

High impact finding

Impact:	High	Confidence:	High
Target:	./contracts/fuses/aave_v3/AaveV3WithPriceOracleMiddlewareBalanceFuse.sol	Type:	Denial of service

Description

The fuse accumulates per-asset balances as signed integers and finally casts the signed USD accumulator to an unsigned integer using `SafeCast.toUint256`. When liabilities exceed assets for the configured substrates, the accumulator becomes negative and the cast reverts. Because `balanceOf` is executed via `delegatecall` during vault balance updates, this revert blocks balance refresh and downstream flows.

The accumulator is built as `int256` and returned with `toUint256`:

Listing 5. Code snippet from contracts/fuses/aave_v3/AaveV3WithPriceOracleMiddlewareBalanceFuse.sol

```
125 balanceTemp += IporMath.convertToWadInt(balanceInLoop * int256(price),
    decimals + priceDecimals);
126 }
127
128 return balanceTemp.toUint256();
```

Vault balance updates invoke this fuse by `delegatecall` and decode the returned `uint256`. Any revert in `balanceOf` bubbles up and aborts the update:

Listing 6. Code snippet from contracts/vaults/lib/PlasmaVaultMarketsLib.sol

```
44 wadBalanceAmountInUSD =
45
   abi.decode(balanceFuse.functionDelegateCall(abi.encodeWithSignature("balanceO
   f()")), (uint256));
```

Balance updates are performed after executions and on withdrawal paths, so a revert in `balanceOf` denies service to these operations:

Listing 7. Code snippet from contracts/vaults/PlasmaVault.sol

```
356 _updateMarketsBalances(markets);
357
358 _addPerformanceFee(totalAssetsBefore);
```

Listing 8. Code snippet from contracts/vaults/PlasmaVault.sol

```
1355 DataToCheck memory dataToCheck =
1356     PlasmaVaultMarketsLib.updateMarketsBalances(markets_, asset(),
     decimals(), _decimalsOffset());
```

In practice, negative net market value is realistic because the system includes borrow fuses for Aave V3, making debt tokens part of the balance calculation. When the signed sum is below zero, `SafeCast.toUint256` reverts, halting the vault's balance update pipeline.

Exploit Scenario

A realistic denial-of-service scenario follows.

Consider the following attack scenario:

1. Alice performs actions through supported fuses to accumulate Aave V3 debt on the vault so that the net value across the configured substrates becomes negative;

2. Alice (or any operator) triggers an action that updates balances (for example, an `execute` call that touches the market, or a withdrawal path);
3. The vault calls `PlasmaVaultMarketsLib.updateMarketsBalances`, which `delegatecall`s `balanceOf` on this fuse; and
4. `balanceOf` attempts `balanceTemp.toUint256()` and reverts because the signed accumulator is negative, causing the entire vault operation to revert and blocking further progress until the position is restored to non-negative.

Recommendation

Return a non-reverting value when the signed accumulator is negative, or propagate a signed value upstream so callers can handle negativity explicitly.

Actionable options:

- Implement saturating behavior in this fuse: if `balanceTemp <= 0`, return `0`; otherwise, return `uint256(balanceTemp)`.
- Alternatively, change the return type to `int256` and update callers to accept signed balances and clamp or handle negativity at the aggregation site.
- Add invariant checks or operational guardrails so that the configured substrates for a market cannot produce a negative net value if unsigned semantics are required.

The least invasive fix is the first option (saturating to zero), which aligns with common balance fuses that avoid reverting on negative net value while preserving unsigned return type expectations.

Go back to Findings Summary

H2: Whitelist bypass in redeem allows non-whitelisted owner/receiver via allowance proxy

High impact finding

Impact:	High	Confidence:	High
Target:	contracts/vaults/extensions/WhitelistWrappedPlasmaVault.sol	Type:	Access control

Description

WhitelistWrappedPlasmaVault.redeem applies `onlyRole(WHITELISTED)` to the caller only and forwards the call to the base implementation. It does not verify that `owner_` and `receiver_` are whitelisted.

The base `redeem` supports the ERC20 allowance path when `msg.sender != owner_` and transfers underlying assets to the arbitrary `receiver_` address with no whitelist validation.

Listing 9. Code snippet from contracts/vaults/extensions/WhitelistWrappedPlasmaVault.sol

```
134 function redeem(uint256 shares_, address receiver_, address owner_)
135     public
136     override
137     onlyRole(WHITELISTED)
138     returns (uint256)
139 {
140     return super.redeem(shares_, receiver_, owner_);
141 }
```

Listing 10. Code snippet from contracts/vaults/extensions/WrappedPlasmaVaultBase.sol

```
504 function redeem(uint256 shares_, address receiver_, address owner_)
505     public
```

```
506     virtual
507     override
508     nonReentrant
509     returns (uint256)
510 {
511     if (shares_ == 0) revert ZeroSharesMint();
512     if (receiver_ == address(0)) revert ZeroReceiverAddress();
513 
514     if (msg.sender != owner_) {
515         _spendAllowance(owner_, msg.sender, shares_);
516     }
517 
518     _calculateFees();
519     uint256 assets = previewRedeem(shares_);
520     if (assets == 0) revert ZeroAssetsWithdraw();
521 
522     _burn(owner_, shares_);
523 
524     ERC4626Upgradeable(PLASMA_VAULT).withdraw(assets, receiver_, address
    (this));
525 
526     lastTotalAssets = totalAssets();
527 
528     emit Withdraw(msg.sender, receiver_, owner_, assets, shares_);
529     return assets;
530 }
```

Because shares are transferrable and approvals are unrestricted, a non-whitelisted holder can delegate redemption to a whitelisted proxy and withdraw assets to a non-whitelisted `receiver_`, bypassing whitelist constraints.

Exploit Scenario

A whitelisted helper can redeem on behalf of a non-whitelisted owner and send assets to a non-whitelisted receiver.

Consider the following attack path:

1. Alice (not whitelisted) acquires wrapper shares via transfer from a whitelisted account.

2. Alice approves Helper (a whitelisted address) to spend her shares on the wrapper token.
3. Helper calls `redeem(shares, receiver_=Alice, owner_=Alice)` on the wrapper.
4. The base implementation spends Alice's allowance, burns her shares, and withdraws underlying to `receiver_` (Alice), who is not whitelisted.
5. Alice exits to underlying despite the whitelist.

Recommendation

Validate whitelist membership for the actual owner and final recipient in `redeem`.

Fix the vulnerability by either:

- Requiring `hasRole(WHITELISTED, owner_)` and `hasRole(WHITELISTED, receiver_)` in the override before forwarding; or
- Requiring `owner_ == msg.sender` and validating `receiver_` is whitelisted (no proxy redemptions).

For comprehensive enforcement, also restrict ERC20 share transfers/approvals so non-whitelisted addresses cannot hold shares or grant allowances.

Go back to Findings Summary

H3: Signed extraction clamps negatives to zero, silently losing sign across mappings

High impact finding

Impact:	High	Confidence:	High
Target:	contracts/fuses/transient_storage/TransientStorageMapperFuse.sol	Type:	Logic error

Description

Signed `INT*` inputs are coerced to `0` during extraction. For every signed variant, `_extractNumericValue` casts the `bytes32` value to the corresponding signed width and then returns `0` when the value is negative. This destroys the sign bit and makes any negative input indistinguishable from an actual `0` after conversion.

This behavior breaks signed semantics and can bypass downstream logic that relies on negative values to represent obligations, deltas, or error/sentinel states. Because `enter` always routes through `_convertValue`, the sign is irreversibly lost for signed mappings before any decimal scaling or repacking is performed.

Affected code path:

- Function `enter` reads a `bytes32` value from transient storage;
- Function `_convertValue` delegates to `_extractNumericValue` for the source `fromType_`; and
- Function `_extractNumericValue` clamps all negative signed values to `0`.

Listing 11. Code snippet from contracts/fuses/transient_storage/TransientStorageMapperFuse.sol

```
129         } else if (dataType_ == DataType.INT256) {
130             int256 signedValue = int256(uint256(value_));
131             return signedValue >= 0 ? uint256(signedValue) : 0;
132         } else if (dataType_ == DataType.INT128) {
133             int128 signedValue = int128(int256(uint256(value_)));
134             return signedValue >= 0 ? uint256(int256(signedValue)) : 0;
135         } else if (dataType_ == DataType.INT64) {
136             int64 signedValue = int64(int256(uint256(value_)));
137             return signedValue >= 0 ? uint256(int256(signedValue)) : 0;
138         } else if (dataType_ == DataType.INT32) {
139             int32 signedValue = int32(int256(uint256(value_)));
140             return signedValue >= 0 ? uint256(int256(signedValue)) : 0;
141         } else if (dataType_ == DataType.INT16) {
142             int16 signedValue = int16(int256(uint256(value_)));
143             return signedValue >= 0 ? uint256(int256(signedValue)) : 0;
144         } else if (dataType_ == DataType.INT8) {
145             int8 signedValue = int8(int256(uint256(value_)));
146             return signedValue >= 0 ? uint256(int256(signedValue)) : 0;
147         }
```

Exploit Scenario

A malicious integrator can make negative values disappear, turning them into zero and bypassing checks in the next fuse.

Consider the following attack scenario:

1. Alice controls a preceding fuse that writes a signed delta (e.g., PnL or debt change) to transient storage as `bytes32(int256(-5))` and declares the type as `INT256`.
2. Alice configures a mapping item so that `enter` reads this value and maps `INT256 -> INT256` with the same decimals.
3. During `enter`, `_convertValue` calls `_extractNumericValue`, which returns `0` for the negative input. Decimal conversion is skipped because decimals match, and `_packNumericValue` re-encodes `0`.

4. Bob's subsequent fuse reads the mapped input as an integer and treats it as "no debt/no negative delta," allowing an otherwise blocked operation (for example, withdrawal without settling liabilities), resulting in protocol loss.

Recommendation

Preserve or explicitly reject negative values instead of silently zeroing them.

Fix the vulnerability by either:

- Supporting signed conversions properly: perform extraction into `int256`, execute decimal scaling using signed arithmetic (scale absolute value then reapply sign), and repack using a signed-aware branch so that negative values remain negative; or
- If negative values are not valid in this pipeline, revert on negative signed inputs with a dedicated error (for example, `TransientStorageMapperFuseUnsupportedNegativeValue()`), and document the precondition.

Additionally:

- Add invariant tests: mapping `INT* -> INT*` with identical decimals must be bit-preserving for all in-range values, including negatives, or must revert by design.
- Include fuzz tests that cover negative domains for all signed widths.

Go back to Findings Summary

H4: Mispriced PT when using getPtToSyRate: missing SY.exchangeRate factor in USD conversion

High impact finding

Impact:	High	Confidence:	High
Target:	./contracts/price_oracle/price_feed/PtPriceFeed.sol	Type:	Logic error

Description

When `USE_PENDLE_ORACLE_METHOD` equals `0`, function `latestRoundData` obtains `unitPrice` via `PendlePYOracleLib.getPtToSyRate`. That rate is denominated in SY per PT (1e18 scale). The code then multiplies `unitPrice` directly by the middleware `assetPrice` (USD per asset), but never converts SY to asset using `IStandardizedYield.exchangeRate`.

As a result, the reported PT price is off by a factor of approximately `1e18 / exchangeRate`. For interest-bearing SY tokens, `exchangeRate` typically grows above `1e18`, so PT becomes systematically underpriced. This can cause incorrect collateral valuation, premature liquidations, and mispriced trades.

Listing 12. Code snippet from contracts/price_oracle/price_feed/PtPriceFeed.sol

```
109         uint256 unitPrice;
110         if (USE_PENDLE_ORACLE_METHOD == 1) {
111             unitPrice =
    PendlePYOracleLib.getPtToAssetRate(IPMarket(PENDLE_MARKET), twapWindow);
112         } else {
113             unitPrice =
    PendlePYOracleLib.getPtToSyRate(IPMarket(PENDLE_MARKET), twapWindow);
114         }
115
```

```
116         (uint256 assetPrice, uint256 priceDecimals) =
117
    IPriceOracleMiddleware(PRICE_MIDDLEWARE).getAssetPrice(ASSET_ADDRESS);
118
119         uint256 scalingFactor = FEED_DECIMALS + priceDecimals - _decimals();
120         price = SafeCast.toInt256((unitPrice * assetPrice) / 10 **
    scalingFactor);
```

`getPtToSyRate` normalizes the PT->asset rate by dividing through an index, returning PT in SY units. Without multiplying back by `SY.exchangeRate`, the conversion to USD omits the SY->asset dimension.

Listing 13. Code snippet from node_modules/@pendle/core-v2/contracts/oracles/PendlePYOracleLib.sol

```
43     /// @notice Similar to getPtToAsset but returns the rate in SY instead
44     function getPtToSyRate(IPMarket market, uint32 duration) internal view
   returns (uint256) {
45         (uint256 syIndex, uint256 pyIndex) = getSYandPYIndexCurrent(market);
46         if (syIndex >= pyIndex) {
47             return getPtToAssetRateRaw(market, duration).divDown(syIndex);
48         } else {
49             return getPtToAssetRateRaw(market, duration).divDown(pyIndex);
50         }
51     }
```

The meaning of `SY.exchangeRate` (asset per SY) is specified in the SY interface:

Listing 14. Code snippet from node_modules/@pendle/core-v2/contracts/interfaces/IStandardizedYield.sol

```
96      * @notice exchangeRate * syBalance / 1e18 must return the asset balance
   of the account
97      * @notice vice-versa, if a user uses some amount of tokens equivalent to
   X asset, the amount of sy
98      he can mint must be X * exchangeRate / 1e18
99
100      * @dev SYUtils's assetToSy & syToAsset should be used instead of raw
   multiplication
101      & division
102
103      */
```

```
104     function exchangeRate() external view returns (uint256 res);
```

Dimensional analysis:

- `unitPrice` from `getPtToSyRate`: SY per PT (1e18 scale);
- `assetPrice` from middleware: USD per asset (with `priceDecimals`);
- Computation in the contract: `(SY/PT) * (USD/asset)`; this leaves an extra `SY/asset` factor. The missing factor is `exchangeRate/1e18` (asset per SY), so the reported price equals the true PT price multiplied by `1e18 / exchangeRate`.

A concrete example: if the true PT-to-asset rate is `0.95e18` and `SY.exchangeRate = 1.2e18`, then `getPtToSyRate` returns about `0.95e18 / 1.2e18`. Multiplying by a $1 asset price yields roughly $0.79 instead of the correct $0.95, underpricing PT by ~16.7%.

Exploit Scenario

A mispriced feed enables profitable liquidations and misquotations wherever PT is used.

Consider the following attack scenario:

1. Alice observes that for the deployed market the feed uses method `0` and that `SY.exchangeRate` is 1.20e18, causing PT to be underpriced by ~16.7%;
2. Alice scans Bob's lending market where PT is accepted as collateral and identifies healthy accounts that now appear undercollateralized under the mispriced feed;
3. Alice calls the protocol's liquidation function to liquidate those positions and captures the liquidation incentive and collateral at a discount; and
4. Bob's protocol incurs unnecessary liquidations and user losses due to the systematically depressed PT valuation in its risk engine.

Recommendation

Prefer computing PT in asset units, not SY units, when converting to USD.

Fix the vulnerability by either:

- Set `USE_PENDLE_ORACLE_METHOD` to `1` and always use `PendlePYOracleLib.getPtToAssetRate` (already 1e18-scaled in asset units); or
- If `getPtToSyRate` must be used, convert SY to asset before applying the middleware USD price:
- Read `SY.exchangeRate()` from the market's SY contract;
- Compute `ptAsset = unitPrice * exchangeRate / 1e18`;
- Then compute `priceUsd = (ptAsset * assetPrice) / 10**(18 + priceDecimals - _decimals())`.

Additionally, add tests that assert both methods produce numerically consistent results within tolerance, and reject deployments configured with method `0` unless the SY exchange rate is provably `1e18`.

Go back to Findings Summary

H5: Double inversion of slippageReverse_ in constructor severely weakens slippage protection

High impact finding

Impact:	High	Confidence:	High
Target:	./contracts/fuses/universal_token_swapper/UniversalTokenSwapperWithVerificationFuse.sol	Type:	Logic error

Description

The constructor misinterprets the input `slippageReverse_`, which the code documents as the reverse slippage tolerance expressed in WAD (that is, `1e18 - slippage`). Instead of storing that threshold as-is, the constructor computes `SLIPPAGE_REVERSE = 1e18 - slippageReverse_`, applying a second inversion.

This turns a typical configuration such as `slippageReverse_ = 0.97e18` (meaning a minimum acceptable output/input USD ratio of 97%) into `SLIPPAGE_REVERSE = 0.03e18` (3%). As a result the subsequent slippage gate in `_executeSwap` will permit swaps that return as little as 3% of the input value, drastically weakening the intended protection.

The threshold is used later as the minimum acceptable USD-denominated output/input ratio:

Listing 15. Code snippet from contracts/fuses/universal_token_swapper/UniversalTokenSwapperWithVerificationFuse.sol

```
98     /// @dev slippageReverse in WAD decimals, 1e18 - slippage;
```

```
99      /// @notice The reverse slippage tolerance (1e18 - slippage)
100     uint256 public immutable SLIPPAGE_REVERSE;
101
102     /// @notice Constructor to initialize the contract
103     /// @param marketId_ The market ID
104     /// @param executor_ The address of the swap executor
105     /// @param slippageReverse_ The reverse slippage tolerance
106     constructor(uint256 marketId_, address executor_, uint256
    slippageReverse_) {
107         if (executor_ == address(0)) {
108             revert UniversalTokenSwapperFuseInvalidExecutorAddress();
109         }
110         VERSION = address(this);
111         MARKET_ID = marketId_;
112         EXECUTOR = payable(executor_);
113         if (slippageReverse_ > 1e18) {
114             revert UniversalTokenSwapperFuseSlippageFail();
115         }
116         SLIPPAGE_REVERSE = 1e18 - slippageReverse_;
117     }
```

Listing 16. Code snippet from contracts/fuses/universal_token_swapper/UniversalTokenSwapperWithVerificationFuse.sol

```
191         (uint256 tokenInPrice, uint256 tokenInPriceDecimals) =
192
    IPriceOracleMiddleware(priceOracleMiddleware).getAssetPrice(swapData.tokenIn
    );
193         (uint256 tokenOutPrice, uint256 tokenOutPriceDecimals) =
194
    IPriceOracleMiddleware(priceOracleMiddleware).getAssetPrice(swapData.tokenOu
    t);
195
196         uint256 amountUsdInDelta = IporMath.convertToWad(
197             result.tokenInDelta * tokenInPrice,
    IERC20Metadata(swapData.tokenIn).decimals() + tokenInPriceDecimals
198         );
199         uint256 amountUsdOutDelta = IporMath.convertToWad(
200             result.tokenOutDelta * tokenOutPrice,
    IERC20Metadata(swapData.tokenOut).decimals() + tokenOutPriceDecimals
201         );
202
203         uint256 quotient = IporMath.division(amountUsdOutDelta * 1e18,
```

```
    amountUsdInDelta);

        if (quotient < SLIPPAGE_REVERSE) {
            revert UniversalTokenSwapperFuseSlippageFail();
        }
```

Given the above, a misconfigured threshold is a direct logic bug rather than a parameterization issue. There are no compensating checks: `_checkSubstrates` only validates targets and function selectors, while the price-based safety net depends entirely on `SLIPPAGE_REVERSE`.

Exploit Scenario

An attacker can cause the vault to execute swaps at catastrophically poor rates without triggering the slippage guard.

Consider the following attack scenario:

1. Bob deploys the fuse and configures `slippageReverse_ = 0.97e18`, expecting a minimum 97% output/input ratio.
2. Alice observes a pending `enter` call that will swap a large `amountIn` using a whitelisted DEX/aggregator target.
3. Alice temporarily moves the market price against the trade (for example by a large pre-trade or sandwiching on the same pool), so the swap would only return about 5% of the input value.
4. The contract computes `quotient = amountUsdOutDelta / amountUsdInDelta` scaled by `1e18`. With the current bug, `SLIPPAGE_REVERSE = 0.03e18`, so `0.05e18 >= 0.03e18` and the transaction does not revert.
5. Bob's vault completes the swap and realizes a loss of approximately 95% of the swapped value.

Recommendation

Assign the stored threshold exactly as documented and use it directly in the

slippage check.

Fix the vulnerability by either:

- Treating `slippageReverse_` as the already inverted threshold and setting `SLIPPAGE_REVERSE = slippageReverse_` in the constructor;
- Or, if the intent was to pass direct slippage, rename the parameter to `slippage_`, document it accordingly, and compute `SLIPPAGE_REVERSE = 1e18 - slippage_`.

Additionally:

- Keep the `slippageReverse_ <= 1e18` validation and consider enforcing a reasonable lower bound according to your risk appetite;
- Add unit tests that assert a configuration like `slippageReverse_ = 0.97e18` reverts when `quotient < 0.97e18` and succeeds when `quotient >= 0.97e18`;
- Review any existing deployments and re-deploy or reconfigure the fuse to apply the corrected threshold.

Go back to Findings Summary

H6: Unauthenticated redemption-lock updates enable third-party account locking via public manager

High impact finding

Impact:	High	Confidence:	High
Target:	./contracts/managers/access/RedemptionDelayLib.sol	Type:	Griefing

Description

Unauthenticated redemption-lock updates allow any third party to impose a time lock on arbitrary accounts by calling the public manager entry point with crafted parameters. The library `lockChecks` writes to the global redemption-lock mapping for the supplied `account_` whenever the selector matches a deposit-like operation, but it does not authenticate that `account_` corresponds to the real actor of the transaction. The manager `canCallAndUpdate` exposes this write externally without restricting who may call it.

Affected components and state:

- Function `RedemptionDelayLib lockChecks` writes `redemptionLock[account_]` on deposit-like selectors and reverts withdraw/redeem/transfer when the lock is active;
- Function `IporFusionAccessManager canCallAndUpdate` is `external` and calls `lockChecks(caller_, selector_)` before delegating to `AccessManager.canCall`; and
- Mapping `RedemptionLocks.redemptionLock` stores the per-account unlock timestamp.

The write path is reachable by anyone because `canCallAndUpdate` accepts arbitrary `caller_` and `selector_` and lacks a binding to `msg.sender` or to the `target_` context. As a result, any EOA can grief-lock a victim by repeatedly setting a fresh lock on the victim's address.

Listing 17. Code snippet from contracts/managers/access/RedemptionDelayLib.sol

```
53     function lockChecks(address account_, bytes4 sig_) internal {
54         if (
55             sig_ == WITHDRAW_SELECTOR || sig_ == REDEEM_SELECTOR || sig_ ==
   TRANSFER_FROM_SELECTOR
56                 || sig_ == TRANSFER_SELECTOR
57         ) {
58             uint256 unlockTime =
   IporFusionAccessManagersStorageLib.getRedemptionLocks().redemptionLock[accoun
   t_];
59             if (unlockTime > block.timestamp) {
60                 revert AccountIsLocked(unlockTime);
61             }
62         } else if (sig_ == DEPOSIT_SELECTOR || sig_ == MINT_SELECTOR || sig_
   == DEPOSIT_WITH_PERMIT_SELECTOR) {
63             IporFusionAccessManagersStorageLib.setRedemptionLocks(account_);
64         }
65     }
```

Listing 18. Code snippet from contracts/managers/access/IporFusionAccessManager.sol

```
163     function canCallAndUpdate(address caller_, address target_, bytes4
    selector_)
164         external
165         override
166         returns (bool immediate, uint32 delay)
167     {
168         RedemptionDelayLib.lockChecks(caller_, selector_);
169         return super.canCall(caller_, target_, selector_);
170     }
171
172     /**
173
174      * @notice Updates whether a target contract is closed for operations
```

```
175      * @param target_ Target contract address
176      * @param closed_ New closed status
177      * @custom:access Restricted to GUARDIAN_ROLE
178      */
179
180     function updateTargetClosed(address target_, bool closed_) external
    override restricted {
181         _setTargetClosed(target_, closed_);
182     }
```

Exploit Scenario

A minimal-cost griefing attack locks a victim's account without their participation by abusing the public manager entry point.

Consider the following attack scenario:

1. Alice, a malicious actor, prepares the function selector for `PlasmaVault.deposit`;
2. Alice calls `IporFusionAccessManager.canCallAndUpdate(victim, anyTarget, PlasmaVault.deposit.selector)` directly;
3. The manager executes `RedemptionDelayLib.lockChecks(victim, PlasmaVault.deposit.selector)`, which writes `redemptionLock[victim] = block.timestamp + REDEMPTION_DELAY_IN_SECONDS`;
4. Bob later attempts to withdraw or redeem from a vault; and
5. Bob's operation reverts with `AccountIsLocked` because the lock check in `lockChecks` sees a future `unlockTime`.

By repeating the external call before the lock expires, Alice can keep Bob perpetually locked out of withdraw/redeem/transfer operations that are guarded by the access manager, creating a durable denial of service.

Recommendation

Bind redemption-lock updates to authenticated context and restrict who

may trigger them.

Fix the vulnerability by either:

- Restricting `canCallAndUpdate` so that only authorized vaults may call it and only for themselves; for example, require `msg.sender == target_` and that `target_` is an allow-listed vault address;
- Removing the `caller_` parameter from the public surface and deriving the affected account inside the vault, then invoking an internal/privileged manager method that cannot be called by arbitrary EOAs; or
- If keeping the current API, add a strong invariant in `canCallAndUpdate` that ties the supplied `caller_` to the actual call context (for example, by making this function callable only by the vault and having the vault pass the actor it decoded from its own calldata).

Additionally, consider scoping the redemption lock to the vault context to minimize blast radius, and ensure the library does not update state based solely on untrusted externally supplied addresses.

Go back to Findings Summary

H7: Deposit path bypasses deposit fee; previews and mint realize fee but deposit does not

High impact finding

Impact:	High	Confidence:	High
Target:	./contracts/vaults/PlasmaVault.sol	Type:	Logic error

Description

The codebase supports a deposit fee via `PlasmaVaultFeesLib.prepareForRealizeDepositFee`, and the fee is correctly realized in `mint` and reflected in previews. However, the `deposit` path delegates to `_deposit`, which never realizes the deposit fee, allowing users to bypass it entirely.

Evidence in implementation:

Listing 19. Code snippet from contracts/vaults/PlasmaVault.sol

```
1261 function _deposit(uint256 assets_, address receiver_) internal returns
     (uint256) {
1262     if (assets_ == 0) {
1263         revert NoAssetsToDeposit();
1264     }
1265     if (receiver_ == address(0)) {
1266         revert Errors.WrongAddress();
1267     }
1268
1269     _realizeManagementFee();
1270
1271     uint256 sharesGross = super.convertToShares(assets_);
1272     uint256 shares = super.deposit(assets_, receiver_);
1273
1274     if (shares == 0) {
1275         revert NoSharesToDeposit();
1276     }
1277
```

```
1278     address withdrawManager =
     PlasmaVaultStorageLib.getWithdrawManager().manager;
1279
1280     if (withdrawManager != address(0) && sharesGross > shares) {
1281         super._mint(withdrawManager, sharesGross - shares);
1282     }
1283     return shares;
1284 }
```

The path above only handles management fee and a rounding difference, minting any gap to the withdraw manager. It does not calculate or realize deposit fee shares.

By contrast, `mint` realizes the deposit fee and transfers fee shares from the receiver to the fee recipient:

Listing 20. Code snippet from contracts/vaults/PlasmaVault.sol

```
623 _realizeManagementFee();
624
625 (address feeRecipient, uint256 feeShares) =
    PlasmaVaultFeesLib.prepareForRealizeDepositFee(shares_);
626
627 depositAssets = super.mint(shares_ + feeShares, receiver_);
628
629 if (feeShares > 0) {
630     _transfer(receiver_, feeRecipient, feeShares);
631     emit DepositFeeRealized(feeRecipient, feeShares);
632 }
```

The previews also reflect the existence of a deposit fee:

Listing 21. Code snippet from contracts/vaults/PlasmaVault.sol

```
733 function previewDeposit(uint256 assets_) public view override returns
    (uint256) {
734     uint256 shares = super.previewDeposit(assets_);
735     (, uint256 feeShares) =
    PlasmaVaultFeesLib.prepareForRealizeDepositFee(shares);
736     return feeShares > 0 ? shares - feeShares : shares;
737 }
738
```

```
739 function previewMint(uint256 shares_) public view override returns (uint256)
    {
740     (, uint256 feeShares) =
    PlasmaVaultFeesLib.prepareForRealizeDepositFee(shares_);
741     return super.previewMint(shares_ + feeShares);
742 }
```

As a result, calling `deposit` avoids the configured deposit fee whereas `mint` charges it and previews assume it is charged. This creates a fee-avoidance vector and inconsistencies for integrators.

Exploit Scenario

A participant can systematically avoid the deposit fee by routing through `deposit` instead of `mint`.

Consider the following attack scenario:

1. Alice observes that a non-zero deposit fee is configured and previews show reduced shares for a given asset amount;
2. Alice calls `deposit` with 10,000 assets; `_deposit` mints her full `convertToShares(10,000)` shares and does not transfer any fee shares to the fee recipient;
3. Alice repeats deposits over time, consistently bypassing the fee while the UI and previews suggest a fee would be charged; and
4. Bob's protocol misses expected fee revenue and presents inconsistent behavior across entry points, undermining economics and integrator trust.

Recommendation

Realize the deposit fee in the `deposit` path to match `mint` and the preview functions.

Fix the vulnerability by either:

- Mirroring `mint` inside `_deposit`: compute `(feeRecipient, feeShares) = PlasmaVaultFeesLib.prepareForRealizeDepositFee(sharesGross)` using `sharesGross = super.convertToShares(assets_)`, then after minting `shares` to `receiver_`, transfer `feeShares` from `receiver_` to `feeRecipient` and emit `DepositFeeRealized` (guarded by `if (feeShares > 0)`); or
- Refactoring fee handling into a shared internal helper called from both `mint` and `_deposit` to ensure a single point of truth and consistent accounting/events.

Keep the rounding-gap mint to the withdraw manager intact, and add tests that assert parity between `deposit` and `mint` (including rounding boundary cases), as well as alignment with `previewDeposit` and `previewMint` outputs.

Go back to Findings Summary

H8: Unsafe cast of signed net balance to uint256 reverts on negative values, causing vault-wide DoS via delegatecall

High impact finding

Impact:	High	Confidence:	High
Target:	contracts/fuses/euler/EulerV2BalanceFuse.sol	Type:	Denial of service

Description

The `balanceOf` function accumulates a signed `int256` net value by adding collateral and subtracting debt per substrate. The result is returned by casting the signed accumulator to `uint256` via `SafeCast.toUint256`.

When the net is negative (for example when the Euler position is temporarily undercollateralized due to borrows, accrued interest, or adverse price moves), `SafeCast.toUint256` reverts. Because the fuse is invoked via `delegatecall` by the vault during market balance refresh, this revert propagates and blocks balance updates for the entire market, making deposit/withdraw/rebalance flows fail until the position becomes positive again.

The implementation clearly subtracts debt from collateral in WAD terms and then performs an unchecked cast to `uint256`.

Listing 22. Code snippet from contracts/fuses/euler/EulerV2BalanceFuse.sol

```
108             if (balanceOfSubAccountInEulerVault > 0) {
109                 balanceOfSubAccountInUnderlyingAsset =
110 
    ERC4626(substrate.eulerVault).convertToAssets(balanceOfSubAccountInEulerVaul
    t);
111                 balance += IporMath.convertToWad(
```

```
112                         balanceOfSubAccountInUnderlyingAsset * price,
    eulerVaultAssetDecimals + priceDecimals
113                     ).toInt256();
114             }
115
116             balance -= IporMath.convertToWad(
117                     IBorrowing(substrate.eulerVault).debtOf(subAccount) *
    price, eulerVaultAssetDecimals + priceDecimals
118                 ).toInt256();
119         }
120
121         /// @dev This value, considering collateral and debt, should never
    be negative
122         return balance.toUint256();
```

This fuse is executed during vault balance refresh via `delegatecall` and its return value is decoded as `uint256`. A revert in the fuse therefore reverts the entire refresh.

Listing 23. Code snippet from contracts/vaults/lib/PlasmaVaultMarketsLib.sol

```
44             wadBalanceAmountInUSD =
45
    abi.decode(balanceFuse.functionDelegateCall(abi.encodeWithSignature("balanceO
    f()")), (uint256));
```

Exploit Scenario

An attacker or operator can place the vault sub-account into a state where debt exceeds collateral so that any attempt to refresh market balances reverts and blocks user operations.

Consider the following attack scenario:

1. Alice uses the Euler fuse to borrow and/or withdraw collateral from the configured Euler vault(s), leaving the sub-account with net debt greater than collateral;

2. Alice (or any user) triggers a vault action that refreshes balances, for example by calling `PlasmaVault.updateMarketsBalances` for the affected market;
3. The vault calls into `PlasmaVaultMarketsLib.updateMarketsBalances`, which `delegatecall`s `EulerV2BalanceFuse.balanceOf`; the function computes a negative `balance` and reaches `SafeCast.toUint256` which reverts; and
4. Bob attempts to deposit or withdraw, but the vault refresh path reverts, preventing his operation and stalling rebalances until the position becomes positive again.

Recommendation

Return semantics must not rely on a signed value being non-negative if it is enforced only by assumption.

Recommended fixes:

- Make `balanceOf` saturate at zero before casting: `return balance <= 0 ? 0 : uint256(balance);`.
- Alternatively, change `balanceOf` to return `int256` and adjust all consumers (notably `PlasmaVaultMarketsLib.updateMarketsBalances`) to decode and handle negative values explicitly.
- Optionally, emit an event or record telemetry whenever a negative net balance is observed so operators can react, and update the inline comment "should never be negative" to reflect enforced behavior.

Either approach removes the revert condition and prevents denial of service during transient undercollateralization.

Go back to Findings Summary

H9: Whitelist bypass in withdraw allows non-whitelisted owner/receiver via allowance proxy

High impact finding

Impact:	High	Confidence:	High
Target:	contracts/vaults/extensions/WhitelistWrappedPlasmaVault.sol	Type:	Access control

Description

WhitelistWrappedPlasmaVault.withdraw gates only the caller with `onlyRole(WHITELISTED)` and forwards the call to the base implementation without validating that `owner_` and `receiver_` are whitelisted.

This enables a whitelisted proxy to withdraw on behalf of a non-whitelisted owner and route underlying assets to a non-whitelisted receiver via the ERC20 allowance path.

Listing 24. Code snippet from contracts/vaults/extensions/WhitelistWrappedPlasmaVault.sol

```
125 function withdraw(uint256 shares_, address receiver_, address owner_)
126     public
127     override
128     onlyRole(WHITELISTED)
129     returns (uint256)
130 {
131     return super.withdraw(shares_, receiver_, owner_);
132 }
```

The base implementation uses the allowance path when `msg.sender != owner_` and transfers assets to the provided `receiver_` without any whitelist checks:

Listing 25. Code snippet from contracts/vaults/extensions/WrappedPlasmaVaultBase.sol

```
402 function withdraw(uint256 assets_, address receiver_, address owner_)
403     public
404     virtual
405     override
406     nonReentrant
407     returns (uint256)
408 {
409     if (assets_ == 0) revert ZeroAssetsWithdraw();
410     if (receiver_ == address(0)) revert ZeroReceiverAddress();
411
412     _calculateFees();
413
414     uint256 shares = previewWithdraw(assets_);
415
416     if (msg.sender != owner_) {
417         _spendAllowance(owner_, msg.sender, shares);
418     }
419
420     _burn(owner_, shares);
421
422     ERC4626Upgradeable(PLASMA_VAULT).withdraw(assets_, receiver_, address
    (this));
423
424     emit Withdraw(msg.sender, receiver_, owner_, assets_, shares);
425
426     lastTotalAssets = totalAssets();
427
428     return shares;
429 }
```

Since the wrapper token inherits unrestricted ERC20 semantics, non-whitelisted addresses can hold shares and grant allowances. Combining this with the lack of whitelist checks on `owner_` and `receiver_` allows whitelist bypass of withdrawals.

Exploit Scenario

A whitelisted helper can withdraw underlying for a non-whitelisted holder and route assets to a non-whitelisted recipient.

Consider the following attack path:

1. Alice (not whitelisted) receives wrapper shares via transfer from a whitelisted account.
2. Alice approves Helper (a whitelisted address) to spend her shares on the wrapper token.
3. Helper calls `withdraw(amount, receiver_=Alice, owner_=Alice)` on the wrapper.
4. The base implementation spends Alice's allowance, burns Alice's shares, and sends underlying assets to `receiver_` (Alice), who is not whitelisted.
5. Alice exits to underlying despite the whitelist.

Recommendation

Enforce whitelist on effective participants, not only on the caller.

Fix the vulnerability by either:

- Validating both `owner_` and `receiver_` have the `WHITELISTED` role in `withdraw` before forwarding to the base implementation; or
- Requiring `owner_ == msg.sender` and also validating `receiver_` is whitelisted (disallow proxy withdrawals).

If the intent is to prevent non-whitelisted accounts from holding shares at all, additionally restrict ERC20 transfers and approvals by overriding token transfer hooks to require both `from` and `to` to be whitelisted and to block approvals to non-whitelisted spenders.

Go back to Findings Summary

H10: latestRoundData returns current block timestamp instead of underlying data timestamp, undermining freshness checks

High impact finding

Impact:	High	Confidence:	High
Target:	./contracts/price_oracle/price_feed/PtPriceFeed.sol	Type:	Standards violation

Description

Function `latestRoundData` returns `time = block.timestamp` unconditionally instead of the timestamp of the data used to compute the price.

This violates the documented semantics of `IPriceFeed.latestRoundData`, where `time` is the timestamp of the data of the round. In this implementation, no upstream timestamps (from the Pendle TWAP observation window or the middleware asset price) are propagated, so stale upstream sources appear fresh to consumers that perform staleness checks like `require(block.timestamp - time <= maxAge)`.

The lack of timestamp propagation can cause protocols to accept stale prices as fresh when an upstream feed is paused, delayed, or otherwise out of date.

Listing 26. Code snippet from contracts/price_oracle/price_feed/PtPriceFeed.sol

```
126         time = block.timestamp;
```

The interface clarifies that `time` is expected to represent the data time:

Listing 27. Code snippet from contracts/price_oracle/price_feed/IPriceFeed.sol

```
11      /// @notice Returns the latest price of an asset, expressed in USD
12      /// @return roundId The round ID from which the data was retrieved
13      /// @return price The latest price of the asset, expressed in USD, with 8
    decimals
14      /// @return startedAt Timestamp of the start of the round
15      /// @return time Timestamp of the data of the round
16      /// @return answeredInRound The round ID from which the answer was
    retrieved
17      /// @dev Notice! The price is expressed always in 8 decimals.
18      function latestRoundData()
19          external
20          view
21          returns (uint80 roundId, int256 price, uint256 startedAt, uint256
    time, uint80 answeredInRound);
```

Exploit Scenario

A stale upstream price can be made to pass freshness checks, enabling outdated valuations.

Consider the following attack scenario:

1. Alice notices the underlying asset price source in the middleware has been stuck at $1.00 for several hours while the true market price dropped to $0.60;
2. Alice supplies PT as collateral to Bob's lending protocol; the feed returns `time = block.timestamp`, so Bob's staleness guard passes and treats the price as fresh;
3. Alice borrows against the artificially high collateral valuation anchored to the stale $1.00 price; and
4. Bob's protocol accrues bad debt when the true price is much lower, as positions become immediately undercollateralized once any honest price is used.

Recommendation

Propagate the true data timestamp and enforce freshness.

Fix the vulnerability by either:

- Extend `IPriceOracleMiddleware` with a method that returns price and `updatedAt` (e.g., `getAssetPriceWithTimestamp`), then set `time` to the minimum of the middleware `updatedAt` and the end of the Pendle TWAP window (which is `block.timestamp`); or
- If extending the middleware is not feasible, add an explicit freshness check against a configurable `maxAge` read from the middleware or set per-feed, and revert when the middleware source is stale.

For Chainlink-like semantics, also set `startedAt` to the start of the measurement window (e.g., `time - TWAP_WINDOW`) and ensure `time` tracks the timestamp of the observations actually used.

Go back to Findings Summary

H11: Missing early-return in _convertValue forces a lossy round-trip even when types and decimals are identical

High impact finding

Impact:	High	Confidence:	High
Target:	contracts/fuses/transient_storage/TransientStorageMapperFuse.sol	Type:	Logic error

Description

`_convertValue` is documented to be a no-op when the source and destination data types and decimals are identical. The implementation, however, only early-returns when either side is `UNKNOWN` and otherwise always performs extraction and repacking. For types whose conversion is not bit-preserving, this forced round-trip can change the data even when no conversion is intended.

This creates two distinct problems:

- Violated API contract: callers expecting identity behavior for "same type, same decimals" will get altered values.
- Unnecessary and potentially lossy conversion: the pipeline goes through `_extractNumericValue` and `_packNumericValue`, which can normalize, narrow, or otherwise change the representation.

Listing 28. Code snippet from contracts/fuses/transient_storage/TransientStorageMapperFuse.sol

```
92         // If types are UNKNOWN or the same with same decimals, return value
   unchanged
93         if (fromType_ == DataType.UNKNOWN || toType_ == DataType.UNKNOWN) {
94             return value_;
```

```
95          }
96
97          // Extract numeric value based on fromType
98          uint256 numericValue = _extractNumericValue(value_, fromType_);
```

Consequence: even when `fromType_ == toType_` and `fromDecimals_ == toDecimals_`, the function does not return early and instead rounds the value through the conversion pipeline, which is not guaranteed to be identity for all types.

Exploit Scenario

A caller can rely on the documented "no-op" behavior to pass values that should remain unchanged, but the forced conversion alters them, bypassing checks that would have triggered on the original representation.

Consider the following attack scenario:

1. Alice prepares a mapping item with `fromType_ = INT256`, `toType_ = INT256`, and identical decimals, and supplies a negative value encoded in `bytes32`.
2. Alice expects an identity mapping per the documentation and config comments, but `_convertValue` does not early-return and instead extracts and repacks the value.
3. The conversion pipeline treats the negative input as a special case and produces a non-identical output (`0`), changing the semantics from "negative" to "zero."
4. Bob's subsequent fuse evaluates the mapped input and finds no negative condition to guard against, enabling an operation that should have been rejected, with financial impact.

Recommendation

Honor the documented identity behavior and avoid unnecessary conversions.

Fix the vulnerability by either:

- Add an explicit early return at the top of `_convertValue` when `fromType_ == toType_ && fromDecimals_ == toDecimals_`, returning `value_` unchanged; or
- Introduce a fast path per data family (unsigned, signed, address, bool, bytes32) that performs a cheap consistency check and only falls back to extraction/packing if a real type/decimal change is required.

Additionally:

- Add unit tests asserting that same-type/same-decimal mappings are bitwise identical for representative values of each type family.
- Consider a configuration-level validation that rejects no-op mappings to reduce surface area.

Go back to Findings Summary

H12: Permissionless rEUL claim enables forced remainder diversion via allowRemainderLoss

High impact finding

Impact:	High	Confidence:	High
Target:	./contracts/rewards_fuses/euler/RewardEulerTokenClaimFuse.sol	Type:	Access control

Description

The `claim` function is externally callable and forwards the untrusted `ClaimData.allowRemainderLoss` flag and caller-supplied `lockTimestamps` directly to `IREUL.withdrawToByLockTimestamps`.

Per the `IREUL` interface, when `allowRemainderLoss` is true and the calculated `remainderAmount` is non-zero for the chosen normalized lock timestamps, that remainder is transferred to the rEUL-configured `receiver` address. When false and any remainder exists, the withdrawal reverts. Because any account can call `claim` and choose both the flag and the timestamps, a malicious caller can intentionally select timestamp combinations that maximize remainders and force value to be sent to an external receiver the vault does not control.

This constitutes a permissionless value-diversion/griefing vector against the vault's `rewardsClaimManager`: the call sends the EUL proceeds to `rewardsClaimManager` as intended, but any non-claimable remainder is irreversibly diverted away from the protocol to the rEUL `receiver`.

Relevant code in the fuse that exposes the untrusted flag and timestamps:

Listing 29. Code snippet from contracts/rewards_fuses/euler/RewardEulerTokenClaimFuse.sol

```
52 function claim(ClaimData calldata data_) external {
53     address rewardsClaimManager =
   PlasmaVaultLib.getRewardsClaimManagerAddress();
54     if (rewardsClaimManager == address(0)) {
55         revert RewardEulerTokenClaimFuseRewardsClaimManagerNotSet();
56     }
57
58     uint256 eulerRewardsManagerBalanceBefore =
   IERC20(EUL).balanceOf(rewardsClaimManager);
59
60     IREUL(rEUL).withdrawToByLockTimestamps(rewardsClaimManager,
   data_.lockTimestamps, data_.allowRemainderLoss);
```

The semantics that enable diversion are defined in the external interface:

Listing 30. Code snippet from contracts/rewards_fuses/euler/ext/IREUL.sol

```
 6 /// @notice Withdraws tokens to a specified account based on multiple
   normalized lock timestamps as per the lock
 7 /// schedule.
 8 /// The remainder of the tokens are transferred to the receiver address
   configured.
 9 /// @param account_ The address to receive the withdrawn tokens
10 /// @param lockTimestamps_ An array of normalized lock timestamps to withdraw
   tokens for
11 /// @param allowRemainderLoss_ If true, is it allowed for the remainder of
   the tokens to be transferred to the
12 /// receiver address configured as per the lock schedule. If false and the
   calculated remainder amount is non-zero,
13 /// the withdrawal will revert.
14 /// @return bool indicating success of the withdrawal
15 function withdrawToByLockTimestamps(address account_, uint256[] memory
   lockTimestamps_, bool allowRemainderLoss_)
16     external
```

Observed properties in the fuse:

- No access control on `claim` (it is `external`).

- The contract uses `data.lockTimestamps` and `data.allowRemainderLoss` exactly as provided by the caller.
- The remainder is sent to a `receiver` configured in `rEUL`, not to the vault.

Impact in production: any address can front-run or grief legitimate claims by setting `allowRemainderLoss = true` and crafting `lockTimestamps` that yield non-zero remainders, permanently reducing the vault's claimable EUL and leaking value to an external receiver.

Exploit Scenario

An untrusted caller can force remainders to be paid to an external receiver by selecting timestamps and enabling the loss flag.

Consider the following attack scenario:

1. Alice observes that the vault has rEUL positions locked under the `rewardsClaimManager` address.
2. Alice queries `IREUL.getLockedAmountsLockTimestamps(rewardsClaimManager)` on-chain to learn the available normalized lock timestamps and estimates remainders with `getWithdrawAmountsByLockTimestamp`.
3. Alice submits a transaction calling `claim` with `allowRemainderLoss = true` and a set of `lockTimestamps` that maximizes non-claimable `remainderAmount`.
4. The fuse forwards these values to `IREUL.withdrawToByLockTimestamps`.
5. EUL proceeds are sent to `rewardsClaimManager`, while the non-claimable remainder is transferred by rEUL to its configured `receiver` address.
6. Bob, who operates the vault, later tries to claim but finds the remaining rewards reduced; the diverted remainder cannot be recovered.

A motivated attacker can repeat the above any time new rEUL accrues, griefing the protocol by continuously siphoning remainders to the external receiver.

Recommendation

Harden the claim path so untrusted callers cannot authorize value to be sent to third-party receivers.

Recommended options:

- Restrict `claim` to an authorized role (for example, `onlyRewardsManager`, `onlyOwner`, or a dedicated operator EOA/automation bot).
- Do not trust caller-provided remainders policy: ignore `data.allowRemainderLoss` and always pass `false` to `withdrawToByLockTimestamps`. Allow the call to revert if a remainder would be created.
- Do not trust caller-provided timestamps: fetch `lockTimestamps` on-chain inside the function for `rewardsClaimManager`, and/or pre-validate with `getWithdrawAmountsByLockTimestamp` to ensure the combination yields zero remainder before attempting the withdrawal.
- If remainder losses must be allowed for operational reasons, ensure the rEUL `receiver` is a vault-controlled address and document/account for the loss in protocol accounting.

Additionally, emit sufficient events to make diversion attempts observable, and document the policy around remainders in operator runbooks.

Go back to Findings Summary

H13: enter is permissionless: anyone can execute EVC batch on fuse-owned Euler subaccounts

High impact finding

Impact:	High	Confidence:	High
Target:	./contracts/fuses/euler/EulerV2BatchFuse.sol	Type:	Access control

Description

The `enter` function is publicly callable and lacks any access control, yet it builds an `IEVC.BatchItem[]` and invokes `EVC.batch` on behalf of Euler subaccounts derived from this contract's address. For any batch item whose `targetContract` is not the EVC itself, the code sets `onBehalfOfAccount` to `EulerFuseLib.generateSubAccountAddress(address(this), subAccountId)`. Because this contract is the direct caller of `EVC.batch`, the EVC authenticates the fuse contract as the owner of those derived subaccounts and executes privileged actions (borrow, repay, deposit, withdraw, enable/disable controller) for the contract-owned accounts.

Validation performed in `_validate` only checks shapes and market permissions; it does not restrict who may initiate the mutation. As a result, any external address can mutate the protocol-owned EVC accounts by calling `enter` with a batch that passes validation, including initiating borrows that create debt on the fuse's subaccounts and enabling controllers.

Listing 31. Code snippet from contracts/fuses/euler/EulerV2BatchFuse.sol

```
60     function enter(EulerV2BatchFuseData memory data_)
61         public
62         returns (uint256 batchSize, address[] memory assets, address[] memory
   vaults)
63     {
```

```
64          IEVC.BatchItem[] memory batchItems = new
    IEVC.BatchItem[](data_.batchItems.length);
65
66          _validate(data_);
67
68          for (uint256 i = 0; i < data_.assetsForApprovals.length; i++) {
69
    ERC20(data_.assetsForApprovals[i]).forceApprove(data_.eulerVaultsForApprovals
    [i], type(uint256).max);
70          }
71
72          // Cache frequently used values for gas optimization
73          address evcAddress = address(EVC);
74          address thisAddress = address(this);
75
76          for (uint256 i = 0; i < data_.batchItems.length; i++) {
77              EulerV2BatchItem memory item = data_.batchItems[i];
78              batchItems[i] = IEVC.BatchItem(
79                  item.targetContract,
80                  item.targetContract != evcAddress
81                      ? EulerFuseLib.generateSubAccountAddress(thisAddress,
    item.onBehalfOfAccount)
82                      : address(0),
83                  0,
84                  item.data
85              );
86          }
87
88          EVC.batch(batchItems);
```

Exploit Scenario

Any external address can execute privileged operations against the fuse-owned Euler subaccounts.

Consider the following attack scenario:

1. Alice observes the list of permitted Euler vaults and subaccounts for this fuse in on-chain configuration;
2. Alice crafts a batch with a `borrow` call to a permitted vault, setting `receiver` to `EulerFuseLib.generateSubAccountAddress(address(this), subId)` and

choosing an allowed `subId`;

3. Alice calls `EulerV2BatchFuse.enter(data)` with her crafted batch;
4. The EVC authenticates the fuse contract as owner of the derived subaccount and executes the borrow; tokens are transferred to the derived subaccount address while the debt is assigned to that account, creating unwanted liability for the protocol-managed position.

Recommendation

Introduce strict access control around batch execution.

Fix the vulnerability by either:

- Restricting `enter` to an authorized role (for example, `onlyOwner` or `onlyRole(ROLE_EXECUTOR)`); and
- Optionally gating `enter` behind a higher-level vault controller that enforces who can trigger EVC mutations.

Additionally:

- Consider logging and rate limiting batch operations; and
- Ensure off-chain or on-chain governance processes authorize any batch affecting protocol-owned subaccounts.

Go back to Findings Summary

H14: User-controlled refund list allows sweeping any ERC20 balances held by the zap contract

High impact finding

Impact:	High	Confidence:	High
Target:	./contracts/zaps/ERC4626ZapInWithNativeToken.sol	Type:	Access control

Description

The `zapIn` function transfers the entire balance of every token listed in user input `assetsToRefundToSender` from the contract to `currentZapSender`, with no linkage to tokens produced during the current call or to the caller's contribution. Any account can list arbitrary token addresses and sweep any ERC20 balances currently held by the zap contract, including leftovers from prior users or accidental transfers.

Affected elements:

- Function `zapIn` and state variable `currentZapSender` determining the refund recipient; and
- Loop over `assetsToRefundToSender` that performs unconditional transfers of this contract's balances.

Listing 32. Code snippet from contracts/zaps/ERC4626ZapInWithNativeToken.sol

```
131         uint256 assetsToRefundToSenderLength =
    zapInData_.assetsToRefundToSender.length;
132         address asset;
133         uint256 balance;
134
135         for (uint256 i; i < assetsToRefundToSenderLength; ++i) {
136             asset = zapInData_.assetsToRefundToSender[i];
137             balance = IERC4626(asset).balanceOf(address(this));
```

```
138             if (balance > 0) {
139                 IERC4626(asset).safeTransfer(currentZapSender, balance);
140             }
141         }
```

Because the contract executes arbitrary external calls before this loop, it is realistic for unrelated tokens to remain in the contract after execution (for example, a user omitted a token from their refund list). A subsequent caller can include that token address in `assetsToRefundToSender` and drain the entire balance to themselves.

Exploit Scenario

A malicious caller can steal stray ERC20 balances left in the contract by prior users.

Consider the following attack scenario:

1. Bob uses `zapIn` and, after his external calls and deposit, a leftover balance of `TOKEN` remains in the zap contract because he forgot to include it in `assetsToRefundToSender`.
2. Alice observes the leftover and submits her own `zapIn` with a minimal, benign call and lists `TOKEN` in `assetsToRefundToSender`.
3. The function executes and reaches the refund loop; it transfers the entire `TOKEN` balance from the zap contract to `currentZapSender` (Alice).
4. Bob's funds are effectively stolen via the public sweeper behavior.

Recommendation

Restrict and account refunds so only per-call deltas are returned to the intended recipient.

Fix the vulnerability by either:

- Tracking per-token balances before executing the user-supplied calls and only refunding the positive delta observed after execution; or
- Restricting `assetsToRefundToSender` to a vetted set of tokens (for example, the vault asset and explicitly whitelisted tokens produced by the allowed call targets), and refund only to `zapInData_.receiver` or a user-specified `refundAddress` verified up-front; or
- Removing the user-controlled refund list entirely and exposing an explicit, pull-based claim function that refunds deltas to the original caller/receiver recorded for the transaction.

These measures prevent arbitrary sweeping of unrelated, pre-existing balances held by the contract.

Go back to Findings Summary

H15: enterTransient is permissionless and executes enter with attacker-staged transient inputs

High impact finding

Impact:	High	Confidence:	High
Target:	contracts/fuses/euler/EulerV2BatchFuse.sol	Type:	Access control

Description

The `enterTransient` function is externally callable and unguarded. It reconstructs `EulerV2BatchFuseData` from transient storage, then directly calls `enter(data)`. Combined with the permissionless `TransientStorageSetInputsFuse.enter`, any address can stage arbitrary inputs and then trigger batch execution using this contract's Euler subaccounts, inheriting the same access-control weakness as `enter`.

Listing 33. Code snippet from contracts/fuses/euler/EulerV2BatchFuse.sol

```
116     function enterTransient() external {
117         bytes32[] memory inputs = TransientStorageLib.getInputs(VERSION);
118
119         // Reconstruct ABI-encoded data from bytes32 chunks
120         uint256 inputsLen = inputs.length;
121         bytes memory encodedData = new bytes(inputsLen * 32);
122         for (uint256 i; i < inputsLen; ++i) {
123             bytes32 chunk = inputs[i];
124             assembly {
125                 mstore(add(encodedData, add(32, mul(i, 32))), chunk)
126             }
127         }
128
129         // Decode the full EulerV2BatchFuseData structure
130         EulerV2BatchFuseData memory data = abi.decode(encodedData,
    (EulerV2BatchFuseData));
```

```
131
132         // Call enter and capture return values
133         (uint256 batchSize, address[] memory assets, address[] memory
    vaults) = enter(data);
```

Exploit Scenario

A two-transaction flow lets an attacker execute an arbitrary batch on the fuse-owned subaccounts.

Consider the following attack scenario:

1. Alice first calls `TransientStorageSetInputsFuse.enter` to write ABI-encoded `EulerV2BatchFuseData` for this fuse into transient storage;
2. Alice then calls `EulerV2BatchFuse.enterTransient()` which decodes those inputs and internally calls `enter(data)`;
3. The EVC executes the items on behalf of the fuse-owned subaccounts, creating or modifying positions without authorization; and
4. Bob's protocol inherits any debt, collateral configuration changes, or transfers resulting from Alice's batch.

Recommendation

Apply the same access control to `enterTransient` as to `enter`.

Fix the vulnerability by either:

- Restricting `enterTransient` to trusted operators (for example, `onlyOwner` or `onlyRole(ROLE_EXECUTOR)`); and
- Restricting `TransientStorageSetInputsFuse.enter` so only an authorized operator can stage inputs for this fuse.

As a defense-in-depth, also validate the caller identity in `enterTransient` against an allowlist maintained by governance.

Go back to Findings Summary

H16: Zero-tolerance limit check in checkLimits enables dust-induced DoS via floor rounding

High impact finding

Impact:	High	Confidence:	Medium
Target:	./contracts/libraries/AssetDistributionProtectionLib.sol	Type:	Denial of service

Description

Zero-tolerance limit check in `checkLimits` enables dust-induced denial of service via floor rounding.

The library computes an absolute per-market cap as `limit = floor(limitInPercentage * totalBalanceInVault / 1e18)` and reverts when `limit < balanceInMarket`. Because `Math.mulDiv` without an explicit rounding mode floors the result, small totals combined with small percentages produce `limit == 0`. Any non-zero `balanceInMarket` then causes an immediate revert. Market balances are derived from fuses that read freely donatable token balances held by the vault, so an external account can donate a trivial amount and make the comparison fail when limits are active.

The logic provides no tolerance threshold for dust and uses strict comparison. As a result, the system becomes fragile during startup, after redemptions, or in markets with thin balances: benign or adversarial dust makes authorized operations revert.

Listing 34. Code snippet from contracts/libraries/AssetDistributionProtectionLib.sol

```
236     function checkLimits(DataToCheck memory data_) internal view {
237         if (!isMarketsLimitsActivated()) {
238             return;
```

```
239         }
240
241         uint256 len = data_.marketsToCheck.length;
242         uint256 limit;
243
244         for (uint256 i; i < len; ++i) {
245             limit = Math.mulDiv(
246
    PlasmaVaultStorageLib.getMarketsLimits().limitInPercentage[data_.marketsToCh
    eck[i].marketId],
247                 data_.totalBalanceInVault,
248                 ONE_HUNDRED_PERCENT
249             );
250             if (limit < data_.marketsToCheck[i].balanceInMarket) {
251                 revert MarketLimitExceeded(
252                     data_.marketsToCheck[i].marketId,
    data_.marketsToCheck[i].balanceInMarket, limit
253                 );
254             }
255         }
256     }
```

Exploit Scenario

A non-privileged account can cause protected operations to revert by donating dust to a tracked market while limits are active.

Consider the following attack scenario:

1. Alice observes that markets limits are activated and a particular market has a configured percentage (for example, 10%).
2. Alice transfers a tiny amount of the market's underlying or share token directly to the vault address so that the vault's `balanceInMarket` becomes greater than zero.
3. Alice times this when the vault's `totalBalanceInVault` is very small (for example, after most funds are withdrawn) so that `limit = floor(10% * totalBalanceInVault)` evaluates to zero or to a value smaller than the dust balance.

4. Bob, a legitimate user or keeper, later triggers any operation that calls `updateMarketsBalances` and then `AssetDistributionProtectionLib.checkLimits` (for example, a deposit, withdrawal, or a maintenance action).
5. The `checkLimits` loop computes the absolute limit using floor rounding and finds `limit < balanceInMarket` for the dusted market.
6. The transaction reverts with `MarketLimitExceeded`, preventing Bob's operation from succeeding and effectively griefing the vault until governance intervenes or balances change.

This griefing also applies when `totalBalanceInVault` is not near zero: by donating to a single tracked market, Alice can push that market's share above its configured percentage and make subsequent protected calls revert.

Recommendation

Harden the limit check against rounding artifacts and dust balances so that small, externally donatable amounts cannot brick protected flows.

Fix the vulnerability by either:

- Computing the absolute cap with ceiling rounding to avoid rounding to zero: `limit = Math.mulDiv(percent, total, 1e18, Math.Rounding.Ceil)`; or
- Introducing a minimal absolute allowance or epsilon (in 18-decimal units) and compare `balanceInMarket` to `limit + epsilon`; or
- Skipping validation for markets below a protocol-defined `dustThreshold` inside `checkLimits` itself instead of relying solely on external pre-hooks.

Additional hardening recommendations:

- Clamp `limit` to at least `dustThreshold` when the configured percentage is non-zero: `limit = max(limit, dustThreshold)`.

- Ensure the dust threshold is configurable via governance and expressed consistently in 18-decimal units to match `balanceInMarket` and `totalBalanceInVault`.
- Keep the strict `<=` semantics by checking `if (balanceInMarket > limit + epsilon)` to avoid weakening the invariant.

Go back to Findings Summary

M1: Public fee realization truncates management fees via pre-mint timestamp advance (griefing)

Medium impact finding

Impact:	Medium	Confidence:	High
Target:	contracts/vaults/extensions/WrappedPlasmaVaultBase.sol	Type:	Griefing

Description

A publicly callable fee realization function advances the management fee clock before minting, causing persistent truncation of accrued fees when the computed amount rounds down to zero. This enables griefing by any account that repeatedly calls the function to zero-out accrual and permanently discard sub-unit fees.

The contract exposes function `realizeFees`, which can be called by anyone.

Listing 35. Code snippet from contracts/vaults/extensions/WrappedPlasmaVaultBase.sol

```
744     function realizeFees() external nonReentrant {
745         _calculateFees();
746     }
```

The internal routine `_realizeManagementFee` updates the `lastUpdateTimestamp` via `PlasmaVaultLib.updateManagementFeeData` before converting the time-based accrual to shares. If the computed `unrealizedFeeInUnderlying` is too small and `convertToShares` returns zero, the function early-returns and the timestamp has already been advanced. This discards the fractional accrual irreversibly.

Listing 36. Code snippet from contracts/vaults/extensions/WrappedPlasmaVaultBase.sol

```
1032     function _realizeManagementFee() internal {
1033         PlasmaVaultStorageLib.ManagementFeeData memory feeData =
     PlasmaVaultLib.getManagementFeeData();
1034         uint256 unrealizedFeeInUnderlying = getUnrealizedManagementFee();
1035
1036         PlasmaVaultLib.updateManagementFeeData();
1037
1038         uint256 unrealizedFeeInShares =
     convertToShares(unrealizedFeeInUnderlying);
1039
1040         if (unrealizedFeeInShares == 0) {
1041             return;
1042         }
1043
1044         _mint(feeData.feeAccount, unrealizedFeeInShares);
1045         emit ManagementFeeRealized(unrealizedFeeInUnderlying,
     unrealizedFeeInShares);
1046     }
```

The underlying accrual is itself computed with integer-flooring, making small deltas evaluate to zero. This further amplifies truncation when `realizeFees` is spammed in short intervals.

Listing 37. Code snippet from contracts/vaults/extensions/WrappedPlasmaVaultBase.sol

```
1006     function _getUnrealizedManagementFee(uint256 totalAssets_) internal
     view returns (uint256) {
1007         PlasmaVaultStorageLib.ManagementFeeData memory feeData =
     PlasmaVaultLib.getManagementFeeData();
1008
1009         uint256 blockTimestamp = block.timestamp;
1010
1011         if (
1012             feeData.feeInPercentage == 0 || feeData.lastUpdateTimestamp ==
     0
1013                 || blockTimestamp <= feeData.lastUpdateTimestamp
1014         ) {
1015             return 0;
1016         }
```

```
1017
1018         return Math.mulDiv(
1019             totalAssets_ * (blockTimestamp - feeData.lastUpdateTimestamp),
1020             feeData.feeInPercentage,
1021             365 days * FEE_PERCENTAGE_DECIMALS_MULTIPLIER
1022         );
1023     }
```

Impact: Any account can regularly call `realizeFees` to advance the clock and clip sub-unit accrual, reducing the effective management fees collected by the recipient. The severity depends on the vault TVL and asset decimals: with small TVL or low-decimal tokens (for example 6 decimals), the per-interval accrual often rounds to zero; even at higher TVL, a fraction of fees is systematically lost to rounding when the function is called too frequently.

Exploit Scenario

Alice calls `realizeFees` at a high frequency to repeatedly advance the fee timestamp before any meaningful accrual can accumulate. Because the contract updates `lastUpdateTimestamp` prior to minting and early-returns when the converted fee shares equal zero, the accrued fractional fees are discarded on each call.

Consider the following attack scenario:

1. Alice observes that the vault accrues less than one base unit of the underlying per block at the current TVL and fee rate.
2. Alice repeatedly calls `realizeFees` every block (or several times per minute), causing `PlasmaVaultLib.updateManagementFeeData` to advance `lastUpdateTimestamp` on each call.
3. Each call computes `unrealizedFeeInUnderlying`, which evaluates to 0 due to integer-flooring in `_getUnrealizedManagementFee`, and then `convertToShares` yields 0 shares, so the function returns without minting.
4. Because the timestamp was advanced first, the sub-unit accrual for that

interval is permanently lost. Alice continues this process for days.

5. Bob, the designated fee recipient, receives materially fewer management fees than expected, especially for low-decimal assets or during low-TVL periods.

Concrete example (illustrative):

1. TVL = 100 USDC (6 decimals), management fee = 2% per year.
2. Per 12-second block: accrual approx 100 * 0.02 / 31,536,000 * 12 = 0.00000000076 USDC, which floors to 0 base units.
3. Alice calls `realizeFees` every block for several hours; each call advances `lastUpdateTimestamp` and discards the sub-unit accrual, resulting in effectively zero fees minted to Bob for that period.

Recommendation

Mitigate the truncation by ensuring the management fee clock is not advanced unless a non-zero amount is realized, or by carrying fractional accrual forward across invocations.

Fix the vulnerability by either:

- Defer state update until after mint succeeds: compute `unrealizedFeeInUnderlying`; if it converts to zero shares, do not call `PlasmaVaultLib.updateManagementFeeData` yet. Only advance `lastUpdateTimestamp` when `unrealizedFeeInShares > 0` and mint occurs. This preserves fractional accrual for the next interval.
- Accumulate fractional residuals: track a `managementFeeRemainder` (in underlying units) in storage. On each call, add the newly computed `unrealizedFeeInUnderlying` to the remainder, convert the sum to shares using floor rounding, mint that amount, and store back any leftover remainder that could not form a full share. This guarantees no loss from integer-flooring even if `realizeFees` is spammed.

- Rate-limit or restrict `realizeFees`: optionally add a `minRealizationInterval` (cooldown) or require an authorized keeper role to call the function. The core loss-prevention fix above should still be applied to fully eliminate rounding loss.
- Realize fees only within mutating flows: since `deposit`, `mint`, `withdraw`, and `redeem` already call `_calculateFees`, consider removing or gating the public `realizeFees` entry point to avoid grief-driven micro-interval realizations.

Implementation sketch (residual-carry approach):

1. Read `residual += _getUnrealizedManagementFee` result.
2. Compute shares using `convertToShares` on `residual` with floor rounding.
3. If `shares > 0`, mint to the fee account and reduce `residual` by the value returned from `convertToAssets` for `shares`; then update `lastUpdateTimestamp`.
4. If `shares == 0`, do not update `lastUpdateTimestamp`; keep `residual` for the next call.

Security considerations:

- Do not round up when converting fees to shares; rounding up overcharges users.
- Make residual storage type large enough for underlying decimals to avoid overflow/underflow.
- Ensure reentrancy protections remain intact when integrating the new logic.

Go back to Findings Summary

M2: Unsafe uint256 cast in AaveV3BalanceFuse.balanceOf causes revert on negative net balance, DoSing balance refresh

Medium impact finding

Impact:	Medium	Confidence:	High
Target:	./contracts/fuses/aave_v3/AaveV3BalanceFuse.sol	Type:	Denial of service

Description

A signed accumulator `balanceTemp` is used to sum per-asset aToken supplies and subtract both stable and variable debts. When the total becomes negative, the function casts `int256` to `uint256` via `SafeCast.toUint256` and reverts.

Because the fuse is called through `delegatecall` by the vault engine and its result is decoded as `uint256` without any error handling, a negative intermediate total turns a read-only balance refresh into a transaction-reverting path.

This can be hit during normal operations that involve borrowing or temporary imbalances across markets, for example when the vault borrows in Aave but deploys the proceeds in another protocol tracked by a different fuse. In that case, this fuse observes debt without the matching asset and the total becomes negative.

Affected components:

- Function `AaveV3BalanceFuse.balanceOf` (accumulator and final cast);
- Library `PlasmaVaultMarketsLib.updateMarketsBalances` (delegatecall and unchecked decode); and

- Any caller that triggers `_updateMarketsBalances` (for example `PlasmaVault.updateMarketsBalances` and `UpdateBalancesPreHook`).

Listing 38. Code snippet from contracts/fuses/aave_v3/AaveV3BalanceFuse.sol

```
81          int256 balanceTemp;
82          int256 balanceInLoop;
83          uint256 decimals;
84          uint256 price; // @dev assume price represented in 8 decimals
85          address asset;
86          address aTokenAddress;
87          address stableDebtTokenAddress;
88          address variableDebtTokenAddress;
89          address plasmaVault = address(this);
90
91          for (uint256 i; i < len; ++i) {
92              balanceInLoop = 0;
93              asset = PlasmaVaultConfigLib.bytes32ToAddress(assetsRaw[i]);
94              decimals = ERC20(asset).decimals();
95              price =
    IAavePriceOracle(IPoolAddressesProvider(AAVE_V3_POOL_ADDRESSES_PROVIDER).getP
    riceOracle())
96                  .getAssetPrice(asset);
97
98              if (price == 0) {
99                  revert Errors.UnsupportedQuoteCurrencyFromOracle();
100             }
101
102             (aTokenAddress, stableDebtTokenAddress,
    variableDebtTokenAddress) = IAavePoolDataProvider(
103
    IPoolAddressesProvider(AAVE_V3_POOL_ADDRESSES_PROVIDER).getPoolDataProvider()
104                 ).getReserveTokensAddresses(asset);
105
106             if (aTokenAddress != address(0)) {
107                 balanceInLoop +=
    int256(ERC20(aTokenAddress).balanceOf(plasmaVault));
108             }
109             if (stableDebtTokenAddress != address(0)) {
110                 balanceInLoop -=
    int256(ERC20(stableDebtTokenAddress).balanceOf(plasmaVault));
111             }
112             if (variableDebtTokenAddress != address(0)) {
113                 balanceInLoop -=
```

```
    int256(ERC20(variableDebtTokenAddress).balanceOf(plasmaVault));
114             }
115
116             balanceTemp += IporMath.convertToWadInt(
117                 balanceInLoop * int256(price), decimals +
    AAVE_ORACLE_BASE_CURRENCY_DECIMALS
118             );
119         }
120
121         return balanceTemp.toUint256();
```

The return path uses `SafeCast.toUint256` on a potentially negative `int256`, which reverts and aborts the whole update cascade.

The fuse is executed via `delegatecall` and decoded as `uint256`, so any revert bubbles up and prevents market balances from being updated.

Listing 39. Code snippet from contracts/vaults/lib/PlasmaVaultMarketsLib.sol

```
44             wadBalanceAmountInUSD =
45
    abi.decode(balanceFuse.functionDelegateCall(abi.encodeWithSignature("balanceO
    f()")), (uint256));
46             dataToCheck.marketsToCheck[i].marketId = markets[i];
```

Realistic production impact:

- Balance refreshes can revert for markets with net borrowing exceeding supplied value within this fuse's scope.
- Operational flows that include a balance refresh (pre-hooks, management actions, or keeper tasks) can be bricked for those markets until the position flips back to non-negative.
- Downstream accounting (total asset tracking, distribution checks, and fee accrual) may stall because the refresh never completes.

Exploit Scenario

An adversary can intentionally push the Aave market portion of the portfolio into a net-negative state, making every balance refresh revert and preventing maintenance operations that depend on up-to-date market balances.

Consider the following attack scenario:

1. Alice, a malicious actor, uses the vault's borrowing capabilities to create a temporary imbalance: she triggers a fuse that borrows assets in Aave such that `stableDebt + variableDebt` exceeds the aToken holdings within this Aave market scope.
2. Alice (or normal market movements) causes a minor price change so the USD-converted net becomes negative for this market.
3. Alice or a routine keeper task invokes a path that refreshes balances, for example `PlasmaVault.updateMarketsBalances` or a pre-hook like `UpdateBalancesPreHook`.
4. The call flows into `PlasmaVaultMarketsLib.updateMarketsBalances`, which performs a `delegatecall` to `balanceOf` and decodes the result as `uint256`.
5. `AaveV3BalanceFuse.balanceOf` computes a negative `balanceTemp` and hits `SafeCast.toUint256`, which reverts.
6. Bob's protocol attempt to update market balances reverts, blocking further accounting updates and potentially preventing other dependent operations from proceeding.

Operationally, this denial of service persists until the net in this market returns to non-negative (for example via liquidation, repayment, or manual rebalancing), at which point the refresh path succeeds again.

Recommendation

Adopt non-reverting handling for negative totals and align types across the balance refresh pipeline so that an Aave-only negative net value does not

brick the whole update.

Fix the vulnerability by either:

- Applying a saturating cast in `AaveV3BalanceFuse.balanceOf` to prevent reverts:
- If `balanceTemp <= 0`, return `0` instead of casting;
- Otherwise, return `uint256(balanceTemp)`.

  ```
  This preserves the non-negative invariant expected by callers and avoids
  reverting delegatecalls.
  ```

- Or, if protocol design requires representing debts explicitly, refactor the interface and pipeline to signed arithmetic:
- Change `IMarketBalanceFuse.balanceOf` to return `int256`;
- Update `PlasmaVaultMarketsLib.updateMarketsBalances` to decode a signed value and propagate it through conversions; and
- Adjust accounting to accept signed market balances or maintain a separate debt tracking channel.

In all cases, ensure:

- Callers never perform `toUint256` on potentially negative values;
- Balance refresh paths cannot revert solely due to a negative intermediate result; and
- Unit tests cover scenarios with net-negative Aave market exposure and cross-market deployments.

Go back to Findings Summary

M3: Composite price feed lacks staleness and round-integrity validation

Medium impact finding

Impact:	Medium	Confidence:	High
Target:	./contracts/price_oracle/price_feed/DualCrossReferencePriceFeed.sol	Type:	Data validation

Description

The contract computes `ASSET_X/USD` by multiplying two external aggregator answers (`ASSET_X/ASSET_Y` and `ASSET_Y/USD`) but does not validate freshness or round integrity for either input. There are no checks for `updatedAt` being non-zero and within an acceptable age, no verification that `answeredInRound >= roundId`, and no synchronization bound between the two timestamps. As a result, a stale or incomplete round from either source can be silently accepted, producing a materially incorrect USD price.

The implementation also discards and zeroes out the metadata tuple it returns (`roundId`, `startedAt`, `updatedAt`, `answeredInRound`). This removes the ability for downstream consumers to implement their own freshness gates even if they wanted to. The only guard present is a sign check that rejects non-positive prices, which does not mitigate staleness or round desynchronization.

Listing 40. Code snippet from contracts/price_oracle/price_feed/DualCrossReferencePriceFeed.sol

```
62      function latestRoundData()
63          external
64          view
65          override
66          returns (uint80 roundId, int256 price, uint256 startedAt, uint256
```

```
    time, uint80 answeredInRound)
67      {
68          (
69              uint80 assetYUsdRoundId,
70              int256 assetYPriceInUsd,
71              uint256 assetYStartedAt,
72              uint256 assetYUpdatedAt,
73              uint80 assetYAnsweredInRound
74          ) = AggregatorV3Interface(ASSET_Y_USD_ORACLE_FEED).latestRoundData();
75
76          (
77              uint80 assetXYRoundId,
78              int256 assetXPriceInAssetY,
79              uint256 assetXYStartedAt,
80              uint256 assetXYUpdatedAt,
81              uint80 assetXYAnsweredInRound
82          ) =
    AggregatorV3Interface(ASSET_X_ASSET_Y_ORACLE_FEED).latestRoundData();
83
84          if (assetXPriceInAssetY <= 0 || assetYPriceInUsd <= 0) revert
    NegativeOrZeroPrice();
85
86          price = IporMath.convertToWad(
87                  assetXPriceInAssetY.toUint256() *
    assetYPriceInUsd.toUint256(),
88                  AggregatorV3Interface(ASSET_X_ASSET_Y_ORACLE_FEED).decimals()
89
90                      +
    AggregatorV3Interface(ASSET_Y_USD_ORACLE_FEED).decimals()
91              ).toInt256();
92
93          return (0, price, 0, 0, 0);
94      }
```

Exploit Scenario

A stale `ASSET_Y/USD` or `ASSET_X/ASSET_Y` answer can be multiplied with a fresh answer from the other feed, yielding an incorrect composite price that is treated as current.

Consider the following attack scenario:

1. Alice monitors `ASSET_Y/USD` and detects that its Chainlink aggregator has

stopped updating due to a regional outage; the last on-chain `updatedAt` is 45 minutes old while `ASSET_X/ASSET_Y` continues updating every minute.

2. Alice interacts with Bob's protocol that relies on this price feed. Bob's protocol calls `latestRoundData` on `DualCrossReferencePriceFeed` and receives a price derived from multiplying the fresh `ASSET_X/ASSET_Y` with the stale `ASSET_Y/USD` value.
3. Because `latestRoundData` does not check `answeredInRound >= roundId`, `updatedAt != 0`, or a max age, and it returns zeros for all metadata, Bob's protocol accepts the composite price as valid.
4. Alice opens a position or manipulates collateralization thresholds at the stale-implied USD price (for example, borrowing more than she should or preventing her position from being liquidated).
5. When the underlying source resumes and prices converge, Alice has already extracted value (e.g., by borrowing underpriced assets or avoiding liquidation), and Bob's protocol suffers a financial loss.

A concrete numeric example: if the fresh `ASSET_X/ASSET_Y` is 0.05 and the stale `ASSET_Y/USD` is 2000 (actual contemporaneous `ASSET_Y/USD` is 2600), `ASSET_X/USD` is reported as 100 instead of the correct 130. Any component using this feed for collateral or pricing decisions will make decisions on a 23% mispricing.

Recommendation

Add strict freshness and round-integrity validation for both underlying aggregators, and propagate composite metadata so that downstream consumers can enforce their own checks.

Fix the vulnerability by:

- Verifying `answeredInRound >= roundId` for both sources and reverting otherwise.

- Reverting if `updatedAt == 0` for either source.
- Enforcing per-feed maximum staleness, e.g., `block.timestamp - updatedAt <= MAX_STALE_Y_USD` and `block.timestamp - updatedAt <= MAX_STALE_XY`, with values set per market risk.
- Enforcing a maximum skew between the two timestamps, e.g., `abs(assetYUpdatedAt - assetXYUpdatedAt) <= MAX_SKEW` to prevent composing answers from significantly different market regimes.
- Returning meaningful metadata: choose conservative composites such as `roundId = 0`, `startedAt = min(assetYStartedAt, assetXYStartedAt)`, `updatedAt = min(assetYUpdatedAt, assetXYUpdatedAt)`, and `answeredInRound = 0` (or introduce a custom interface) so that downstream systems can still verify freshness using `updatedAt`.
- Optionally, add circuit breakers to revert on extreme deviations between the two legs or when either feed is flagged by its provider.

These checks should be implemented before computing `price` and before returning, ensuring that no stale or incomplete rounds are ever used to derive `ASSET_X/USD`.

Go back to Findings Summary

M4: Third-party deposits/mints can grief-lock arbitrary receivers for the full redemption delay

Medium impact finding

Impact:	Medium	Confidence:	High
Target:	./contracts/managers/access/RedemptionDelayLib.sol	Type:	Griefing

Description

Deposit and mint flows can impose a redemption lock on arbitrary receivers because the enforcement key is the `receiver` address passed by the vault, not the transaction initiator. When deposits are public, any user can send a minimal amount of shares to a victim and, as a side effect, set the victim's lock for the full redemption delay window.

In the manager/vault integration, the vault passes the `receiver` into the access manager, which then calls `lockChecks(receiver, <deposit-like selector>)`. The library, in turn, unconditionally writes the lock for that `account_`.

Affected components and state:

- Function `PlasmaVault _checkCanCall` calls the manager with `receiver` for deposit-like selectors;
- Function `IporFusionAccessManager canCallAndUpdate` calls `RedemptionDelayLib lockChecks` with the supplied `caller_` (which the vault set to `receiver` in this context); and
- Function `IporFusionAccessManagersStorageLib setRedemptionLocks` writes to `redemptionLock[receiver]` without amount or consent checks.

Listing 41. Code snippet from contracts/vaults/PlasmaVault.sol

```
1402            (immediate, delay) =
    IporFusionAccessManager(authority()).canCallAndUpdate(caller_, address(
    this), sig);
1403        } else if (this.deposit.selector == sig || this.mint.selector ==
    sig) {
1404            (, address receiver) = abi.decode(_msgData()[4:], (uint256,
    address));
1405
1406            /// @dev check if the receiver of shares has access to deposit
    or mint and setup delay
1407            IporFusionAccessManager(authority()).canCallAndUpdate(receiver,
    address(this), sig);
1408            /// @dev check if the caller has access to deposit or mint and
    setup delay
1409            (immediate, delay) =
    AuthorityUtils.canCallWithDelay(authority(), caller_, address(this), sig);
1410        } else if (this.depositWithPermit.selector == sig) {
1411            (, address receiver,,,,) = abi.decode(_msgData()[4:], (uint256,
    address, uint256, uint8, bytes32, bytes32));
1412
1413            /// @dev check if the receiver of shares has access to
    depositWithPermit and setup delay
1414            IporFusionAccessManager(authority()).canCallAndUpdate(receiver,
    address(this), sig);
1415            /// @dev check if the caller has access to depositWithPermit
    and setup delay
1416            (immediate, delay) =
    AuthorityUtils.canCallWithDelay(authority(), caller_, address(this), sig);
```

Listing 42. Code snippet from contracts/managers/access/IporFusionAccessManager.sol

```
163     function canCallAndUpdate(address caller_, address target_, bytes4
    selector_)
164         external
165         override
166         returns (bool immediate, uint32 delay)
167     {
168         RedemptionDelayLib.lockChecks(caller_, selector_);
169         return super.canCall(caller_, target_, selector_);
170     }
171
172     /**
```

```
173
174      * @notice Updates whether a target contract is closed for operations
175      * @param target_ Target contract address
176      * @param closed_ New closed status
177      * @custom:access Restricted to GUARDIAN_ROLE
178      */
179
180     function updateTargetClosed(address target_, bool closed_) external
    override restricted {
181         _setTargetClosed(target_, closed_);
182     }
```

Listing 43. Code snippet from contracts/managers/access/RedemptionDelayLib.sol

```
53     function lockChecks(address account_, bytes4 sig_) internal {
54         if (
55             sig_ == WITHDRAW_SELECTOR || sig_ == REDEEM_SELECTOR || sig_ ==
   TRANSFER_FROM_SELECTOR
56                 || sig_ == TRANSFER_SELECTOR
57         ) {
58             uint256 unlockTime =
   IporFusionAccessManagersStorageLib.getRedemptionLocks().redemptionLock[accoun
   t_];
59             if (unlockTime > block.timestamp) {
60                 revert AccountIsLocked(unlockTime);
61             }
62         } else if (sig_ == DEPOSIT_SELECTOR || sig_ == MINT_SELECTOR || sig_
   == DEPOSIT_WITH_PERMIT_SELECTOR) {
63             IporFusionAccessManagersStorageLib.setRedemptionLocks(account_);
64         }
65     }
```

Exploit Scenario

A third party can grief-lock a victim by making a dust deposit on their behalf when deposits are public.

Consider the following attack scenario:

1. Alice, a malicious actor, calls `PlasmaVault.deposit` with `assets = 1` (or other minimal positive amount) and `receiver = Bob`;

2. The vault invokes `IporFusionAccessManager.canCallAndUpdate(receiver, address(this), PlasmaVault.deposit.selector)`;
3. The manager executes `RedemptionDelayLib.lockChecks(Bob, PlasmaVault.deposit.selector)`, which sets `redemptionLock[Bob] = block.timestamp + REDEMPTION_DELAY_IN_SECONDS`;
4. Bob later calls `withdraw` or `redeem` on the same vault; and
5. The pre-call access check re-enters the manager, and `lockChecks` observes Bob's active lock and reverts with `AccountIsLocked`.

Alice can repeat the dust deposit before the lock expires to keep Bob locked for as long as desired.

Recommendation

Prevent third-party deposits and mints from unilaterally setting another user's redemption lock.

Recommended approaches (select one or combine):

- Only set the lock when the `receiver` equals the transaction initiator (the entity actually providing funds), i.e., base the lock on `caller_` rather than an arbitrary `receiver` supplied by the vault;
- Require explicit consent from the `receiver` before a third party's deposit can update their lock (for example, an opt-in bit in the manager, or a signed acceptance provided by the receiver);
- Introduce a minimum threshold for deposits/mints to trigger a lock (discourages dust griefing), and reject zero/near-zero amounts for `receiver != caller_`;
- Alternatively, provide a `depositFor`/`mintFor` flow that does not set the receiver's lock and is constrained by role or separate guardrails.

All changes should preserve the original safety goal (prevent rapid deposit

-withdraw cycles by the same actor) while eliminating the ability for unrelated third parties to impose locks on victims.

Go back to Findings Summary

M5: Validation accepts onEulerFlashLoan callback to this contract, but the function is not implemented (batch reverts)

Medium impact finding

Impact:	Medium	Confidence:	High
Target:	./contracts/fuses/euler/EulerV2BatchFuse.sol	Type:	Logic error

Description

The validator accepts batch items whose `targetContract == address(this)` when the calldata selector equals `CallbackHandlerEuler.onEulerFlashLoan.selector`. However, `EulerV2BatchFuse` does not implement `onEulerFlashLoan` and does not inherit `CallbackHandlerEuler`. Such items pass validation but will revert at execution because no matching function exists on this contract.

Within EVC, when `batch` is invoked by this fuse and an item targets this fuse (`targetContract == msg.sender` from EVC's perspective), the EVC skips authentication and calls the target. The call then fails due to missing function implementation, reverting the entire batch and causing a denial of service for any flow that depends on a flash-loan callback.

Listing 44. Code snippet from contracts/fuses/euler/EulerV2BatchFuse.sol

```
209     function _validatePlasmaVaultCallback(EulerV2BatchItem memory item_)
    internal view {
210         bytes4 selector = bytes4(_slice(item_.data, 0, 4));
211         if (selector == CallbackHandlerEuler.onEulerFlashLoan.selector) {
212             return;
213         }
214         revert UnsupportedOperation();
```

Exploit Scenario

A crafted batch that relies on a flash-loan callback reverts at execution.

Consider the following attack scenario:

1. Alice builds a batch that requests a flash loan on an Euler vault and then adds a callback item targeting `address(this)` with selector `onEulerFlashLoan`;
2. Alice submits the batch through `enter` or `enterTransient`, which passes `_validatePlasmaVaultCallback`;
3. The EVC executes the callback item by calling `EulerV2BatchFuse` with `msg.sender` equal to the EVC;
4. The call reverts because `EulerV2BatchFuse` does not implement `onEulerFlashLoan`, reverting the entire batch and blocking the intended flash-loan flow.

Recommendation

Align validation with executable targets for flash-loan callbacks.

Fix the vulnerability by either:

- Implementing `onEulerFlashLoan(bytes calldata)` in `EulerV2BatchFuse` with the expected semantics; or
- Changing validation to require `targetContract` be a dedicated callback handler that actually implements the function (for example, `CallbackHandlerEuler`), and ensuring the batch encodes that handler as the target.

Add tests to ensure that any selector allowed by validation corresponds to a callable function on the specified target.

Go back to Findings Summary

M6: updateFuseState treats fuseType==0 as not-found sentinel, causing DoS for type-0 fuses; existence check inconsistent with other update paths

Medium impact finding

Impact:	Medium	Confidence:	High
Target:	./contracts/fuses/whitelist/FuseWhitelist.sol	Type:	Logic error

Description

The library function `FuseWhitelistLib.updateFuseState` uses `fuseInfo.fuseType == 0` as a sentinel to detect a non-existent fuse. Because `addFuseType` permits registering fuse type ID `0`, and `addFuseInfo` saves the chosen `fuseType_` for the given `fuseAddress_`, any fuse legitimately configured with type `0` is treated as non-existent in the state update path and the call reverts with `FuseNotFound`.

This check is inconsistent with other update paths that validate existence via `fuseInfo.fuseAddress == address(0)` (e.g. `updateFuseType` and `updateFuseMetadata`). As a result, fuses of type `0` cannot have their state changed and, in one onboarding path, adding such a fuse reverts during the final state update.

Affected functions and variables:

- Function `FuseWhitelist.updateFuseState` (delegates to the library);
- Function `FuseWhitelistLib.updateFuseState` (faulty existence check);
- Function `FuseWhitelist.addFuses` (calls `updateFuseState` after `addFuseInfo`, so the whole transaction reverts for type `0`);
- Functions `FuseWhitelistLib.updateFuseType` and

`FuseWhitelistLib.updateFuseMetadata` (use address-based existence check);

- Variable `FuseInfo.fuseType` (misused as existence sentinel); and
- Variable `FuseInfo.fuseAddress` (correct existence sentinel elsewhere).

Listing 45. Code snippet from contracts/fuses/whitelist/FuseWhitelistLib.sol

```
402     function updateFuseState(address fuseAddress_, uint16 fuseState_)
    internal {
403         FusesStates storage fusesStates = _getFusesStatesSlot();
404
405         if (bytes(fusesStates.fusesStates[fuseState_]).length == 0) {
406             revert InvalidFuseState(fuseState_);
407         }
408
409         FuseInfo storage fuseInfo =
    _getFuseListByAddressSlot().fusesByAddress[fuseAddress_];
410
411         if (fuseInfo.fuseType == 0) {
412             revert FuseNotFound(fuseAddress_);
413         }
414
415         fuseInfo.fuseState = fuseState_;
416         emit FuseStateUpdated(fuseAddress_, fuseState_, fuseInfo.fuseType);
```

The above uses `fuseInfo.fuseType == 0` as "not found". However, type `0` can be created and assigned:

Listing 46. Code snippet from contracts/fuses/whitelist/FuseWhitelistLib.sol

```
172     function addFuseType(uint16 fuseId_, string calldata fuseTypeId_)
    internal {
173         if (bytes(fuseTypeId_).length == 0) {
174             revert EmptyFuseType();
175         }
176
177         FusesTypes storage fusesTypes = _getFusesTypesSlot();
178         if (bytes(fusesTypes.fusesTypes[fuseId_]).length != 0) {
```

```
179             revert FuseTypeAlreadyExists(fuseId_);
180         }
181
182         fusesTypes.fusesTypes[fuseId_] = fuseTypeId_;
183         fusesTypes.fusesIds.push(fuseId_);
184
185         emit FuseTypeAdded(fuseId_, fuseTypeId_);
186     }
```

And `addFuseInfo` immediately persists the provided type and address:

Listing 47. Code snippet from contracts/fuses/whitelist/FuseWhitelistLib.sol

```
388         fuseInfoByAddress.fusesByAddress[fuseAddress_].fuseType = fuseType_;
389         fuseInfoByAddress.fusesByAddress[fuseAddress_].fuseAddress =
    fuseAddress_;
390         fuseInfoByAddress.fusesByAddress[fuseAddress_].fuseState = 0;
391         fuseInfoByAddress.fusesByAddress[fuseAddress_].timestamp =
    deploymentTimestamp_;
```

By contrast, other update paths correctly detect non-existence via a zero `fuseAddress`:

Listing 48. Code snippet from contracts/fuses/whitelist/FuseWhitelistLib.sol

```
326     function updateFuseType(address fuseAddress_, uint16 fuseType_) internal
    {
327         FuseInfo storage fuseInfo =
    _getFuseListByAddressSlot().fusesByAddress[fuseAddress_];
328         FusesTypes storage fusesTypes = _getFusesTypesSlot();
329
330         if (fuseInfo.fuseAddress == address(0)) {
331             revert FuseNotFound(fuseAddress_);
332         }
```

This inconsistency propagates to the onboarding flow, where `addFuses` finishes by calling `updateFuseState`:

Listing 49. Code snippet from contracts/fuses/whitelist/FuseWhitelist.sol

```
109         for (uint256 i; i < length; ++i) {
110             if (FuseWhitelistLib.isFuseAddressExists(fuses_[i])) {
111                 revert FuseWhitelistFuseAlreadyExists(fuses_[i]);
112             }
113             FuseWhitelistLib.addFuseToListByType(types_[i], fuses_[i]);
114             FuseWhitelistLib.addFuseInfo(types_[i], fuses_[i],
    deploymentTimestamps_[i]);
115             FuseWhitelistLib.addFuseToMarketId(fuses_[i]);
116             FuseWhitelistLib.updateFuseState(fuses_[i], states_[i]);
```

Realistic impact:

- Any fuse with type ID `0` cannot have its state updated via `updateFuseState` because it will always revert with `FuseNotFound`.
- The `addFuses` path will revert when attempting to initialize state for a fuse of type `0`, blocking successful onboarding in that configuration.
- Operators may end up unable to transition states (e.g., enabling/disabling a fuse) for those fuses, which can impede incident response or lifecycle management.

Exploit Scenario

A malicious or careless configuration can make state management impossible for affected fuses.

Consider the following attack scenario:

1. Alice has management roles to configure whitelists. She registers fuse type `0` and adds a fuse with that type using `addFuses`.
2. Alice (or an automated onboarding process) triggers the onboarding call. The call reaches `FuseWhitelistLib.updateFuseState` to set the initial state.
3. The library checks `fuseInfo.fuseType == 0` and wrongly treats the fuse as

non-existent, reverting with `FuseNotFound`.

4. Bob, who operates the protocol, is unable to onboard or later change the state of this fuse, resulting in a denial of service on administrative state updates.

Alternatively, if the fuse was first added with a non-zero type and later changed:

1. Alice calls `updateFuseType` to change the fuse to type `0` (this succeeds because existence is checked using `fuseAddress`).
2. Later, when Bob attempts `updateFuseState` (for example, to disable the fuse during an incident), the call always reverts with `FuseNotFound`, preventing timely mitigation.

Recommendation

Unify and harden existence checks across all update paths.

Recommended changes:

- In `FuseWhitelistLib.updateFuseState`, determine existence using `fuseInfo.fuseAddress == address(0)` (consistent with `updateFuseType` and `updateFuseMetadata`), not by inspecting `fuseInfo.fuseType`.
- Additionally, decide one of the following policy options and enforce it explicitly:
- Forbid type `0`: in `addFuseType`, require `fuseId_ != 0`, and audit existing data for type `0` before deployment; or
- Allow type `0`: ensure no logic path interprets `fuseType == 0` as non-existence and update any comments/tests accordingly.
- Add unit tests that cover:
- Adding a fuse with type `0` and subsequently updating its state;

- Migrating an existing fuse to type `0` and performing state updates; and
- Negative tests for truly non-existent fuses using the zero `fuseAddress` sentinel.

These changes make `updateFuseState` consistent with the rest of the codebase and remove the denial-of-service vector for legitimate type `0` fuses.

Go back to Findings Summary

M7: withdraw transfers fewer assets than requested, violating ERC-4626 semantics

Medium impact finding

Impact:	Medium	Confidence:	High
Target:	./contracts/vaults/PlasmaVault.sol	Type:	Standards violation

Description

ERC-4626 requires that withdraw(assets, receiver, owner) transfers exactly assets of the underlying to the receiver while burning the number of shares previewed by previewWithdraw(assets). The implementation in `withdraw` reduces the transferred assets by converting a fee share amount back into assets and also burns those fee shares separately. As a result, the receiver gets fewer tokens than the `assets_` argument, violating ERC-4626 semantics and breaking integrators that assume standard behavior.

Specifically, the function computes `shares = convertToShares(assets_)`, obtains `feeSharesToBurn`, then calls `_withdraw` with `assets_ - convertToAssets(feeSharesToBurn)` and `shares - feeSharesToBurn`, and finally burns `feeSharesToBurn` from the owner. Total shares burned equal `convertToShares(assets_)`, but the assets transferred are strictly less than `assets_`.

Listing 50. Code snippet from contracts/vaults/PlasmaVault.sol

```
714         uint256 shares = convertToShares(assets_);
715
716         uint256 feeSharesToBurn =
    WithdrawManager(withdrawManager).canWithdrawFromUnallocated(shares);
717
718         withdrawnShares = shares - feeSharesToBurn;
719
720         super._withdraw(
```

```
721             _msgSender(), receiver_, owner_, assets_ -
    super.convertToAssets(feeSharesToBurn), withdrawnShares
722         );
723
724         if (feeSharesToBurn > 0) {
725             if (_msgSender() != owner_) {
726                 _spendAllowance(owner_, _msgSender(), feeSharesToBurn);
727             }
728
729             _burn(owner_, feeSharesToBurn);
730         }
731     }
```

Affected components:

- Function `withdraw` in contract `PlasmaVault`;
- Fee handling variables `feeSharesToBurn`, and conversions via `convertToShares` and `convertToAssets`.

Impact in production:

- Users receive fewer underlying tokens than they requested via `withdraw(assets, …)`, leading to predictable shortfalls equal to the converted fee amount.
- Any integrator that assumes ERC-4626 semantics (exact-asset withdrawals) may mis-account, underflow downstream obligations, or revert due to slippage checks, leading to denial-of-service of integrations or unintended losses.

Exploit Scenario

An attacker or arbitrageur can exploit the standards deviation to underpay downstream protocols that trust ERC-4626 semantics or cause DoS by inducing systematic shortfalls.

Consider the following attack scenario:

1. Alice holds shares and needs exactly 1,000,000 units of the underlying to fulfill an external obligation.
2. Alice calls `withdraw(1_000_000, receiver, owner)` expecting exactly 1,000,000 units as per ERC-4626.
3. The vault transfers `1_000_000 - convertToAssets(feeSharesToBurn)` and burns the corresponding shares, and then burns the fee shares separately.
4. Bob's integration, which assumes exact-asset withdrawals, either reverts due to shortfall or credits Alice for the full 1,000,000 units based on the requested amount, resulting in accounting mismatch and potential financial loss.

Recommendation

Make `withdraw` conform to ERC-4626 semantics by transferring exactly the `assets_` amount and charging fees exclusively via shares burned, not by reducing the asset transfer.

Recommended approach:

- Compute `sharesToBurn = super.previewWithdraw(assets_)` so that fees are reflected in the share amount, not in the asset amount.
- Call `super._withdraw(_msgSender(), receiver_, owner_, assets_, sharesToBurn)` without subtracting any fee from `assets_`.
- If a withdrawal fee is required, incorporate it in `sharesToBurn` only (e.g., add fee shares to the previewed shares) and do not reduce the asset transfer.
- Ensure `previewWithdraw` returns exactly the number of shares that `withdraw` will burn for a given `assets_`, preserving ERC-4626 invariants for integrators.

This keeps the invariant: withdraw(assets) transfers exactly `assets` and burns `previewWithdraw(assets)` shares.

Go back to Findings Summary

M8: previewWithdraw divides by fee instead of 1e18 - fee, inflating required shares

Medium impact finding

Impact:	Medium	Confidence:	High
Target:	contracts/vaults/PlasmaVault.sol	Type:	Logic error

Description

Function `previewWithdraw` applies the withdraw fee using the fee value as the divisor instead of the remaining fraction `1e18 - fee`.

With a typical fee of 1%, the current logic multiplies the returned share requirement by roughly 100x. This breaks ERC4626 preview semantics for `previewWithdraw`, diverges from the actual withdraw flow, and causes integrators and UIs to quote grossly inflated share requirements.

Listing 51. Code snippet from contracts/vaults/PlasmaVault.sol

```
757 function previewWithdraw(uint256 assets_) public view override returns
    (uint256) {
758     address withdrawManager =
    PlasmaVaultStorageLib.getWithdrawManager().manager;
759
760     if (withdrawManager != address(0)) {
761         /// @dev get withdraw fee in shares with 18 decimals
762         uint256 withdrawFee =
    WithdrawManager(withdrawManager).getWithdrawFee();
763
764         if (withdrawFee > 0) {
765             return Math.mulDiv(super.previewWithdraw(assets_), 1e18,
    withdrawFee);
766         }
767     }
768     return super.previewWithdraw(assets_);
769 }
```

In contrast, function `previewRedeem` scales by `1e18 - withdrawFee`, which matches the intended net-of-fee semantics. For consistency and quote correctness, `previewWithdraw` must divide by `1e18 - withdrawFee` (not by `withdrawFee`).

Exploit Scenario

A user-facing integrator relies on `previewWithdraw` to determine how many shares must be burned to withdraw a target asset amount while a withdraw fee is configured.

Consider the following attack scenario:

1. Alice holds shares in the vault while a 1% withdrawal fee is set;
2. Alice's wallet dApp calls `previewWithdraw` for 100 assets and receives an answer inflated by ~100x due to division by the fee;
3. The dApp either refuses to submit the transaction (thinking Alice lacks sufficient shares) or over-requests approvals to cover the exaggerated amount; and
4. Bob's protocol exposes broken quotes to users, leading to failed withdrawals, mispricing, and potential loss of integrator trust and revenue.

Recommendation

Replace the divisor with the remaining fraction of value after fees. The returned share requirement should be adjusted by `1e18 - withdrawFee`.

Fix the vulnerability by updating `previewWithdraw` as follows:

- Compute `baseShares = super.previewWithdraw(assets_)`;
- If a fee is active, return `Math.mulDiv(baseShares, 1e18, 1e18 - withdrawFee)`; otherwise return `baseShares`.

Add unit tests to cover fee edge cases (0, small values like 1e14–1e16, and

near 100% caps), and include property tests asserting that `previewWithdraw` is consistent with the actual `withdraw` flow within rounding bounds.

Go back to Findings Summary

M9: ERC4626.deposit without minSharesOut enables share-slippage from MEV donations and rate drift

Medium impact finding

Impact:	Medium	Confidence:	High
Target:	./contracts/zaps/ERC4626ZapInWithNativeToken.sol	Type:	Front-running

Description

The `zapIn` flow deposits the entire computed asset balance into the `ERC4626` vault via `vault.deposit(depositAssetBalance, receiver)` without validating the number of shares minted. Only a minimum-asset threshold (`minAmountToDeposit`) is enforced. This leaves users exposed to exchange-rate movement between transaction submission and execution (for example, MEV donations to the vault that raise `totalAssets` without increasing `totalSupply`), resulting in fewer shares minted for the same asset amount.

Affected elements:

- Function `zapIn` (both overloads);
- State variable `currentZapSender` is involved in refund flow but no protection exists for min shares; and
- External call `vault.deposit` is made without using the returned `shares` value nor a preview-based bound.

Listing 52. Code snippet from contracts/zaps/ERC4626ZapInWithNativeToken.sol

```
118         IERC4626 vault = IERC4626(zapInData_.vault);
119         uint256 depositAssetBalance =
    IERC4626(vault.asset()).balanceOf(address(this));
```

```
120
121         if (depositAssetBalance < zapInData_.minAmountToDeposit) {
122             revert InsufficientDepositAssetBalance();
123         }
124
125         if (referralContractAddress != address(0) && referralCode_ !=
    bytes32(0)) {
126
    ReferralPlasmaVault(referralContractAddress).emitReferralForZapIn(currentZap
    Sender, referralCode_);
127         }
128
129         vault.deposit(depositAssetBalance, zapInData_.receiver);
130
131         uint256 assetsToRefundToSenderLength =
    zapInData_.assetsToRefundToSender.length;
```

The call to `vault.deposit` mints shares at the vault's current conversion rate. A frontrunner can donate underlying assets directly to the vault contract prior to inclusion, increasing `totalAssets` while `totalSupply` remains unchanged, thereby making each share represent more assets and causing the user's deposit to mint fewer shares than intended. Because no `minSharesOut` is enforced, the transaction cannot protect the user from such adverse price movement.

Exploit Scenario

An attacker can front-run a user's zap to reduce the victim's minted shares by donating underlying to the vault.

Consider the following attack scenario:

1. Bob submits a transaction calling `zapIn` that will deposit a fixed amount of underlying into the vault.
2. Alice observes Bob's transaction in the mempool and transfers underlying tokens directly to the ERC4626 vault contract, increasing `totalAssets` without minting new shares.

3. Bob's transaction executes next and calls `vault.deposit(depositAssetBalance, receiver)` from the zap.
4. The vault mints fewer shares to Bob than expected due to the worsened conversion rate; Bob suffers value loss relative to his intent.

Recommendation

Introduce a minimum-shares-out guard and enforce preview-based slippage checks.

Fix the vulnerability by either:

- Extending `ZapInData` to include a `minSharesOut` parameter and checking the returned value from `vault.deposit`:
- Compute `uint256 shares = vault.deposit(depositAssetBalance, zapInData_.receiver);` and `require(shares >= zapInData_.minSharesOut, "Slippage");`.
- Alternatively, pre-compute the expected shares using `vault.previewDeposit(depositAssetBalance)` and revert when `previewedShares` is below a user-specified bound, then confirm after deposit that the actual minted shares meet the same threshold.

These checks ensure deposits revert if share output deteriorates due to MEV donations or other exchange-rate drift.

Go back to Findings Summary

M10: exit misuses maxCollateralAmount as exact value, causing API mismatch and revert-prone withdrawals

Medium impact finding

Impact:	Medium	Confidence:	High
Target:	./contracts/fuses/morpho/MorphoCollateralFuse.sol	Type:	Logic error

Description

The `exit` function accepts `maxCollateralAmount` that, per the struct and NatSpec, denotes the maximum amount to withdraw. However, the implementation forwards this value directly to `MORPHO.withdrawCollateral` which interprets it as an exact amount and reverts if the request would exceed the withdrawable collateral or breach the position's health. This creates an API mismatch: callers can reasonably pass a sentinel (for example, `type(uint256).max`) to withdraw up to all collateral, but the call will revert instead of withdrawing what is actually available.

Affected components:

- Function `exit` in contract `MorphoCollateralFuse`.
- Struct `MorphoCollateralFuseExitData` and its field `maxCollateralAmount`.
- External call `IMorpho.withdrawCollateral`.

Realistic impact in production:

- Revert-prone workflows when integrators or frontends pass a ceiling value. Minor state drifts (interest accrual, other users' actions) between quote time and execution can turn a previously valid ceiling into an exact amount that is no longer valid, causing the whole transaction or multicall

to revert.

- Consistency issues within this contract: `enter` clamps to the available token balance before calling `supplyCollateral`, while `exit` lacks any clamping or pre-checks, amplifying the fragility of withdrawals.

Listing 53. Code snippet from contracts/fuses/morpho/MorphoCollateralFuse.sol

```
24 /// @notice Structure for exiting (withdrawCollateral) from the Morpho
   protocol
25 /// @param morphoMarketId The Morpho market ID (bytes32) to withdraw
   collateral from
26 /// @param maxCollateralAmount The maximum amount of collateral tokens to
   withdraw (in collateral token decimals)
27 struct MorphoCollateralFuseExitData {
28     /// @notice The Morpho market ID to withdraw collateral from
29     bytes32 morphoMarketId;
30     /// @notice The maximum amount of collateral tokens to withdraw (in
   collateral token decimals)
31     uint256 maxCollateralAmount;
32 }
```

The function below forwards `maxCollateralAmount` as an exact amount to Morpho and returns the same value as the withdrawn amount, with no pre-checks or clamping.

Listing 54. Code snippet from contracts/fuses/morpho/MorphoCollateralFuse.sol

```
122         if (data_.maxCollateralAmount == 0) {
123             return (address(0), bytes32(0), 0);
124         }
125
126         if (!PlasmaVaultConfigLib.isMarketSubstrateGranted(MARKET_ID,
    data_.morphoMarketId)) {
127             revert MorphoCollateralUnsupportedMarket("exit",
    data_.morphoMarketId);
128         }
129
130         MarketParams memory marketParams =
```

```
    MORPHO.idToMarketParams(Id.wrap(data_.morphoMarketId));
131
132         MORPHO.withdrawCollateral(marketParams, data_.maxCollateralAmount,
    address(this), address(this));
133
134         asset = marketParams.collateralToken;
135         market = data_.morphoMarketId;
136         amount = data_.maxCollateralAmount;
```

Exploit Scenario

The issue is exploitable as a transaction-level DoS and reliability hazard: any caller who interprets `maxCollateralAmount` as a ceiling can trigger a revert instead of a partial withdrawal. This breaks orchestrated flows and batch operations that should degrade gracefully.

Consider the following attack scenario:

1. Alice prepares a multicall to withdraw collateral using this fuse and swaps the withdrawn collateral on a DEX, passing `maxCollateralAmount = type(uint256).max` to indicate "withdraw up to all".
2. Between Alice's off-chain quote and on-chain execution, Bob slightly increases utilization on the same market (or interest accrues), reducing Alice's safe withdrawable collateral by a small amount.
3. The fuse calls `MORPHO.withdrawCollateral` with the exact `maxCollateralAmount` value, which now exceeds the safe withdrawable amount by a tiny margin.
4. The call reverts. Alice's entire multicall reverts, incurring gas costs and potentially missing an arbitrage window or leaving subsequent recipe steps unexecuted.

A second, purely deterministic scenario:

1. Alice computes a withdrawable ceiling off-chain using current state and passes that as `maxCollateralAmount` in a batched transaction.

2. Before the fuse executes `withdrawCollateral`, another user withdraws a small amount of collateral from the same market, moving prices/ratios just enough to invalidate Alice's ceiling.
3. The call reverts, breaking Alice's batch even though a slightly smaller withdrawal would still be healthy.

Recommendation

Align the API with the intended semantics and make withdrawals degrade gracefully instead of reverting.

Two safe options exist:

- If the intent is "exact amount" semantics:
- Rename `MorphoCollateralFuseExitData.maxCollateralAmount` to `collateralAmount` and update the NatSpec accordingly.
- Keep forwarding the value to `MORPHO.withdrawCollateral` as an exact amount and return the actual amount withdrawn.
- Update any frontend/integration to stop using sentinel "max" values. Clearly document that the call will revert if the requested amount exceeds the safe withdrawable collateral.
- If the intent is "maximum (ceiling) amount" semantics:
- Compute a safe withdrawable upper bound on-chain and clamp to it before calling Morpho:
- Optionally call `MORPHO.accrueInterest(marketParams)` to avoid stale accounting.
- Read `Position memory p = MORPHO.position(Id.wrap(data_.morphoMarketId), address(this));` and `Market memory m = MORPHO.market(Id.wrap(data_.morphoMarketId));` and `MarketParams memory marketParams = MORPHO.idToMarketParams(…)`.

- Convert `p.borrowShares` to assets using `m.totalBorrowAssets` /`m.totalBorrowShares` and consult the `marketParams.oracle`/`lltv` to compute the minimum collateral required to keep the position healthy.
- Let `maxSafe = p.collateral - requiredCollateral`. Clamp: `assets = min(data_.maxCollateralAmount, max(0, maxSafe))`.
- Call `MORPHO.withdrawCollateral(marketParams, assets, address(this), address(this));` and return `assets`.
- Alternatively, expose a dedicated `exitMax` or `previewExitMax` helper that returns the max withdrawable amount so callers can pass an exact amount confidently.

In all cases, update NatSpec and struct names to precisely reflect the enforced semantics, and avoid returning `data_.maxCollateralAmount` blindly. Instead, return the actual amount withdrawn (post-clamping) so that integrations receive truthful accounting.

Go back to Findings Summary

M11: DoS in updateFuseState: existence check uses fuseType == 0, blocking valid type-0 fuses

Medium impact finding

Impact:	Medium	Confidence:	High
Target:	./contracts/fuses/whitelist/FuseWhitelistLib.sol	Type:	Logic error

Description

This function incorrectly treats `fuseInfo.fuseType == 0` as the sentinel for a missing fuse entry. In this library, fuse existence is consistently determined by `fuseInfo.fuseAddress != address(0)`, and fuse type `0` is a valid, registrable ID. As a result, any fuse added under type `0` will be permanently denied state transitions by `updateFuseState`, reverting with `FuseNotFound` even though the fuse is present.

Affected elements:

- Function `updateFuseState` uses `fuseInfo.fuseType == 0` to determine non-existence.
- State variables `FuseInfo.fuseType`, `FuseInfo.fuseAddress`.
- Functions `addFuseType` and `addFuseInfo` allow and persist type `0` as a valid type, making `0` an invalid sentinel for non-existence.

Listing 55. Code snippet from contracts/fuses/whitelist/FuseWhitelistLib.sol

```
402 function updateFuseState(address fuseAddress_, uint16 fuseState_) internal {
403     FusesStates storage fusesStates = _getFusesStatesSlot();
404
405     if (bytes(fusesStates.fusesStates[fuseState_]).length == 0) {
406         revert InvalidFuseState(fuseState_);
407     }
```

```
408
409     FuseInfo storage fuseInfo =
    _getFuseListByAddressSlot().fusesByAddress[fuseAddress_];
410
411     if (fuseInfo.fuseType == 0) {
412         revert FuseNotFound(fuseAddress_);
413     }
414
415     fuseInfo.fuseState = fuseState_;
416     emit FuseStateUpdated(fuseAddress_, fuseState_, fuseInfo.fuseType);
417 }
```

The check above contradicts how existence is verified elsewhere in the library, where `fuseInfo.fuseAddress == address(0)` is used instead. For example, `updateFuseType` performs a correct existence check by address:

Listing 56. Code snippet from contracts/fuses/whitelist/FuseWhitelistLib.sol

```
326 function updateFuseType(address fuseAddress_, uint16 fuseType_) internal {
327     FuseInfo storage fuseInfo =
    _getFuseListByAddressSlot().fusesByAddress[fuseAddress_];
328     FusesTypes storage fusesTypes = _getFusesTypesSlot();
329
330     if (fuseInfo.fuseAddress == address(0)) {
331         revert FuseNotFound(fuseAddress_);
332     }
333
334     if (bytes(fusesTypes.fusesTypes[fuseType_]).length == 0 ||
    fuseInfo.fuseType == fuseType_) {
335         revert InvalidFuseTypeId(fuseType_);
336     }
```

Moreover, type `0` is explicitly permitted by `addFuseType`, which only prevents duplicates and empty type names. This makes `0` a legitimate type ID and thus unusable as a sentinel for non-existence:

Listing 57. Code snippet from contracts/fuses/whitelist/FuseWhitelistLib.sol

```
172 function addFuseType(uint16 fuseId_, string calldata fuseTypeId_) internal {
```

```
173     if (bytes(fuseTypeId_).length == 0) {
174         revert EmptyFuseType();
175     }
176
177     FusesTypes storage fusesTypes = _getFusesTypesSlot();
178     if (bytes(fusesTypes.fusesTypes[fuseId_]).length != 0) {
179         revert FuseTypeAlreadyExists(fuseId_);
180     }
181
182     fusesTypes.fusesTypes[fuseId_] = fuseTypeId_;
183     fusesTypes.fusesIds.push(fuseId_);
184
185     emit FuseTypeAdded(fuseId_, fuseTypeId_);
186 }
```

Impact assessment:

- Any fuse legitimately registered with type `0` cannot have its state changed through `updateFuseState`; the call will always revert with `FuseNotFound`.
- This blocks governance or automation flows that rely on state transitions (e.g., enabling, disabling, quarantining a fuse), creating an availability/operational DoS for the affected fuses.
- Wake evidence: state writes and event emission are located at lines 415–416 of the function shown above, but are unreachable for type `0` fuses due to the incorrect existence check.

Exploit Scenario

Consider the following scenario:

1. Alice, a system admin, registers a new fuse type with ID `0` using the provided administration flow.
2. Alice then adds a fuse under type `0` via `addFuseInfo`, which sets `fuseInfo.fuseType = 0` and `fuseInfo.fuseAddress = <nonzero>`.
3. Later, Alice attempts to move the fuse to a new operational state by

calling `updateFuseState`.

4. The call reverts with `FuseNotFound` because the function checks `fuseInfo.fuseType == 0` instead of whether `fuseInfo.fuseAddress` is zero.
5. The fuse remains stuck in its previous state, and operational procedures depending on state transitions fail.

Recommendation

Align existence checks across the library and avoid using `fuseType == 0` as a sentinel:

- In `updateFuseState`, determine existence via `fuseInfo.fuseAddress == address(0)` and revert accordingly.
- Alternatively, reserve `0` as an invalid type ID system-wide and reject it in `addFuseType`, `addFuseInfo`, etc. This is more intrusive and requires a migration plan; prefer the first option for backward compatibility.

Operational guidance:

- Add regression tests ensuring that a fuse registered under type `0` can successfully transition state.
- Consider adding an explicit boolean `exists` flag in `FuseInfo` to make intent clear and reduce future sentinel misuse.

Go back to Findings Summary

M12: Repayment blocked when borrowing is disabled due to over-restrictive validation

Medium impact finding

Impact:	Medium	Confidence:	High
Target:	contracts/fuses/euler/EulerV2BatchFuse.sol	Type:	Denial of service

Description

Repayment is incorrectly gated by the `canBorrow` configuration flag. In `_validateEulerVault`, both `repay` and `repayWithShares` branches require `EulerFuseLib.canBorrow(…) == true`. If borrowing has been administratively disabled for a vault/subaccount, this logic prevents repayment through the fuse even when debt is outstanding, creating an unnecessary denial of service for risk reduction.

Listing 58. Code snippet from contracts/fuses/euler/EulerV2BatchFuse.sol

```
232         } else if (selector == IBorrowing.repay.selector || selector ==
    IBorrowing.repayWithShares.selector) {
233             (, address subaccount) = abi.decode(dataAfterSelector, (uint256,
    address));
234             bool canRepay = EulerFuseLib.canBorrow(MARKET_ID,
    item_.targetContract, item_.onBehalfOfAccount);
235             if (
236                 !canRepay
237                     || subaccount !=
    EulerFuseLib.generateSubAccountAddress(address(this),
    item_.onBehalfOfAccount)
238             ) {
239                 revert EulerVaultInvalidPermissions();
```

Exploit Scenario

Once borrowing is turned off for a market/subaccount, positions cannot be

repaid via this fuse.

Consider the following attack scenario:

1. Bob disables `canBorrow` for a market/subaccount after detecting elevated risk;
2. Alice (an authorized operator) attempts to repay outstanding debt via this fuse using `repay` or `repayWithShares`;
3. The `_validateEulerVault` check rejects the operation because `canBorrow` is false even though repayment should be permitted;
4. The account remains unhealthy for longer, increasing liquidation risk and operational risk for Bob's protocol.

Recommendation

Decouple repayment from the `canBorrow` flag.

Fix the vulnerability by either:

- Using a dedicated `canRepay` or `canOperate` predicate for `repay` and `repayWithShares` branches; or
- Allowing repayment whenever the subaccount matches and a positive debt exists, regardless of `canBorrow`.

As a defense-in-depth measure, consider allowing repayment even when supply/borrow are paused, to minimize risk during incident response.

Go back to Findings Summary

M13: exit returns zero in success path due to missing assignment in _performWithdraw

Medium impact finding

Impact:	Medium	Confidence:	High
Target:	./contracts/fuses/morpho/MorphoSupplyWithCallBackDataFuse.sol	Type:	Logic error

Description

Named return variable `amount` is never set in the non-exception branch of `_performWithdraw`. When `catchExceptions_` is false (the path used by `exit`), the function calls `MORPHO.withdraw` and emits `MorphoSupplyFuseExit`, but it does not assign the returned `assetsWithdrawn` to `amount`.

As a result, Solidity returns the default value `0` for the named return variable and `_performWithdraw` returns `0` even when assets were successfully withdrawn to `address(this)`. The internal `_exit` propagates this `amount` to the public `exit`, so callers observe `amount = 0` on success. This can break on-chain workflows and mislead off-chain integrations that rely on accurate return values for accounting, routing, or follow-up actions.

Listing 59. Code snippet from contracts/fuses/morpho/MorphoSupplyWithCallBackDataFuse.sol

```
255 function _performWithdraw(
256     MarketParams memory marketParams_,
257     bytes32 morphoMarketId_,
258     uint256 assets_,
259     uint256 shares_,
260     bool catchExceptions_
261 ) private returns (uint256 amount) {
262     if (catchExceptions_) {
263         try MORPHO.withdraw(marketParams_, assets_, shares_, address(this),
    address(this)) returns (
```

```
            uint256 assetsWithdrawn,
            uint256 /* sharesWithdrawn */
        ) {
            amount = assetsWithdrawn;
            emit MorphoSupplyFuseExit(VERSION, marketParams_.loanToken,
morphoMarketId_, amount);
        } catch {
            /// @dev if withdraw failed, continue with the next step
            amount = 0;
            emit MorphoSupplyFuseExitFailed(VERSION,
marketParams_.loanToken, morphoMarketId_);
        }
    } else {
        (uint256 assetsWithdrawn,) = MORPHO.withdraw(marketParams_, assets_,
shares_, address(this), address(this));
        emit MorphoSupplyFuseExit(VERSION, marketParams_.loanToken,
morphoMarketId_, assetsWithdrawn);
    }
}
```

The `else` branch obtains `assetsWithdrawn` but never assigns it to `amount`.

As shown below, `_exit` forwards the result of `_performWithdraw` to the caller, making the incorrect `0` observable via `exit`:

Listing 60. Code snippet from contracts/fuses/morpho/MorphoSupplyWithCallBackDataFuse.sol

```
function _exit(MorphoSupplyFuseExitData memory data_, bool catchExceptions_)
    internal
    returns (address asset, bytes32 market, uint256 amount)
{
    if (data_.amount == 0) {
        return (address(0), bytes32(0), 0);
    }

    if (!PlasmaVaultConfigLib.isMarketSubstrateGranted(MARKET_ID,
data_.morphoMarketId)) {
        revert MorphoSupplyFuseUnsupportedMarket("exit",
data_.morphoMarketId);
    }

    MarketParams memory marketParams =
MORPHO.idToMarketParams(Id.wrap(data_.morphoMarketId));
```

```
215     Id id = marketParams.id();
216
217     MORPHO.accrueInterest(marketParams);
218
219     uint256 totalSupplyAssets = MORPHO.totalSupplyAssets(id);
220     uint256 totalSupplyShares = MORPHO.totalSupplyShares(id);
221
222     uint256 shares = MORPHO.supplyShares(id, address(this));
223
224     if (shares == 0) {
225         return (address(0), bytes32(0), 0);
226     }
227
228     uint256 assetsMax = shares.toAssetsDown(totalSupplyAssets,
    totalSupplyShares);
229
230     if (assetsMax == 0) {
231         return (address(0), bytes32(0), 0);
232     }
233
234     asset = marketParams.loanToken;
235     market = data_.morphoMarketId;
236
237     if (data_.amount >= assetsMax) {
238         amount = _performWithdraw(marketParams, data_.morphoMarketId, 0,
    shares, catchExceptions_);
239     } else {
240         amount = _performWithdraw(marketParams, data_.morphoMarketId,
    data_.amount, 0, catchExceptions_);
241     }
242 }
```

Exploit Scenario

An attacker can profit from downstream integrations that trust `exit`'s return value to drive accounting or follow-up actions.

Consider the following attack scenario:

1. Alice holds a position in the specified Morpho market and invokes a redemption workflow that calls `exit` on this fuse;
2. The withdrawal succeeds and assets are transferred to `address(this)`, but

`exit` returns `amount = 0` due to the missing assignment;

3. Bob's orchestrator relies on the returned `amount` to update accounting or size a follow-up swap, so it credits Alice with zero and/or skips the next step;
4. The assets remain stranded in the fuse contract or are mis-accounted, breaking invariants and enabling Alice to repeat redemptions to cause persistent mispricing, griefing, or to extract value from incorrect fee/refund logic; and
5. Bob's protocol suffers financial loss and operational risk due to incorrect balances and stale routing decisions.

Recommendation

Assign the withdrawn amount to the named return variable in the non-exception branch and keep emitted events consistent with the returned value.

Fix the vulnerability by:

- In `_performWithdraw`, set `amount = assetsWithdrawn` in the `catchExceptions_ == false` branch and emit the event with `amount`;
- Adding unit tests that assert `exit` returns the actual withdrawn assets for both `catchExceptions_` branches;
- Avoiding footguns with named return variables by either using explicit local variables and a final `return amount;` or by returning the tuple directly from the `withdraw` call; and
- Keeping this implementation aligned with `MorphoSupplyFuse._performWithdraw`, which correctly assigns the value in both branches.

Go back to Findings Summary

M14: Asymmetric units across enter and exit: enter returns shares, exit returns assets

Medium impact finding

Impact:	Medium	Confidence:	High
Target:	contracts/fuses/euler/EulerV2SupplyFuse.sol	Type:	Logic error

Description

The public API of `EulerV2SupplyFuse` returns inconsistent units across complementary actions. Function `enter` returns the number of vault shares minted, while function `exit` returns the amount of underlying assets withdrawn. This unit asymmetry breaks API symmetry and invites integration mistakes where callers assume both paths are expressed in the same unit.

In ERC-4626, `deposit` returns the number of shares minted. Here, `enter` decodes that value and returns it to the caller. Conversely, `exit` computes an asset-denominated amount and returns that asset amount to the caller.

Affected elements:

- Function `enter` (returns minted shares)
- Function `exit` (returns assets)

Listing 61. Code snippet from contracts/fuses/euler/EulerV2SupplyFuse.sol

```
104 /* solhint-disable avoid-low-level-calls */
105 depositedAmount = abi.decode(
106     EVC.call(
107         data_.eulerVault,
108         plasmaVault,
109         0,
110         abi.encodeWithSelector(ERC4626Upgradeable.deposit.selector,
```

```
    transferAmount, subAccount)
111     ),
112     (uint256)
113 );
114 /* solhint-enable avoid-low-level-calls */
115 emit EulerV2SupplyEnterFuse(VERSION, data_.eulerVault, depositedAmount,
    subAccount);
```

The variable `depositedAmount` is the number of ERC-4626 shares minted, not the amount of assets transferred.

Listing 62. Code snippet from contracts/fuses/euler/EulerV2SupplyFuse.sol

```
141 uint256 finalVaultAssetAmount = IporMath.min(
142     data_.maxAmount,
143     ERC4626Upgradeable(data_.eulerVault)
144
    .convertToAssets(ERC4626Upgradeable(data_.eulerVault).balanceOf(subAccount))
145 );
```

The `finalVaultAssetAmount` is computed in asset units.

Listing 63. Code snippet from contracts/fuses/euler/EulerV2SupplyFuse.sol

```
160 withdrawnAmount = finalVaultAssetAmount;
161
162 emit EulerV2SupplyExitFuse(VERSION, data_.eulerVault, withdrawnAmount,
    subAccount);
```

Here, `withdrawnAmount` is asset-denominated. Depending on the share exchange rate and rounding, shares and assets are not interchangeable. If an integrator assumes both `enter` and `exit` return asset units, accounting and limit enforcement can be silently skewed.

Exploit Scenario

A realistic failure mode appears when an integrator expects both paths to

return asset-denominated values and enforces risk caps or performs accounting using those returns.

Consider the following attack scenario:

1. Alice targets a vault whose share price is greater than 1 asset per share due to accrued yield.
2. Alice calls `enter` with a `maxAmount` of X assets; the function returns Y, the number of shares minted where Y < X when the exchange rate > 1.
3. Bob's integration incorrectly treats the returned Y as an asset amount and updates exposure/caps using Y instead of X.
4. Alice repeats deposits to exceed Bob's intended asset cap because every deposit is underreported by the exchange rate factor, leading to risk underestimation.

Recommendation

Unify units across `enter` and `exit` and make the API contract explicit. Two safe approaches:

- Return assets for both operations: in `enter`, return the actual asset amount sent to the vault (e.g., `transferAmount`) instead of the minted shares; keep `exit` returning the asset amount withdrawn as it does today.
- Return shares for both operations: in `exit`, redeem by shares (or compute shares via `convertToShares`) and return the number of shares burned; update public/NatSpec documentation accordingly.
- Document units explicitly in function docs and return variable names.
- Add invariant tests to assert unit consistency across `enter` and `exit` paths.
- If changing return types is not possible for backward compatibility, add new functions with consistent units and deprecate the old ones.

Go back to Findings Summary

M15: Unchecked narrowing in _packNumericValue truncates or two's-complement wraps values (including address truncation)

Medium impact finding

Impact:	Medium	Confidence:	High
Target:	contracts/fuses/transient_storage/TransientStorageMapperFuse.sol	Type:	Logic error

Description

`_packNumericValue` narrows a `uint256` into smaller widths or different kinds without any range validation. For unsigned targets (`UINT128/64/32/16/8`) and `ADDRESS`, higher bits are silently truncated. For signed targets (`INT128/64/32/16/8`), the value is first downcast to the smaller unsigned width and then reinterpreted as signed, which can two's-complement wrap into negative numbers.

This behavior can corrupt mapped outputs and break invariants in downstream fuses that assume values are in-range for their declared type. Because `enter` always runs values through `_packNumericValue` for the destination type, any oversized input can be coerced into an unexpected, attacker-chosen in-range value in the destination domain.

Listing 64. Code snippet from contracts/fuses/transient_storage/TransientStorageMapperFuse.sol

```
177     function _packNumericValue(uint256 value_, DataType dataType_) internal
    pure returns (bytes32) {
178         if (dataType_ == DataType.UINT256 || dataType_ == DataType.BYTES32)
    {
179             return bytes32(value_);
180         } else if (dataType_ == DataType.UINT128) {
```

```
181             return bytes32(uint256(uint128(value_)));
182         } else if (dataType_ == DataType.UINT64) {
183             return bytes32(uint256(uint64(value_)));
184         } else if (dataType_ == DataType.UINT32) {
185             return bytes32(uint256(uint32(value_)));
186         } else if (dataType_ == DataType.UINT16) {
187             return bytes32(uint256(uint16(value_)));
188         } else if (dataType_ == DataType.UINT8) {
189             return bytes32(uint256(uint8(value_)));
190         } else if (dataType_ == DataType.ADDRESS) {
191             return bytes32(uint256(uint160(value_)));
192         } else if (dataType_ == DataType.BOOL) {
193             return bytes32(uint256(value_ != 0 ? 1 : 0));
194         } else if (dataType_ == DataType.INT256) {
195             return bytes32(value_);
196         } else if (dataType_ == DataType.INT128) {
197             return bytes32(uint256(int256(int128(uint128(value_)))));
198         } else if (dataType_ == DataType.INT64) {
199             return bytes32(uint256(int256(int64(uint64(value_)))));
200         } else if (dataType_ == DataType.INT32) {
201             return bytes32(uint256(int256(int32(uint32(value_)))));
202         } else if (dataType_ == DataType.INT16) {
203             return bytes32(uint256(int256(int16(uint16(value_)))));
204         } else if (dataType_ == DataType.INT8) {
205             return bytes32(uint256(int256(int8(uint8(value_)))));
206         }
```

Exploit Scenario

Oversized values can be coerced into attacker-chosen in-range values in the destination type by relying on truncation/wrapping.

Consider the following attack scenario:

1. Alice supplies a large numeric value through a mapping item (for example, `value_ = 2**200 + x`) and targets `toType_ = ADDRESS` in the next fuse.
2. `_packNumericValue` truncates the value to 160 bits via `uint160(value_)`, producing the low 160 bits as an address that Alice chooses by selecting `x`.
3. Bob's downstream fuse reads the mapped input as an `address` and sends

tokens or executes calls to the truncated address, which Alice controls, causing loss of funds.

Additional variants include:

1. Mapping `value_ = type(uint256).max` to a smaller unsigned width, yielding all-one bit patterns (`UINT128 -> 2**128-1`, etc.).
2. Mapping a value like `value_ = 257` to `INT8`, which becomes `1` after `uint8` narrowing and signed re-interpretation, bypassing checks that expected a large out-of-range value to be rejected upstream.

Recommendation

Validate ranges before narrowing and fail closed on out-of-range inputs.

Fix the vulnerability by either:

- Add explicit bounds checks per target type inside `_packNumericValue` and revert when `value_` exceeds the representable range (for example, `require(value_ <= type(uint128).max)` before casting to `uint128`, `require(value_ <= type(uint160).max)` for `ADDRESS`, and `require(value_ <= uint256(type(int128).max))` for signed targets when converting from an unsigned accumulator); or
- Perform type-aware conversions earlier in `_convertValue` so that signed/unsigned domains are preserved, and carry an explicit failure path if the source value does not fit the destination.

Additionally:

- Add property tests that assert no silent truncation/wrapping occurs during packing.
- Document any intentionally lossy conversions if they are part of the spec, and enforce them with explicit checks.

Go back to Findings Summary

M16: Unsafe conversion in toBytes32(bytes): unmasked mload on short inputs yields dirty higher-order bytes

Medium impact finding

Impact:	Medium	Confidence:	High
Target:	./contracts/libraries/TypeConversionLib.sol	Type:	Logic error

Description

The library function `toBytes32(bytes)` performs a raw 32-byte load from memory using `mload(add(value_, 32))` without masking based on `value_.length`.

When `value_.length` is between 1 and 31, the higher-order portion of the loaded 32-byte word is not guaranteed to be zeroed and can contain whatever resided in memory previously. Returning such a word yields non-canonical `bytes32` values whose content depends on ambient memory state rather than strictly on the input. This can lead to inconsistent keys, brittle comparisons, and unintended data disclosure when the result is persisted or emitted.

Affected function:

- `TypeConversionLib.toBytes32(bytes)`

Impact in production systems:

- If the `bytes` input comes from user-controlled calldata and the result is used as a key, identifier, or flag, the trailing uninitialized bytes can produce divergent values for logically identical inputs or across different code paths. This can break deduplication, mapping/accounting, and

authorization checks.

- If the result is logged or returned, it can expose portions of prior in-call memory that were never intended to be observable.

Listing 65. Code snippet from contracts/libraries/TypeConversionLib.sol

```
94 function toBytes32(bytes memory value_) internal pure returns (bytes32
   result) {
95     if (value_.length == 0) return 0x0;
96     assembly {
97         result := mload(add(value_, 32))
98     }
99 }
```

Why this is unsafe:

- `mload(add(value_, 32))` reads 32 bytes starting at the first data word of the dynamic `bytes` array.
- For inputs shorter than 32 bytes, only the left-most `value_.length` bytes originate from the array, while the remaining 32 - `value_.length` bytes are not masked and may contain stale memory. The function returns these bytes as-is.

Safer patterns:

- Enforce `value_.length == 32` and revert otherwise; or
- If shorter inputs are allowed, zero-pad the unused bytes based on `value_.length` before returning.

Exploit Scenario

A practical exploitation path exists whenever the library output is used as a key, identifier, or selector without post-processing.

Consider the following attack scenario:

1. Alice supplies a user-controlled `bytes` payload of length 12 to a function that converts it with `TypeConversionLib.toBytes32(bytes)` and uses the result as a mapping key for her account entry;
2. Alice triggers the conversion through different code paths that manipulate memory before the conversion (for example, by passing in arrays that cause intermediate allocations), causing the higher-order bytes returned by `mload` to differ while the visible 12-byte payload stays the same;
3. The contract writes to or verifies against distinct mapping slots because the trailing unmasked bytes differ, defeating deduplication or allowlist checks; and
4. Bob's protocol stores multiple inconsistent entries for the same logical identifier, enabling balance segregation, bypass of gating, or other integrity violations.

A secondary risk exists when the returned `bytes32` is emitted or returned: the trailing unmasked bytes can leak in-call memory contents into logs or upstream consumers, making internal data inadvertently observable.

Recommendation

Adopt one of the following safe approaches, depending on the intended semantics:

- Strict 32-byte input: require exactly one full word and revert otherwise.
- Variable-length input: explicitly zero-pad or mask the unused bytes based on `value_.length` before returning.

If variable-length input must be supported, a minimal assembly approach is:

`result := mload(add(value_, 32))` followed by `let len := mload(value_)` and `if lt(len, 32) { let mask := not(sub(shl(mul(8, sub(32, len)), 1), 1)) result := and(result, mask) }`.

Alternatively, if the goal is to derive a fixed-size identifier from arbitrary-length input, prefer `bytes32(keccak256(value_))`, which is deterministic, canonical, and avoids partial-word handling altogether.

Go back to Findings Summary

M17: Unvalidated vault address can permanently trap user assets (no post-transfer check or rescue)

Medium impact finding

Impact:	Medium	Confidence:	High
Target:	./contracts/vaults/extensions/ReferralPlasmaVault.sol	Type:	Logic error

Description

ReferralPlasmaVault.deposit trusts the caller-supplied `vault_` address and handles tokens as if it were a well-behaved ERC4626 vault.

The function first queries the asset token from `vault_`, pulls `assets_` from the user into this contract, grants `vault_` an allowance, emits a referral event, and finally calls `deposit`. There is no post-condition that verifies the vault actually pulled the tokens from this contract. A non-compliant or adversarial contract can implement the `asset()` and `deposit(uint256,address)` signatures, return success, and never transfer tokens. In that case, the user's tokens remain held by `ReferralPlasmaVault`.

The contract provides no token rescue path, and `setZapInAddress` renounces ownership, making it impossible to add a rescue function later. If a user interacts with a malicious `vault_`, assets can be permanently stuck.

Listing 66. Code snippet from contracts/vaults/extensions/ReferralPlasmaVault.sol

```
51 address assetAddress = IERC4626(vault_).asset();
52 IERC20(assetAddress).safeTransferFrom(msg.sender, address(this), assets_);
53 IERC20(assetAddress).forceApprove(vault_, assets_);
54
```

```
55 emit ReferralEvent(msg.sender, referralCode_);
56 return IERC4626(vault_).deposit(assets_, receiver_);
```

Exploit Scenario

A malicious contract can expose the ERC4626 surface without actually pulling tokens, causing user assets to be trapped in this wrapper.

Consider the following attack scenario:

1. Alice deploys a fake vault that returns a plausible asset address and makes `deposit` return a nonzero value without calling `transferFrom`.
2. Alice tricks Bob into calling `deposit` with `vault_` set to the fake vault and some `assets_`.
3. The wrapper transfers Bob's tokens into itself, approves the fake `vault_`, emits the referral event, and calls `deposit`.
4. The fake `deposit` returns success without transferring tokens; Bob receives (bogus) shares while his tokens remain in `ReferralPlasmaVault`.
5. There is no `withdraw` or rescue function, and ownership may have been renounced; Bob cannot recover his tokens and suffers a total loss equal to `assets_`.

Recommendation

Harden the wrapper against non-compliant vaults and provide an escape hatch:

- Restrict `vault_` to an allowlist managed by governance. Reject deposits to unknown vaults.
- Verify that the vault actually consumed the tokens after the external call. Record this contract's asset balance immediately before calling `deposit`, call `IERC4626(vault_).deposit(assets_, receiver_)`, then re-read the balance and `require(balanceBefore - balanceAfter == assets_)`.

- Immediately revoke the temporary allowance with `forceApprove(vault_, 0)` after the call.
- Add a controlled `recoverERC20(address token, address to, uint256 amount)` function callable by the owner/governance to rescue stray assets before ownership is renounced.
- If immutability is desired, set the allowlist and deploy with ownership already renounced only after the rescue path is in place.

Go back to Findings Summary

M18: Global per-account redemption lock (not vault-scoped) causes cross-vault coupling and DoS

Medium impact finding

Impact:	Medium	Confidence:	Medium
Target:	./contracts/managers/access/RedemptionDelayLib.sol	Type:	Logic error

Description

The redemption-delay mechanism is stored per account globally and not scoped to a particular vault. A deposit or mint in one vault sets `redemptionLock[account]` that will be enforced when the same account later withdraws, redeems, or transfers in any other vault that is managed by the same access manager instance. This cross-vault coupling can cause unintended denial of service and surprising UX when positions are independent.

The storage layout defines a single-dimension mapping keyed only by the user address, and the setter writes to that key without considering the target vault.

Listing 67. Code snippet from contracts/managers/access/IporFusionAccessManagersStorageLib.sol

```
13 struct RedemptionLocks {
14     /// @notice Maps user addresses to their deposit timestamp
15     /// @dev Used to enforce redemption delays after deposits
16     mapping(address acount => uint256 depositTime) redemptionLock;
17 }
```

Listing 68. Code snippet from contracts/managers/access/IporFusionAccessManagersStorageLib.sol

```
112     function setRedemptionLocks(address account_) internal {
113         uint256 redemptionDelay = IIporFusionAccessManager(address(
    this)).REDEMPTION_DELAY_IN_SECONDS();
114         if (redemptionDelay == 0) {
115             return;
116         }
117         RedemptionLocks storage redemptionLocks = getRedemptionLocks();
118         uint256 redemptionLock = uint256(block.timestamp) + redemptionDelay;
119         redemptionLocks.redemptionLock[account_] = redemptionLock;
120         emit RedemptionDelayForAccountUpdated(account_, redemptionLock);
121     }
```

The enforcement logic ignores the target vault and relies only on the selector and the supplied account.

Listing 69. Code snippet from contracts/managers/access/RedemptionDelayLib.sol

```
53     function lockChecks(address account_, bytes4 sig_) internal {
54         if (
55             sig_ == WITHDRAW_SELECTOR || sig_ == REDEEM_SELECTOR || sig_ ==
   TRANSFER_FROM_SELECTOR
56                 || sig_ == TRANSFER_SELECTOR
57         ) {
58             uint256 unlockTime =
   IporFusionAccessManagersStorageLib.getRedemptionLocks().redemptionLock[accoun
   t_];
59             if (unlockTime > block.timestamp) {
60                 revert AccountIsLocked(unlockTime);
61             }
62         } else if (sig_ == DEPOSIT_SELECTOR || sig_ == MINT_SELECTOR || sig_
   == DEPOSIT_WITH_PERMIT_SELECTOR) {
63             IporFusionAccessManagersStorageLib.setRedemptionLocks(account_);
64         }
65     }
```

Exploit Scenario

A user's timely exit from Vault B can be blocked by a deposit into Vault A

when both vaults share the same access manager instance.

Consider the following attack scenario:

1. Alice, a malicious actor, performs a dust `deposit` into Vault A with `receiver = Bob`;
2. Vault A calls the manager, which executes `lockChecks(Bob, PlasmaVault.deposit.selector)` and sets a fresh lock for Bob's address;
3. Bob, who holds a position in Vault B, calls `withdraw` on Vault B; and
4. Vault B's access check reuses the same manager instance and reverts with `AccountIsLocked` because Bob's global lock set by Vault A is still active.

By repeating the dust deposit in Vault A before the lock expires, Alice can keep Bob locked out of operations in Vault B, even though his position there is unrelated to Vault A.

Recommendation

Scope redemption locks per vault to avoid cross-vault coupling and denial of service.

Recommended changes:

- Change storage from `mapping(address => uint256) redemptionLock` to `mapping(address vault => mapping(address account => uint256)) redemptionLock`; and
- Thread the `target_` (vault) context through the enforcement logic:
- Update `RedemptionDelayLib lockChecks` to accept the vault address and use it as the first key when reading/writing the lock;
- Update `IporFusionAccessManager canCallAndUpdate` to pass `target_` into `lockChecks` and to only allow calls from authorized vaults (`require(msg.sender == target_)`); and

- Adjust getters like `getAccountLockTime` to be vault-scoped.

As an operational alternative, deploy one access-manager instance per vault so that locks are not shared across otherwise independent vaults.

Go back to Findings Summary

M19: Asymmetric virtual offsets in conversions enable share inflation at low TVL

Medium impact finding

Impact:	Medium	Confidence:	Medium
Target:	contracts/vaults/PlasmaVault.sol	Type:	Arithmetics

Description

`_convertToShares` and `_convertToAssets` use asymmetric virtual offsets: the share side adds `_SHARE_SCALE_MULTIPLIER()` (with `DECIMALS_OFFSET = 2` so the multiplier is 100) to the supply, while the asset side adds only `+1` to `totalAssets()`. This asymmetry makes the ratio extremely favorable to minters when `totalAssets()` is very small, enabling outsized share minting for dust deposits and capturing a disproportionate share of future yield.

When `totalAssets()` is near 0 (including exactly 0 due to fee accounting), `_convertToShares` computes approximately `assets * (supply + 100) / 1`, which can mint O(assets * supply) shares for a minimal deposit. While those shares cannot immediately withdraw more than was deposited (because `_convertToAssets` uses the same asymmetric offsets in reverse), they entitle the minter to an outsized fraction of subsequent profits, a classic low-TVL share-inflation vector.

Listing 70. Code snippet from contracts/vaults/PlasmaVault.sol

```
1457     function _convertToShares(uint256 assets, Math.Rounding rounding)
     internal view virtual override returns (uint256) {
1458         uint256 supply = totalSupply();
1459
1460         return supply == 0
1461             ? assets * _SHARE_SCALE_MULTIPLIER()
1462             : assets.mulDiv(supply + _SHARE_SCALE_MULTIPLIER(),
     totalAssets() + 1, rounding);
```

```
1463     }
1464
1465     function _convertToAssets(uint256 shares, Math.Rounding rounding)
     internal view virtual override returns (uint256) {
1466         uint256 supply = totalSupply();
1467
1468         return supply == 0
1469             ? shares.mulDiv(1, _SHARE_SCALE_MULTIPLIER(), rounding)
1470             : shares.mulDiv(totalAssets() + 1, supply +
     _SHARE_SCALE_MULTIPLIER(), rounding);
1471     }
```

The precondition for exploitation is feasible in this codebase: `totalAssets()` explicitly returns 0 when unrealized management fees exceed or equal gross assets, which can happen around fee realization or after substantial withdrawals.

Listing 71. Code snippet from contracts/vaults/PlasmaVault.sol

```
1013     function totalAssets() public view virtual override returns (uint256) {
1014         uint256 grossTotalAssets = _getGrossTotalAssets();
1015         uint256 unrealizedManagementFee =
     PlasmaVaultFeesLib.getUnrealizedManagementFee(grossTotalAssets);
1016
1017         if (unrealizedManagementFee >= grossTotalAssets) {
1018             return 0;
1019         } else {
1020             return grossTotalAssets - unrealizedManagementFee;
1021         }
1022     }
1023
1024     /// @notice Returns the total assets in the vault for a specific market
1025     /// @dev Provides market-specific asset tracking without considering
     fees
1026     ///
1027     /// Balance Tracking:
1028     /// 1. Market Assets
1029     ///    - Protocol-specific positions
1030     ///    - Deposited collateral
1031     ///    - Earned yields
1032     ///    - Pending operations
1033     ///
```

Affected components:

- Functions `_convertToShares` and `_convertToAssets` in contract `PlasmaVault`.
- The virtual offset constants `_SHARE_SCALE_MULTIPLIER()` versus `+1` on `totalAssets()`.

Impact in production:

- A dust depositor can mint disproportionately many shares when TVL is near zero, then passively siphon future yield at the expense of existing LPs.
- The total supply cap may limit the absolute number of shares, but does not prevent skewed distribution at low TVL.

Exploit Scenario

An opportunistic user can wait for moments when `totalAssets()` is near 0 and mint a disproportionate amount of shares for a dust deposit to capture future yields.

Consider the following attack scenario:

1. After large withdrawals and fee accrual, Bob's vault has `totalAssets()` equal to 0 (or 1 unit) while total supply remains positive.
2. Alice deposits a tiny amount (e.g., 1 unit). `_convertToShares` mints roughly `1 * (supply + 100) / 1` shares because of the `1` in the denominator and `_SHARE_SCALE_MULTIPLIER()` in the numerator.
3. Alice now owns a non-trivial fraction of the supply obtained for minimal capital.
4. When markets later accrue profits, Alice redeems and realizes a disproportionate share of yield. Bob, an honest LP, suffers reduced returns relative to his prior ownership.

Recommendation

Use symmetric virtual offsets of the same scale for both conversions, or remove virtual offsets entirely, to ensure conversions interpolate correctly around low TVL.

Recommended approach:

- Define a virtual liquidity constant `V` with consistent units (assets) and apply it symmetrically:
- `_convertToShares`: `assets * (supply + V) / (totalAssets + V)`
- `_convertToAssets`: `shares * (totalAssets + V) / (supply + V)`
- Choose `V` relative to `10^decimals` so that units match; do not mix `1` on assets with `_SHARE_SCALE_MULTIPLIER()` on shares.
- Alternatively, bootstrap the vault with an initial seed deposit that mints shares at a fixed ratio and disallow deposits while `totalAssets()` is below a safe threshold to avoid dust mints.
- Ensure that all preview functions reflect the exact same math to keep integrators consistent and prevent mint-at-zero TVL edge cases.

Go back to Findings Summary

M20: Whitelist checks only caller; non-whitelisted receivers and owners permitted via receiver/allowance

Medium impact finding

Impact:	Medium	Confidence:	Medium
Target:	./contracts/vaults/extensions/WhitelistWrappedPlasmaVault.sol	Type:	Access control

Description

The wrapper enforces `onlyRole(WHITELISTED)` on the caller across deposit, mint, withdraw, and redeem, but it never validates that `receiver_` or `owner_` are whitelisted.

This leaves two paths for whitelist evasion:

- A whitelisted caller can mint/deposit shares directly to any `receiver_` (including non-whitelisted accounts), enabling non-whitelisted addresses to hold wrapper shares.
- A whitelisted caller can withdraw/redeem on behalf of a non-whitelisted `owner_` using allowances and can route assets to an arbitrary (possibly non-whitelisted) `receiver_`.

The deposit/mint overrides apply the role check only to `msg.sender` and forward parameters unchanged:

Listing 72. Code snippet from contracts/vaults/extensions/WhitelistWrappedPlasmaVault.sol

```
117 function deposit(uint256 assets_, address receiver_) public override
    onlyRole(WHITELISTED) returns (uint256) {
118     return super.deposit(assets_, receiver_);
```

```
119 }
120
121 function mint(uint256 shares_, address receiver_) public override
    onlyRole(WHITELISTED) returns (uint256) {
122     return super.mint(shares_, receiver_);
123 }
```

The base deposit implementation only checks that `receiver_` is not the zero address; it does not enforce any whitelist on the recipient of newly minted shares:

Listing 73. Code snippet from contracts/vaults/extensions/WrappedPlasmaVaultBase.sol

```
175 function deposit(uint256 assets_, address receiver_) public virtual override
    nonReentrant returns (uint256) {
176     if (assets_ == 0) revert ZeroAssetsDeposit();
177     if (receiver_ == address(0)) revert ZeroReceiverAddress();
178
179     _calculateFees();
180
181     uint256 shares = previewDeposit(assets_);
182
183     if (shares == 0) revert ZeroSharesMint();
184
185     IERC20(asset()).safeTransferFrom(msg.sender, address(this), assets_);
186
187     IERC20(asset()).forceApprove(PLASMA_VAULT, assets_);
188
189     ERC4626Upgradeable(PLASMA_VAULT).deposit(assets_, address(this));
190
191     _mint(receiver_, shares);
192
193     emit Deposit(msg.sender, receiver_, assets_, shares);
194
195     lastTotalAssets = totalAssets();
196     return shares;
197 }
```

In production, this means non-whitelisted parties can be granted or accumulate shares and later exit to underlying through a whitelisted proxy, undermining KYC/whitelist policy objectives.

Exploit Scenario

A whitelisted minter can create shares for a non-whitelisted recipient.

Consider the following attack path:

1. Bob (whitelisted) calls `deposit(assets, receiver_=Alice)` where Alice is not whitelisted.
2. The wrapper mints shares directly to `receiver_` (Alice) without checking her whitelist status.
3. Alice now holds wrapper shares and can later approve a whitelisted proxy to withdraw/redeem to her address.

Recommendation

Clarify the intended whitelist policy and enforce it on all relevant actors, not just `msg.sender`.

Fix the issue by:

- For deposit/mint: require `hasRole(WHITELISTED, receiver_)` before minting shares to `receiver_`.
- For withdraw/redeem: require `hasRole(WHITELISTED, owner_)` and `hasRole(WHITELISTED, receiver_)`, or require `owner_ == msg.sender` and validate `receiver_` is whitelisted.

For end-to-end enforcement, also restrict the ERC20 share token so only whitelisted addresses can hold or receive shares by overriding transfer/transferFrom hooks to block non-whitelisted `to` (and optionally `from`) addresses.

Go back to Findings Summary

L1: Dust supply shares can become stuck due to early return when assetsMax == 0 in _exit

Low impact finding

Impact:	Low	Confidence:	High
Target:	./contracts/fuses/morpho/MorphoSupplyFuse.sol	Type:	Logic error

Description

When converting this contract's Morpho `supplyShares` to assets inside function `_exit`, the code computes `assetsMax = shares.toAssetsDown(totalSupplyAssets, totalSupplyShares)` and returns early if the result is `0`.

This early return prevents using Morpho's shares-based withdraw path that would burn the shares even when the conversion to assets rounds down to `0`. As a result, a non-zero `supplyShares` balance can become permanently stranded as "dust" inside Morpho from the perspective of this Fuse, preventing full exit/migration until enough assets accrue to make `assetsMax > 0`.

The logic therefore breaks the invariant that calling `exit` or `instantWithdraw` can fully unwind the position, and it creates operational risk for clean market exits, migrations, and precise accounting.

Listing 74. Code snippet from contracts/fuses/morpho/MorphoSupplyFuse.sol

```
166         uint256 assetsMax = shares.toAssetsDown(totalSupplyAssets,
    totalSupplyShares);
167
168         if (assetsMax == 0) {
169             return (address(0), bytes32(0), 0);
170         }
```

```
171
172         asset = marketParams.loanToken;
173         market = data_.morphoMarketId;
174
175         if (data_.amount >= assetsMax) {
176             amount = _performWithdraw(marketParams, data_.morphoMarketId, 0,
    shares, catchExceptions_);
177         } else {
178             amount = _performWithdraw(marketParams, data_.morphoMarketId,
    data_.amount, 0, catchExceptions_);
179         }
```

This behavior contradicts Morpho's own `withdraw` implementation, which accepts a non-zero `shares` parameter and burns those shares even if the resulting `assets` equals `0` due to rounding down. Therefore, the Fuse could always proceed with the shares-based withdraw to clear dust, but the early return blocks it.

Listing 75. Code snippet from node_modules/@morpho-org/morpho-blue/src/Morpho.sol

```
200     function withdraw(
201         MarketParams memory marketParams,
202         uint256 assets,
203         uint256 shares,
204         address onBehalf,
205         address receiver
206     ) external returns (uint256, uint256) {
207         Id id = marketParams.id();
208         require(market[id].lastUpdate != 0, ErrorsLib.MARKET_NOT_CREATED);
209         require(UtilsLib.exactlyOneZero(assets, shares),
    ErrorsLib.INCONSISTENT_INPUT);
210         require(receiver != address(0), ErrorsLib.ZERO_ADDRESS);
211         // No need to verify that onBehalf != address(0) thanks to the
    following authorization check.
212         require(_isSenderAuthorized(onBehalf), ErrorsLib.UNAUTHORIZED);
213
214         _accrueInterest(marketParams, id);
215
216         if (assets > 0) shares =
    assets.toSharesUp(market[id].totalSupplyAssets,
    market[id].totalSupplyShares);
217         else assets = shares.toAssetsDown(market[id].totalSupplyAssets,
```

```
    market[id].totalSupplyShares);
218
219         position[id][onBehalf].supplyShares -= shares;
220         market[id].totalSupplyShares -= shares.toUint128();
221         market[id].totalSupplyAssets -= assets.toUint128();
222
223         require(market[id].totalBorrowAssets <=
    market[id].totalSupplyAssets, ErrorsLib.INSUFFICIENT_LIQUIDITY);
224
225         emit EventsLib.Withdraw(id, msg.sender, onBehalf, receiver, assets,
    shares);
226
227         IERC20(marketParams.loanToken).safeTransfer(receiver, assets);
228
229         return (assets, shares);
```

Exploit Scenario

An attacker or any user can intentionally create and strand dust shares that this Fuse cannot clear, making it impossible to fully exit a market without out-of-band intervention.

Consider the following attack scenario:

1. Alice supplies a very small amount that mints a non-zero number of Morpho `supplyShares`, but for which `shares.toAssetsDown(totalSupplyAssets, totalSupplyShares)` later evaluates to `0`.
2. Alice (or Bob, the integrator) calls `exit` or `instantWithdraw` on the Fuse with any non-zero `amount`.
3. The Fuse reaches the `assetsMax == 0` branch and returns early without calling the shares-based Morpho `withdraw`, leaving Alice's shares in Morpho and the Fuse's position non-zero.
4. Bob's protocol cannot fully unwind or migrate the market via this Fuse and must deposit more funds to change the ratio or perform a privileged/manual call path, increasing operational burden and potentially

blocking automated offboarding flows.

Recommendation

Remove the early return and always attempt a shares-based withdrawal when the asset conversion rounds down to zero.

A safe approach is:

- If `shares == 0`: return as today.
- Else compute `assetsMax = shares.toAssetsDown(…)`.
- If `assetsMax == 0`: call `_performWithdraw(marketParams, morphoMarketId, 0, shares, catchExceptions_)` to burn all shares and return `(asset, market, 0)`.
- Else keep the existing logic of choosing between assets-based or shares-based withdrawal depending on `data_.amount` vs `assetsMax`.

Additional hardening:

- Add unit tests covering rounding-to-zero conversions to ensure dust shares are cleared.
- Document that `exit` and `instantWithdraw` must be able to settle the position completely, even when no assets are receivable.

Go back to Findings Summary

L2: Caught-withdraw path returns requested amount, misreporting actual withdrawal

Low impact finding

Impact:	Low	Confidence:	High
Target:	./contracts/fuses/aave_v3/AaveV3SupplyFuse.sol	Type:	Logic error

Description

When `catchExceptions_` is true, function `_performWithdraw` catches a revert from Aave's `withdraw` and returns `(asset_, finalAmount_)` even though nothing was actually withdrawn.

This misreports the outcome to callers by equating the requested amount with the actual withdrawn amount. While `instantWithdraw` currently ignores the return value, this behavior is fragile and can easily cause mis-accounting if the return value is ever used by internal call sites during refactors or new flows.

Affected function and variables:

- Function `_performWithdraw` in `contracts/fuses/aave_v3/AaveV3SupplyFuse.sol`.
- Parameter `catchExceptions_` controls the behavior.
- Events `AaveV3SupplyFuseExit` and `AaveV3SupplyFuseExitFailed`.

Listing 76. Code snippet from contracts/fuses/aave_v3/AaveV3SupplyFuse.sol

```
226 if (catchExceptions_) {
227     try aavePool.withdraw(asset_, finalAmount_, address(this)) returns
    (uint256 withdrawnAmount) {
228         emit AaveV3SupplyFuseExit(VERSION, asset_, withdrawnAmount);
```

```
229         return (asset_, withdrawnAmount);
230     } catch {
231         /// @dev if withdraw failed, continue with the next step
232         emit AaveV3SupplyFuseExitFailed(VERSION, asset_, finalAmount_);
233         return (asset_, finalAmount_);
234     }
```

Exploit Scenario

Consider the following attack scenario:

1. Alice triggers an emergency withdrawal path that internally calls `_performWithdraw` with `catchExceptions_ = true` for an illiquid asset on Aave.
2. Aave's `withdraw` reverts due to insufficient liquidity or a reserve freeze.
3. `_performWithdraw` catches the error and returns `(asset_, finalAmount_)`, which looks like a successful full withdrawal.
4. Bob's protocol, trusting the function's return value, over-credits internal accounting and proceeds to subsequent steps that assume funds exist, leading to downstream failures or accounting drift.

Recommendation

Make the return value reflect the actual outcome when catching exceptions.

Recommended options:

- On `catch`, return `(asset_, 0)` to indicate that no funds were withdrawn; or
- Extend the API to return a success flag along with the amount, e.g., `(bool success, address asset, uint256 amount)`, and set `success = false` on catch; or
- Emit only the failure event and rethrow if the caller must not proceed without a successful withdrawal.

At a minimum, change the `catch` branch to return zero amount so internal

callers cannot misinterpret a failed withdrawal as a successful one.

Go back to Findings Summary

L3: Native ETH sent to the executor is stuck (no payable receive and no withdrawal path)

Low impact finding

Impact:	Low	Confidence:	High
Target:	./contracts/fuses/universal_token_swapper/SwapExecutorRestricted.sol	Type:	Logic error

Description

`execute` is `external` and nonpayable, and the DEX calls are invoked via `Address.functionCall` without forwarding any `msg.value`. The contract declares no `receive` or `fallback` functions marked `payable`, and it exposes no method to withdraw native ETH. If ETH is sent to this address (e.g., by forced transfer using `selfdestruct`, or by an integration that unwraps WETH to ETH with recipient set to the executor), that ETH cannot be forwarded or recovered by design.

Operational impact: Integrations that inadvertently direct native ETH to the executor will encounter reverts (no payable receiver) or create stranded ETH (via forced transfers). This leads to asset safety and operational risks.

Listing 77. Code snippet from contracts/fuses/universal_token_swapper/SwapExecutorRestricted.sol

```
54      function execute(SwapExecutorData memory data_) external restricted {
55          uint256 len = data_.dexs.length;
56          for (uint256 i; i < len; ++i) {
57              data_.dexs[i].functionCall(data_.dexsData[i]);
58          }
```

Exploit Scenario

Any ETH that arrives cannot be recovered.

Consider the following attack scenario:

1. Alice deploys a minimal contract that immediately `selfdestruct`s to the executor's address, forcing 1 wei of ETH into it.
2. Alice (or any operator) later calls `execute` to continue normal operations, but there is no function to withdraw native ETH, nor any use of `functionCallWithValue` to forward ETH.
3. The ETH stays permanently stuck at the executor address.
4. Bob's protocol cannot recover the ETH, and the balance may accumulate over time if more forced transfers occur.

Recommendation

Harden ETH handling and provide an explicit recovery path.

Fix the issue by:

- Adding a `receive() external payable {}` function to accept ETH safely (or revert intentionally with a clear error if ETH must never be accepted).
- Adding a restricted withdrawal function, e.g., `function withdrawETH(address to, uint256 amount) external restricted`, to sweep native ETH.
- If downstream integrations require ETH, extend `SwapExecutorData` to carry per-call value and use `Address.functionCallWithValue` accordingly, validating sums against `msg.value`.

Go back to Findings Summary

L4: ETH sent to the contract is unrecoverable (no payable receiver and no withdrawal)

Low impact finding

Impact:	Low	Confidence:	High
Target:	./contracts/fuses/universal_token_swapper/SwapExecutorRestricted.sol	Type:	Logic error

Description

The contract declares no `receive()`/`fallback()` functions marked `payable` and exposes no native-ETH withdrawal method. Although `execute` never forwards value and is itself nonpayable, ETH can still be transferred to the contract (e.g., via `selfdestruct`). Any such ETH becomes stranded because there is no mechanism to sweep it out.

This creates avoidable asset-loss risk for operators if ETH is ever sent to the executor address by mistake or via force.

Listing 78. Code snippet from contracts/fuses/universal_token_swapper/SwapExecutorRestricted.sol

```
54      function execute(SwapExecutorData memory data_) external restricted {
55          uint256 len = data_.dexs.length;
56          for (uint256 i; i < len; ++i) {
57              data_.dexs[i].functionCall(data_.dexsData[i]);
58          }
```

Exploit Scenario

ETH can be permanently locked at the contract address.

Consider the following attack scenario:

1. Alice (an external actor) forces 0.1 ETH into the executor using a contract that `selfdestruct`s to its address.
2. Alice later triggers operations that would normally transfer assets out via `execute`; however, there is no function capable of sending native ETH from the executor.
3. The 0.1 ETH remains locked indefinitely at the executor address.
4. Bob's protocol has no on-chain mechanism to recover the ETH without a code change and redeployment.

Recommendation

Introduce explicit ETH handling and rescue capabilities.

Recommended changes:

- Implement `receive() external payable {}` to accept ETH or a payable `fallback()` if dynamic calls might return ETH.
- Add a `restricted` rescue function to withdraw native ETH, e.g., `withdrawETH(address to, uint256 amount)`.
- Consider emitting events on receipt and withdrawal to aid monitoring and incident response.

Go back to Findings Summary

L5: removeFuseMetadata does not revert on absent metadataId and still emits update event

Low impact finding

Impact:	Low	Confidence:	High
Target:	./contracts/fuses/whitelist/FuseWhitelistLib.sol	Type:	Logic error

Description

The implementation contradicts its own specification by not reverting when `metadataId_` is absent for a given fuse and still emitting an update event. The docstring promises a revert if the metadata ID is not found, but the function unconditionally clears the mapping and then performs a best-effort removal from `metadataIds`. If the ID is not present, the loop falls through without reverting, and the function emits `FuseMetadataUpdated` with an empty array, which can mislead off-chain consumers and monitoring systems.

Affected elements:

- Function `removeFuseMetadata`.
- State variables `FuseInfo.metadata` and `FuseInfo.metadataIds`.
- Event `FuseMetadataUpdated` is used for removals with no existence checks.

Listing 79. Code snippet from contracts/fuses/whitelist/FuseWhitelistLib.sol

```
458 /// @dev Reverts if:
459 /// - fuseAddress_ is not found
460 /// - metadataId_ is not found for the fuse
461 function removeFuseMetadata(address fuseAddress_, uint256 metadataId_)
    internal {
462     FuseListByAddress storage fuseInfoByAddress =
    _getFuseListByAddressSlot();
463
```

```
464     if (fuseInfoByAddress.fusesByAddress[fuseAddress_].fuseAddress ==
    address(0)) {
465         revert FuseNotFound(fuseAddress_);
466     }
467
468     FuseInfo storage fuseInfo =
    fuseInfoByAddress.fusesByAddress[fuseAddress_];
469
470     fuseInfo.metadata[metadataId_] = new bytes32[](0);
471
472     for (uint256 i; i < fuseInfo.metadataIds.length; ++i) {
473         if (fuseInfo.metadataIds[i] == metadataId_) {
474             fuseInfo.metadataIds[i] =
    fuseInfo.metadataIds[fuseInfo.metadataIds.length - 1];
475             fuseInfo.metadataIds.pop();
476             break;
477         }
478     }
479
480     emit FuseMetadataUpdated(fuseAddress_, metadataId_, new bytes32[](0));
481 }
```

Impact assessment:

- Callers expecting a revert on unknown `metadataId_` will receive a successful transaction instead, violating invariants in higher-level workflows.
- Off-chain indexers and alerting systems can be misled by the `FuseMetadataUpdated` event into believing metadata was removed when it never existed, creating data quality issues and potential operational mistakes.
- From a state perspective, the mapping is set to an explicit empty array even when it was previously unset; the semantic difference is negligible on-chain, but the event emission increases confusion and off-chain attack surface.

Wake evidence: lines 470 and 480 show the unconditional write and event emission; the loop does not revert if the ID is absent.

Exploit Scenario

Consider the following scenario:

1. Alice, an operator, mistakenly attempts to remove metadata ID `9999` that was never added to a fuse.
2. The call to `removeFuseMetadata` succeeds, because there is no check ensuring that `metadataId_` exists in `fuseInfo.metadataIds`.
3. The function emits `FuseMetadataUpdated(fuse, 9999, [])`, which Bob's off-chain indexer interprets as a removal of existing metadata.
4. Bob updates dashboards and automation based on the event, believing metadata was deleted, while on-chain state was effectively unchanged.

Recommendation

Enforce the documented precondition and make event semantics explicit:

- Before modification, require that `metadataId_` exists for the fuse (e.g., scan `fuseInfo.metadataIds` or maintain a mapping `hasMetadata[metadataId_]` for O(1) checks). Revert with a dedicated error when the ID is absent.
- Emit a dedicated removal event (e.g., `FuseMetadataRemoved`) or continue using `FuseMetadataUpdated` but only after a successful existence check, so off-chain consumers can rely on event integrity.
- Optionally normalize state by deleting the mapping key rather than assigning `new bytes32[](0)` if gas permits, to reduce ambiguity between unset and empty.

Testing guidance:

- Add tests that confirm a revert when `metadataId_` is not present and that the correct event fires only when removal actually occurs.

Go back to Findings Summary

L6: Double transfer when tokenIn == tokenOut triggers revert and denies execution

Low impact finding

Impact:	Low	Confidence:	High
Target:	./contracts/fuses/universal_token_swapper/SwapExecutorRestricted.sol	Type:	Denial of service

Description

The function `execute` reads both `balanceTokenIn` and `balanceTokenOut` before performing any transfers. When `data_.tokenIn` equals `data_.tokenOut`, both reads refer to the same ERC20 token and thus produce the same pre-transfer amount. The code then performs two transfers using those pre-read values. After the first transfer empties the contract's token balance, the second transfer attempts to transfer the same amount again and reverts via `SafeERC20.safeTransfer` due to insufficient balance.

Affected function: `execute`.

Affected variables: `data_.tokenIn`, `data_.tokenOut`, `balanceTokenIn`, `balanceTokenOut`.

Realistic impact: Any route that intentionally or accidentally results in the same token for input and output will revert at settlement time, denying execution. No assets are lost within the transaction itself (the revert rolls back all internal effects), but legitimate orchestrations that pre-fund the executor may be temporarily blocked until a subsequent successful call moves the funds.

Listing 80. Code snippet from contracts/fuses/universal_token_swapper/SwapExecutorRestricted.sol

```
60          uint256 balanceTokenIn = IERC20(data_.tokenIn).balanceOf(address(
    this));
61          uint256 balanceTokenOut = IERC20(data_.tokenOut).balanceOf(address
    (this));
62
63          if (balanceTokenIn > 0) {
64              IERC20(data_.tokenIn).safeTransfer(msg.sender, balanceTokenIn);
65          }
66
67          if (balanceTokenOut > 0) {
68              IERC20(data_.tokenOut).safeTransfer(msg.sender, balanceTokenOut);
69          }
```

Exploit Scenario

An authorized caller can reliably trigger a revert when input and output tokens coincide.

Consider the following attack scenario:

1. Alice (the authorized `RESTRICTED` caller) pre-funds the executor with 1,000 units of token `T` for an upcoming route.
2. Alice calls `execute` with `data_.tokenIn == data_.tokenOut == T` and a route that leaves the executor holding 1,000 `T` at the end (or even no DEX calls).
3. The function reads `balanceTokenIn = balanceTokenOut = 1,000`. It then transfers 1,000 `T` once, and immediately attempts a second transfer of 1,000 `T` based on the pre-read `balanceTokenOut`.
4. The second transfer reverts because the executor's current balance is already zero; the entire transaction rolls back and the route cannot complete. Bob's protocol experiences a denial of service for that route until the parameters are corrected.

Recommendation

Detect and handle the `tokenIn == tokenOut` case, or re-order balance reads to avoid using stale pre-transfer values.

Fix the vulnerability by either:

- If `data_.tokenIn == data_.tokenOut`, read the balance once and perform a single transfer of that token to `msg.sender`.
- Otherwise, read and transfer each token independently as intended.
- Alternatively, re-read `balanceTokenOut` after transferring `tokenIn` (and only transfer it if still non-zero), but the explicit equality check is clearer and avoids unnecessary reads.

Go back to Findings Summary

L7: Unconditional ETH refund to sender can revert for non-payable contracts, causing denial of service

Low impact finding

Impact:	Low	Confidence:	High
Target:	./contracts/zaps/ERC4626ZapInWithNativeToken.sol	Type:	Denial of service

Description

After completing the zap, the contract unconditionally attempts to send any leftover native ETH to `currentZapSender` using `Address.sendValue`. If `currentZapSender` is a contract without a payable `receive`/`fallback`, this transfer reverts and bricks otherwise successful zaps for such integrations. The behavior also enables griefing if ETH is forcibly sent to the zap during execution, making the final refund fail.

Affected elements:

- Function `zapIn` final native refund logic; and
- State variable `currentZapSender` determines the refund recipient.

Listing 81. Code snippet from contracts/zaps/ERC4626ZapInWithNativeToken.sol

```
145         if (nativeTokenBalance > 0) {
146             Address.sendValue(payable(currentZapSender),
    nativeTokenBalance);
147         }
```

OpenZeppelin `Address.sendValue` reverts on failure. When the sender is a non-payable contract, this causes a hard revert even when all prior steps succeeded, preventing integrations with such contracts and opening a grief

vector via forced ETH.

Exploit Scenario

A non-payable contract calling the zap cannot receive leftover ETH and will see its transaction revert.

Consider the following attack scenario:

1. Bob implements an integration contract that calls `zapIn` but has no payable `receive`/`fallback` (by design).
2. During the zap, one of the external calls sends back a small amount of ETH to the zap contract, leaving a positive `address(this).balance`.
3. The function reaches the refund step and calls `Address.sendValue(payable(currentZapSender), nativeTokenBalance)` to Bob's contract.
4. The ETH transfer fails; `Address.sendValue` reverts, bricking the entire zap for Bob despite all prior steps succeeding.

Recommendation

Make ETH refunds opt-in and compatible with non-payable receivers.

Fix the vulnerability by either:

- Adding a `refundNativeTo` field to `ZapInData` allowing users to specify an EOA or a known payable address (or zero address to skip refunds), and skipping the refund when unset; or
- Wrapping leftover ETH into WETH and transferring WETH to the receiver (or `refundNativeTo`) instead of forcing an ETH transfer; or
- Attempting `sendValue`, and on failure, wrapping to WETH and transferring WETH, preventing a revert.

These changes avoid bricking flows for non-payable contracts and remove a griefing vector.

Go back to Findings Summary

L8: Redundant pre-approval to Morpho before flashLoan widens allowance window and wastes gas

Low impact finding

Impact:	Low	Confidence:	High
Target:	contracts/fuses/morpho/ MorphoFlashLoanFuse.sol	Type:	Trust model

Description

The fuse is executed via `delegatecall` from `PlasmaVault.execute`, therefore `address(this)` inside function `enter` is the `PlasmaVault` itself. The code sets an ERC20 allowance for `MORPHO` before calling `flashLoan`, while the callback path also sets the exact same allowance just-in-time via `CallbackHandlerLib.handleCallback`.

This pre-approval both widens the trust/allowance window and duplicates gas costs:

- Trust window: `MORPHO` receives spending power over the vault's `asset` from the beginning of the external `flashLoan` call. If `MORPHO` is compromised or misconfigured, it can call `transferFrom` and pull up to `tokenAmount` immediately, before the callback finishes. The same result would be achievable safely by granting the allowance only at the end of the callback, immediately before `MORPHO` attempts to pull the repayment.
- Gas duplication: the allowance is set twice to the same value (pre-call and then again inside the callback) and then reset to zero after the call. For non-standard tokens that enforce the approve(0) pattern, `forceApprove` may incur two writes, further increasing costs.

Affected elements:

- Function `enter` in `contracts/fuses/morpho/MorphoFlashLoanFuse.sol`;
- Immutable `MORPHO` (approval spender);
- Callback flow in `CallbackHandlerLib.handleCallback` that sets the allowance.

Listing 82. Code snippet from contracts/fuses/morpho/MorphoFlashLoanFuse.sol

```
98 ERC20(data_.token).forceApprove(address(MORPHO), data_.tokenAmount);
99
100 MORPHO.flashLoan(
101     data_.token,
102     data_.tokenAmount,
103     abi.encode(
104         CallbackData({
105             asset: data_.token,
106             addressToApprove: address(MORPHO),
107             amountToApprove: data_.tokenAmount,
108             actionData: data_.callbackFuseActionsData
109         })
110     )
111 );
112
113 ERC20(data_.token).forceApprove(address(MORPHO), 0);
```

Listing 83. Code snippet from contracts/libraries/CallbackHandlerLib.sol

```
95 CallbackData memory calls = abi.decode(data, (CallbackData));
96
97 PlasmaVault(address(this)).executeInternal(abi.decode(calls.actionData,
   (FuseAction[])));
98
99 ERC20(calls.asset).forceApprove(calls.addressToApprove,
   calls.amountToApprove);
```

Exploit Scenario

The allowance to `MORPHO` is granted before control is transferred to the

external protocol. Because the fuse runs in the context of the `PlasmaVault` (via `delegatecall`), this pre-approval temporarily gives `MORPHO` permission to pull `asset` tokens directly from the vault even before the callback sets an allowance.

Consider the following attack scenario:

1. Alice controls or compromises the `MORPHO` contract for the configured market;
2. Alice triggers the vault to run function `enter` with `token` equal to an asset the vault already holds and `tokenAmount` large enough;
3. The vault pre-approves `MORPHO` for `tokenAmount` and then calls `MORPHO.flashLoan`;
4. During `flashLoan`, Alice's `MORPHO` immediately calls `transferFrom` to pull `min(vaultBalance(token), tokenAmount)` from the vault, before the callback finishes; and
5. Bob, the vault's LPs, suffer a loss of the siphoned tokens while the transaction still succeeds, because the protocol later proceeds to complete the flash loan and the post-call zeroing does not revert prior transfers.

Recommendation

Remove the redundant pre-approval and rely on the just-in-time approval performed in the callback.

Fix the vulnerability by either:

- Deleting the pre-call `forceApprove` in function `enter` (line 98) and keeping the post-call zeroing to clear the callback-set allowance; or
- If a pre-approval is strictly required by a future integration, set it to zero first and grant it only immediately before the repayment step in the

callback, never before the external `flashLoan` call.

Additional hardening:

- Validate that `CallbackData.addressToApprove` is equal to `address(MORPHO)` and `CallbackData.asset` equals `data_.token` to prevent approving arbitrary spenders/tokens via mis-encoded callback data;
- Keep allowances minimal and time-bounded (grant only `tokenAmount` and zero them right after the external call returns).

Go back to Findings Summary

L9: Zero-amount supply in enter can revert on Aave and halt execution pipeline

Low impact finding

Impact:	Low	Confidence:	High
Target:	./contracts/fuses/aave_v3/AaveV3SupplyFuse.sol	Type:	Logic error

Description

A zero-amount call to Aave V3 `supply` can be made by function `enter` when `data_.amount > 0` but the contract holds fewer or no tokens of `data_.asset`.

The code computes `finalAmount = min(balanceOf(this), data_.amount)` and unconditionally approves and calls Aave's `supply` with `finalAmount`. When `finalAmount == 0`, Aave V3 reverts with an invalid amount error, unexpectedly halting execution.

This can occur whenever upstream orchestration mis-estimates the vault's balance (for example, a prior step failed to transfer tokens) or races leave the contract with a zero balance at call time. The early return at the top of `enter` only guards `data_.amount == 0` and does not protect against an empty balance for a non-zero requested amount.

Affected function and variables:

- Function `enter` in `contracts/fuses/aave_v3/AaveV3SupplyFuse.sol`.
- Local variable `finalAmount`.
- External calls to `ERC20.forceApprove` and `IPool.supply`.

Listing 84. Code snippet from contracts/fuses/aave_v3/AaveV3SupplyFuse.sol

```
104 IPool aavePool =
    IPool(IPoolAddressesProvider(AAVE_V3_POOL_ADDRESSES_PROVIDER).getPool());
105
106 uint256 finalAmount = IporMath.min(ERC20(data_.asset).balanceOf(address(
    this)), data_.amount);
107
108 ERC20(data_.asset).forceApprove(address(aavePool), finalAmount);
109
110 aavePool.supply(data_.asset, finalAmount, address(this), 0);
```

Exploit Scenario

Consider the following attack scenario:

1. Alice, an allowed operator, triggers a route that calls function `enter` with `amount = 100_000` of an asset.
2. The vault unexpectedly holds `0` units of that asset because a prior step failed to fund it.
3. Function `enter` computes `finalAmount = 0` and calls Aave's `supply` with `0`.
4. Aave reverts on zero amounts, and the entire orchestrated transaction fails, preventing Bob's protocol from progressing this action until funds are actually present.

Recommendation

Short-circuit when there is nothing to supply, and avoid calling Aave with a zero amount.

A practical fix in `enter` is to add:

- A guard returning early when `finalAmount == 0`; and
- Optionally, an informational event to aid observability.

For example, before the approval/supply calls:

- Compute `finalAmount` as today; then
- If `finalAmount == 0`, return `(data_.asset, 0)`.

This ensures the function behaves predictably when unfunded, avoids an unnecessary revert from Aave, and preserves the rest of the execution pipeline.

Go back to Findings Summary

L10: latestRoundData leaves metadata unset (roundId/startedAt/answeredInRound = 0), breaking AggregatorV3 consumer assumptions

Low impact finding

Impact:	Low	Confidence:	High
Target:	./contracts/price_oracle/price_feed/PtPriceFeed.sol	Type:	Standards violation

Description

Function `latestRoundData` implements a Chainlink-like interface but never assigns `roundId`, `startedAt`, or `answeredInRound`. Those values are therefore returned as zero.

Many on-chain consumers expect non-zero metadata to perform basic sanity and staleness checks (e.g., `require(startedAt > 0)`, `require(answeredInRound >= roundId)`). Returning default zeros breaks such expectations and can cause integrations to revert or skip safety checks.

Listing 85. Code snippet from contracts/price_oracle/price_feed/PtPriceFeed.sol

```
101     /// @inheritdoc IPriceFeed
102     function latestRoundData()
103         external
104         view
105         returns (uint80 roundId, int256 price, uint256 startedAt, uint256
    time, uint80 answeredInRound)
106     {
107         uint32 twapWindow = TWAP_WINDOW;
108 
109         uint256 unitPrice;
110         if (USE_PENDLE_ORACLE_METHOD == 1) {
111             unitPrice =
```

```
    PendlePYOracleLib.getPtToAssetRate(IPMarket(PENDLE_MARKET), twapWindow);
112         } else {
113             unitPrice =
    PendlePYOracleLib.getPtToSyRate(IPMarket(PENDLE_MARKET), twapWindow);
114         }
115
116         (uint256 assetPrice, uint256 priceDecimals) =
117
    IPriceOracleMiddleware(PRICE_MIDDLEWARE).getAssetPrice(ASSET_ADDRESS);
118
119         uint256 scalingFactor = FEED_DECIMALS + priceDecimals - _decimals();
120         price = SafeCast.toInt256((unitPrice * assetPrice) / 10 **
    scalingFactor);
121
122         if (price <= 0) {
123             revert PriceOracleInvalidPrice();
124         }
125
126         time = block.timestamp;
127     }
```

In this body, only `price` and `time` are assigned. No storage or logic exists to produce meaningful `roundId`, `startedAt`, or `answeredInRound` values.

Exploit Scenario

A consumer that relies on Chainlink-like metadata will revert or skip price usages.

Consider the following attack scenario:

1. Alice interacts with Bob's protocol, which calls `latestRoundData` and requires `startedAt > 0` and `answeredInRound >= roundId` before acting on a price;
2. Alice's transaction triggers the call to this feed, which returns zero for those fields;
3. The protocol's checks fail and the transaction reverts, disrupting deposits, withdrawals, or liquidations; and

4. Bob's protocol experiences a denial of service until the feed is replaced or the checks are relaxed, risking stuck positions and operational downtime.

Recommendation

Populate Chainlink-style metadata consistently with the observed window and data provenance.

Fix the vulnerability by either:

- Maintain a synthetic, monotonically increasing `roundId` (e.g., derived from `block.number` or a counter anchored to observation windows) and set `answeredInRound = roundId` on each call; and
- Set `time` to the actual data timestamp (see the timestamp recommendation) and set `startedAt = time - TWAP_WINDOW` to reflect the start of the measurement window.

If exact Chainlink semantics are not intended, avoid exposing a Chainlink-like interface or document the deviations clearly so integrators can adapt their checks.

Go back to Findings Summary

L11: Division by zero in slippage check when USD-in delta rounds to zero

Low impact finding

Impact:	Low	Confidence:	High
Target:	contracts/fuses/universal_token_swapper/UniversalTokenSwapperFuse.sol	Type:	Denial of service

Description

Function `_checkSlippage` converts token deltas to USD in WAD via `IporMath.convertToWad`. For high effective decimal scales (`tokenDecimals + priceDecimals > 18`) and very small inputs, this conversion can floor a positive value to `0`. The subsequent ratio calculation then divides by `amountUsdInDelta`, causing a division-by-zero revert.

This creates a denial-of-service condition for dust-sized swaps or low-priced assets: otherwise valid swaps revert in the slippage check rather than being evaluated against tolerance.

Affected elements:

- Function `_checkSlippage` (denominator derived from USD-in delta)
- Library `IporMath.convertToWad` (floors when scaling down)
- Library `IporMath.division` (no zero-denominator guard)

Listing 86. Code snippet from contracts/fuses/universal_token_swapper/UniversalTokenSwapperFuse.sol

```
196 uint256 amountUsdInDelta = IporMath.convertToWad(
197     tokenInDelta_ * tokenInPrice, IERC20Metadata(tokenIn_).decimals() +
    tokenInPriceDecimals
198 );
```

```
199 uint256 amountUsdOutDelta = IporMath.convertToWad(
200     tokenOutDelta_ * tokenOutPrice, IERC20Metadata(tokenOut_).decimals() +
    tokenOutPriceDecimals
201 );
202
203 uint256 quotient = IporMath.division(amountUsdOutDelta * 1e18,
    amountUsdInDelta);
```

Listing 87. Code snippet from contracts/libraries/math/IporMath.sol

```
20 function convertToWad(uint256 value_, uint256 assetDecimals_) internal pure
   returns (uint256) {
21     if (value_ > 0) {
22         if (assetDecimals_ == WAD_DECIMALS) {
23             return value_;
24         } else if (assetDecimals_ > WAD_DECIMALS) {
25             return division(value_, BASIS_OF_POWER ** (assetDecimals_ -
   WAD_DECIMALS));
26         } else {
27             return value_ * BASIS_OF_POWER ** (WAD_DECIMALS -
   assetDecimals_);
28         }
29     } else {
30         return value_;
31     }
32 }
```

For example, with `tokenIn` having 18 decimals and `tokenInPrice` having 8 decimals, the scale is 26. If `tokenInDelta * tokenInPrice < 1e8`, the conversion returns `0`, and the following `division(amountUsdOutDelta * 1e18, amountUsdInDelta)` reverts.

This issue does not enable theft but can break orchestrated operations that include small or dust-sized swaps.

Exploit Scenario

An adversary can force slippage checks to revert for tiny trades, interrupting composite operations that rely on this fuse.

Consider the following attack scenario:

- Alice submits a trade with a very small `amountIn` (or targets an asset with a very low USD price and high price-decimal scale);
- Alice triggers the swap through a route that otherwise returns the expected `tokenOut` amount;
- The USD-in delta floors to `0` during `convertToWad`, making the denominator zero; and
- Bob's transaction reverts in `_checkSlippage`, causing a denial of service for dust-sized operations.

Recommendation

Harden the slippage calculation against zero denominators and premature truncation.

Practical mitigations:

- Add an explicit guard before the division:
- `require(amountUsdInDelta > 0, "SlippageCheck/ZeroUsdIn");`
- Preserve precision until the final ratio instead of rounding early:
- Compute the ratio using a 512-bit mulDiv-style helper so both numerator and denominator are scaled consistently before division;
- Or scale `tokenInDelta_ * tokenInPrice` and `tokenOutDelta_ * tokenOutPrice` by a common factor so neither floors to zero prior to the ratio.

Operationally, define a minimum effective input threshold (in USD WAD) below which swaps are rejected with a clear error, and add tests that exercise small-amount boundaries.

Go back to Findings Summary

L12: Enter event unit mismatch: emits minted shares as supplyAmount documented as assets

Low impact finding

Impact:	Low	Confidence:	High
Target:	contracts/fuses/euler/EulerV2SupplyFuse.sol	Type:	Logic error

Description

The event declared for entering the fuse documents `supplyAmount` as the amount of assets deposited, yet the code emits the number of shares minted. This unit mismatch can mislead off-chain indexers or risk engines that rely on emitted events to account exposures and enforce policies.

Listing 88. Code snippet from contracts/fuses/euler/EulerV2SupplyFuse.sol

```
39 /// @notice Emitted when assets are successfully deposited into an Euler V2
   vault
40 /// @param version The address of this fuse contract version
41 /// @param eulerVault The address of the Euler V2 vault receiving the deposit
42 /// @param supplyAmount The amount of assets deposited into the vault
43 /// @param subAccount The sub-account address used for the deposit
44 event EulerV2SupplyEnterFuse(address version, address eulerVault, uint256
   supplyAmount, address subAccount);
```

The documentation states that `supplyAmount` is the asset amount.

Listing 89. Code snippet from contracts/fuses/euler/EulerV2SupplyFuse.sol

```
104 /* solhint-disable avoid-low-level-calls */
105 depositedAmount = abi.decode(
106     EVC.call(
107         data_.eulerVault,
108         plasmaVault,
```

```
109         0,
110         abi.encodeWithSelector(ERC4626Upgradeable.deposit.selector,
    transferAmount, subAccount)
111     ),
112     (uint256)
113 );
114 /* solhint-enable avoid-low-level-calls */
115 emit EulerV2SupplyEnterFuse(VERSION, data_.eulerVault, depositedAmount,
    subAccount);
```

Per ERC-4626, `deposit` returns the number of shares minted, so `depositedAmount` is share-denominated. Emitting it as `supplyAmount` contradicts the event documentation. Such inconsistencies propagate to dependent systems (analytics, risk limits, monitoring) and can cause silent under- or over-reporting of exposures.

Exploit Scenario

Off-chain systems that consume `EulerV2SupplyEnterFuse` to track asset inflows can be misled by the unit mismatch and enforce the wrong policies.

Consider the following attack scenario:

1. Alice selects a vault where 1 share is worth more than 1 asset due to accrued yield.
2. Alice deposits X assets through `enter`; the event emits Y minted shares as `supplyAmount` with Y < X.
3. Bob's indexer interprets `supplyAmount` as an asset amount and underestimates the protocol's exposure by counting Y instead of X.
4. Alice repeats deposits to exceed exposure thresholds that are enforced based on events, bypassing monitoring and alerts.

Recommendation

Align the event payload and documentation with the actual units being emitted.

- Emit asset amount in `EulerV2SupplyEnterFuse`: emit `transferAmount` (the asset amount sent) as `supplyAmount` so the event matches its documentation.
- Alternatively, emit shares explicitly: keep emitting the minted share amount but rename the parameter to `sharesMinted` and update the NatSpec to describe shares, not assets.
- Optionally emit both `assetsSupplied` and `sharesMinted` for clarity if event size is acceptable.
- State units explicitly in all events and function docs.
- Provide migration notes for indexers and analytics.
- Add tests asserting event units versus ERC-4626 behavior at various exchange rates (using preview functions).

Go back to Findings Summary

L13: Arbitrary-call zap can exfiltrate any ERC20 residue via unrestricted refund sweep

Low impact finding

Impact:	Low	Confidence:	High
Target:	./contracts/zaps/ERC4626ZapInWithNativeToken.sol	Type:	Trust model

Description

Function `zapIn` executes arbitrary, caller-supplied calls using this contract as `msg.sender` and subsequently refunds any positive balances for arbitrary token addresses listed in `assetsToRefundToSender`. Because there is no allowlist for call targets/selectors and no accounting of pre-existing balances, any user can craft a transaction that causes residual tokens held by the zap to be transferred to the caller.

Two key behaviors combine to enable this:

- Arbitrary external execution using this contract as the caller; and
- Unrestricted sweep of any positive ERC20 balance for caller-chosen token addresses.

Listing 90. Code snippet from contracts/zaps/ERC4626ZapInWithNativeToken.sol

```
108         results = new bytes[](callsLength);
109         for (uint256 i; i < callsLength; i++) {
110             if (zapInData_.calls[i].nativeTokenAmount > 0) {
111                 results[i] = zapInData_.calls[i].target
112                     .functionCallWithValue(zapInData_.calls[i].data,
    zapInData_.calls[i].nativeTokenAmount);
113             } else {
114                 results[i] =
    zapInData_.calls[i].target.functionCall(zapInData_.calls[i].data);
115             }
```

```
116         }
117
118         IERC4626 vault = IERC4626(zapInData_.vault);
119         uint256 depositAssetBalance =
    IERC4626(vault.asset()).balanceOf(address(this));
120
121         if (depositAssetBalance < zapInData_.minAmountToDeposit) {
122             revert InsufficientDepositAssetBalance();
123         }
124
125         if (referralContractAddress != address(0) && referralCode_ !=
    bytes32(0)) {
126
    ReferralPlasmaVault(referralContractAddress).emitReferralForZapIn(currentZap
    Sender, referralCode_);
127         }
128
129         vault.deposit(depositAssetBalance, zapInData_.receiver);
130
131         uint256 assetsToRefundToSenderLength =
    zapInData_.assetsToRefundToSender.length;
132         address asset;
133         uint256 balance;
134
135         for (uint256 i; i < assetsToRefundToSenderLength; ++i) {
136             asset = zapInData_.assetsToRefundToSender[i];
137             balance = IERC4626(asset).balanceOf(address(this));
138             if (balance > 0) {
139                 IERC4626(asset).safeTransfer(currentZapSender, balance);
140             }
141         }
142
143         uint256 nativeTokenBalance = address(this).balance;
144
145         if (nativeTokenBalance > 0) {
146             Address.sendValue(payable(currentZapSender),
    nativeTokenBalance);
147         }
```

In practice, this lets anyone drain any ERC20 tokens accidentally sent to, or left in, the zap. For example, a previous operation's rounding leftovers, a misrouted transfer, or airdrops become withdrawable by the next arbitrary caller. The function also forwards all native-token residue unconditionally.

Exploit Scenario

An opportunistic user can drain non-zero ERC20 balances already present on the zap, or purposefully stage a minimal deposit to satisfy `minAmountToDeposit` and then sweep unrelated tokens.

Consider the following attack scenario:

1. Alice observes that the zap contract currently holds `TOKEN_X` from a prior interaction;
2. Alice crafts `zapInData` with a single benign call (e.g., a no-op or a call that transfers a minimal deposit asset to the zap), sets `minAmountToDeposit` to a trivially satisfied amount, and includes `TOKEN_X` in `assetsToRefundToSender`;
3. The zap executes the user-supplied call(s), then iterates `assetsToRefundToSender`, reads `balanceOf(this)` for `TOKEN_X`, and transfers the entire positive balance to `currentZapSender`; and
4. Bob's protocol (or users who accidentally sent tokens) loses `TOKEN_X` as it is swept by Alice.

A variant:

1. Alice first approves the zap for the deposit asset or supplies native tokens to meet `minAmountToDeposit` via a `WETH.deposit` call in `calls`;
2. Alice also includes in `calls` a `TOKEN_Y.transfer(Alice, amount)` using `functionCall` so the zap itself performs the transfer; and
3. After the arbitrary call sequence, the refund loop sweeps any remaining token balances, further extracting value.

Recommendation

Constrain arbitrary execution and scope refunds to the current zap's deltas only.

Recommended hardening steps:

- Introduce an allowlist of permissible call targets and, where feasible, function selectors. Disallow direct `transfer`/`approve` calls except to known-helper contracts.
- Track pre-balances for each token in `assetsToRefundToSender` and the native token at the start of `zapIn`, and only refund the positive delta accrued during the current invocation. Do not transfer pre-existing balances.
- Optionally require `assetsToRefundToSender` to be a subset of tokens touched by the call sequence, or maintain a curated list of refundable assets.
- Consider reverting if any unexpected positive balance remains after the operation, rather than sweeping it to the caller.
- Provide a controlled `recoverERC20` for accidental deposits if ownership is not renounced; otherwise design for zero-residuals and prevent stray balances by construction.

Go back to Findings Summary

L14: Deposit wrapper lacks minSharesOut slippage protection, enabling front-running to reduce minted shares

Low impact finding

Impact:	Low	Confidence:	Medium
Target:	./contracts/vaults/extensions/ReferralPlasmaVault.sol	Type:	Front-running

Description

The wrapper forwards deposits to arbitrary ERC4626 vaults but offers no user-specified minimum-shares safeguard. The amount of shares minted by `deposit` depends on the vault's current exchange rate and can be manipulated intra-block (e.g., via MEV deposits/withdrawals, fees, or rounding).

Because `deposit` does not accept a `minSharesOut` argument and does not bound the outcome, users can be front-run and receive fewer shares than expected.

Listing 91. Code snippet from contracts/vaults/extensions/ReferralPlasmaVault.sol

```
47 function deposit(address vault_, uint256 assets_, address receiver_, bytes32
   referralCode_)
48     external
49     returns (uint256)
50 {
51     address assetAddress = IERC4626(vault_).asset();
52     IERC20(assetAddress).safeTransferFrom(msg.sender, address(this),
   assets_);
53     IERC20(assetAddress).forceApprove(vault_, assets_);
54
55     emit ReferralEvent(msg.sender, referralCode_);
```

```
56     return IERC4626(vault_).deposit(assets_, receiver_);
57 }
```

Exploit Scenario

An MEV participant can skew the vault's share price around a victim's transaction to reduce the shares the victim receives.

Consider the following attack scenario:

1. Bob submits a transaction to deposit 1,000 units into a price-sensitive ERC4626 vault via this wrapper.
2. Alice observes the mempool and front-runs with a small deposit/withdraw sequence that worsens the exchange rate for the next depositor (Bob).
3. Bob's transaction executes next; `deposit` mints fewer shares than Bob anticipated because the rate moved against him.
4. Alice then completes her sequence (e.g., withdraws) to restore the prior state while pocketing the value extracted from Bob's worse rate.
5. The wrapper returns the reduced shares to Bob; without `minSharesOut`, Bob cannot revert the transaction and suffers a deterministic value loss.

Recommendation

Add explicit slippage controls and optional timing bounds:

- Extend the function signature to accept `uint256 minSharesOut` and enforce `require(sharesOut >= minSharesOut, "slippage")` after calling `deposit`.
- Optionally, support a `deadline` parameter to prevent inclusion in later blocks.
- For users that prefer assets-bounded semantics, offer a complementary `mint` variant with a `maxAssetsIn` parameter and require the vault to respect

it.

- Document that interacting with dynamic/fee-bearing ERC4626 vaults without slippage parameters exposes users to MEV/front-running risk.

Go back to Findings Summary

L15: Trusting distributor return value can strand surplus rewards in MorphoClaimFuse

Low impact finding

Impact:	Low	Confidence:	Medium
Target:	./contracts/rewards_fuses/morpho/MorphoClaimFuse.sol	Type:	Trust model

Description

The `claim` function relies on the return value of `IUniversalRewardsDistributor.claim` to decide how many reward tokens to forward. It does not compute the actual amount received using a balance delta, and it does not sweep any residual token balances held by the fuse.

If the distributor transfers more tokens than it reports via the return value (for example due to a bug, rounding/truncation discrepancy, adversarial upgrade of the distributor, or any unexpected transfer that occurs during the external call), the surplus remains stranded on `MorphoClaimFuse`. Subsequent claims also forward only the newly returned value, not the accumulated on-contract balance. The contract exposes no rescue/sweep path, so such tokens cannot be recovered by the system.

Realistic impact in production:

- Under nominal conditions, the official distributor should return the exact amount transferred. However, this assumption creates a single point of failure: a misimplementation, misconfiguration, or adversarial upgrade of the whitelisted distributor would silently short-forward rewards. The loss manifests as rewards being stuck on this fuse rather than reaching the `rewardsClaimManager`, leading to under-distribution and accounting mismatch until the fuse is replaced or upgraded.

- Because the function only transfers `rewardsAmount`, any previously stuck balance on the fuse is not forwarded in future claims, compounding the discrepancy.

Listing 92. Code snippet from contracts/rewards_fuses/morpho/MorphoClaimFuse.sol

```
90 uint256 rewardsAmount =
   IUniversalRewardsDistributor(universalRewardsDistributor)
91     .claim(address(this), rewardsToken, claimable, proof);
92
93 if (rewardsAmount > 0) {
94     IERC20(rewardsToken).safeTransfer(rewardsClaimManager, rewardsAmount);
95
96     emit MorphoClaimFuseRewardsClaimed(VERSION, rewardsToken, rewardsAmount,
   rewardsClaimManager);
97 }
```

Evidence and observations:

- The forwarding amount equals the external return value, not the balance delta of `rewardsToken` held by this contract.
- There is no `sweep`, `rescue`, or similar function in `MorphoClaimFuse` to recover arbitrary token balances held by the fuse.

Exploit Scenario

A malicious or buggy distributor can cause the fuse to forward fewer tokens than it actually receives, stranding the surplus on the fuse with no recovery path.

Consider the following attack scenario:

1. Alice operates or influences a whitelisted `UniversalRewardsDistributor` for the relevant `MARKET_ID`.
2. Alice deploys or upgrades the distributor so that it transfers 120 `rewardsToken` to the claimant but returns 100 from `claim` (e.g., due to

adversarial logic or a bug).

3. Alice (or anyone) calls `MorphoClaimFuse.claim` with a valid `proof` for `rewardsToken`.
4. The fuse executes `claim`, receives 120 tokens, but forwards only the returned `rewardsAmount` of 100 to `rewardsClaimManager`; the extra 20 remain stranded on `MorphoClaimFuse`.
5. Bob, representing the `rewardsClaimManager` and downstream recipients, receives fewer rewards than available, while the stranded balance accumulates on the fuse and cannot be swept.

Recommendation

Adopt balance-delta accounting and/or actively drain any residual rewards balance after claiming.

Fix the vulnerability by either:

- Compute the claimed amount via pre/post balance delta and forward exactly the delta, avoiding reliance on the distributor's return value. For example, capture `balanceBefore`, invoke `claim`, and forward `IERC20(rewardsToken).balanceOf(address(this)) - balanceBefore` to `rewardsClaimManager`.
- Alternatively, after `claim` forward the entire `IERC20(rewardsToken).balanceOf(address(this))` to `rewardsClaimManager` to ensure no dust remains on the fuse.
- Optionally, add a governance-controlled `rescueTokens(address token, uint256 amount, address to)` function that can sweep unexpected balances to a safe recipient if any residue still appears due to third-party transfers.

Additional hardening (optional):

- If you keep using the return value, assert invariant consistency, e.g., revert when `balanceDelta > rewardsAmount` to prevent silent short-forwarding.

Go back to Findings Summary

L16: Unrestricted external multicall lets anyone transfer tokens owned by the zap contract

Low impact finding

Impact:	Low	Confidence:	Medium
Target:	./contracts/zaps/ERC4626ZapInWithNativeToken.sol	Type:	Trust model

Description

The `zapIn` loop executes an unbounded sequence of user-supplied external calls as this contract (`msg.sender` context), optionally forwarding native ETH. Any caller can invoke `ERC20.transfer` or `ERC20.transferFrom` on tokens for which this contract holds balances or allowances, moving funds to arbitrary recipients. This creates a cross-user leakage vector for any tokens inadvertently left on the contract from previous zaps or mistaken transfers.

Affected elements:

- Function `zapIn` executing arbitrary calls with `Address.functionCall` and `functionCallWithValue`;
- State variable `currentZapSender` is unrelated to this movement, as the calls run with contract authority; and
- No target whitelist or selector filtering is enforced.

Listing 93. Code snippet from contracts/zaps/ERC4626ZapInWithNativeToken.sol

```
108         results = new bytes[](callsLength);
109         for (uint256 i; i < callsLength; i++) {
110             if (zapInData_.calls[i].nativeTokenAmount > 0) {
111                 results[i] = zapInData_.calls[i].target
112                     .functionCallWithValue(zapInData_.calls[i].data,
    zapInData_.calls[i].nativeTokenAmount);
```

```
113             } else {
114                 results[i] =
    zapInData_.calls[i].target.functionCall(zapInData_.calls[i].data);
115             }
116         }
```

By pointing `target` at an ERC20 token and encoding `transfer(attacker, balanceOf(this))`, the caller can drain any token balance this contract holds before the later refund loop executes. This behavior turns the zap into a generic proxy capable of moving its own assets for any caller.

Exploit Scenario

A caller can move arbitrary ERC20 balances held by the zap contract via the multicall loop.

Consider the following attack scenario:

1. Bob runs `zapIn` and, due to call specifics, `TOKEN` is left in the zap contract.
2. Alice submits her own `zapIn` and, in `calls[0]`, sets `target = TOKEN` and `data = abi.encodeWithSelector(TOKEN.transfer.selector, alice, TOKEN.balanceOf(address(this)))`.
3. The loop executes `Address.functionCall(target, data)`, which calls `TOKEN.transfer(alice, balance)` with the zap contract as `msg.sender`.
4. The zap transfers all its `TOKEN` balance to Alice before any refunds, stealing Bob's residual funds.

Recommendation

Constrain execution to vetted targets/selectors or remove contract-authority calls.

Fix the vulnerability by either:

- Enforcing an allowlist of call targets and permitted function selectors that

cannot move balances held by the zap contract (for example, only specific router functions); or

- Executing swaps/operations through a dedicated allowance-holder helper (e.g., `ZAP_IN_ALLOWANCE_CONTRACT`) that never holds funds and revokes allowances post-call, while forbidding direct calls from the zap contract to arbitrary targets; or
- Rejecting calls that encode ERC20 `transfer`, `transferFrom`, or `approve` selectors and any other balance-moving functions; or
- Restructuring the design so user actions are performed with user-owned allowances/permits (e.g., Permit2), avoiding the zap holding residual balances.

These mitigations prevent arbitrary movement of the zap's own token balances by untrusted callers.

Go back to Findings Summary

L17: Division-by-zero in slippage verification for dust-sized swaps (DoS)

Low impact finding

Impact:	Low	Confidence:	Medium
Target:	./contracts/fuses/universal_token_swapper/UniversalTokenSwapperWithVerificationFuse.sol	Type:	Denial of service

Description

The slippage verification inside function `_executeSwap` divides by `amountUsdInDelta` that is computed from the input token delta and its USD price using `IporMath.convertToWad`. When the traded notional is extremely small (for example, `result.tokenInDelta == 1` wei and `tokenInPrice` is low), the conversion to WAD can round down to zero. This makes `amountUsdInDelta == 0` and the subsequent division reverts with an arithmetic error.

Prices returned by `IPriceOracleMiddleware` use `QUOTE_CURRENCY_DECIMALS = 18`. Therefore, `assetDecimals_` passed into `IporMath.convertToWad` is `IERC20Metadata(token).decimals() + 18`. For typical 18-decimal tokens this converts by dividing by `10^(36-18) = 1e18`, truncating toward zero. If `result.tokenInDelta * tokenInPrice < 1e18`, the converted USD amount becomes zero.

Affected functions:

- `enter` and `enterTransient` via the shared internal implementation `_executeSwap`;
- slippage comparison against `SLIPPAGE_REVERSE` in `_executeSwap`.

Illustrative example: `tokenIn` has 18 decimals and is priced at $0.10

(`tokenInPrice = 1e17`). For `result.tokenInDelta = 1`, the product is `1 * 1e17`. Converting to WAD with `assetDecimals_ = 36` yields `1e17 / 1e18 = 0`. The code then executes `IporMath.division(amountUsdOutDelta * 1e18, amountUsdInDelta)`, dividing by zero and reverting. This creates an avoidable denial-of-service for dust-sized swaps.

Listing 94. Code snippet from contracts/fuses/universal_token_swapper/UniversalTokenSwapperWithVerificationFuse.sol

```
196         uint256 amountUsdInDelta = IporMath.convertToWad(
197             result.tokenInDelta * tokenInPrice,
    IERC20Metadata(swapData.tokenIn).decimals() + tokenInPriceDecimals
198         );
199         uint256 amountUsdOutDelta = IporMath.convertToWad(
200             result.tokenOutDelta * tokenOutPrice,
    IERC20Metadata(swapData.tokenOut).decimals() + tokenOutPriceDecimals
201         );
202
203         uint256 quotient = IporMath.division(amountUsdOutDelta * 1e18,
    amountUsdInDelta);
204
205         if (quotient < SLIPPAGE_REVERSE) {
206             revert UniversalTokenSwapperFuseSlippageFail();
207         }
```

The rounding behavior that causes `amountUsdInDelta` to become zero originates from `IporMath.convertToWad` for assets with combined decimals greater than 18, and the division helper simply forwards Solidity's integer division.

Listing 95. Code snippet from contracts/libraries/math/IporMath.sol

```
20     function convertToWad(uint256 value_, uint256 assetDecimals_) internal
   pure returns (uint256) {
21         if (value_ > 0) {
22             if (assetDecimals_ == WAD_DECIMALS) {
23                 return value_;
24             } else if (assetDecimals_ > WAD_DECIMALS) {
25                 return division(value_, BASIS_OF_POWER ** (assetDecimals_ -
```

```
    WAD_DECIMALS));
            } else {
                return value_ * BASIS_OF_POWER ** (WAD_DECIMALS -
    assetDecimals_);
            }
        } else {
            return value_;
        }
    }
```

Listing 96. Code snippet from contracts/libraries/math/IporMath.sol

```
    function division(uint256 x_, uint256 y_) internal pure returns (uint256
    z_) {
        z_ = x_ / y_;
    }
```

Exploit Scenario

An attacker can make the slippage verification path revert by forcing `amountUsdInDelta` to round to zero.

Consider the following attack scenario:

1. Alice observes that the contract holds a small balance of `tokenIn` and that its USD price is low enough to make dust-sized trades round to zero after conversion.
2. Alice ensures a minimal balance is present (if needed) by transferring 1 wei of `tokenIn` to the contract.
3. Alice calls `enter` with `amountIn = 1` and a no-op or benign `SwapExecutorEthData` so that the swap execution itself does not revert.
4. After the external calls return, `result.tokenInDelta` is 1. `amountUsdInDelta = convertToWad(1 * tokenInPrice, 18 + 18)` evaluates to 0 due to truncation.
5. The code computes `quotient = IporMath.division(amountUsdOutDelta * 1e18, amountUsdInDelta)` which divides by zero and reverts.

6. Bob's protocol transaction fails and wastes gas. Repeating the call causes persistent transaction-level DoS until the logic is fixed or dust trades are filtered.

Recommendation

Add an explicit guard before the division and consider computing the slippage ratio without an intermediate WAD conversion that can round tiny notionals to zero.

Fix the vulnerability by either:

- Failing fast on zero notional after conversion:
- Compute `amountUsdInDelta` and immediately revert with `UniversalTokenSwapperFuseSlippageFail` if it equals 0, before any division.
- Avoiding a zero denominator entirely by computing the ratio in high precision on raw amounts and prices:
- Use a 512-bit safe mul-div helper (for example, OpenZeppelin `Math.mulDiv`) and compute: `quotient = Math.mulDiv(result.tokenOutDelta * tokenOutPrice, 1e18, result.tokenInDelta * tokenInPrice);`

  > This keeps the denominator > 0 as long as `result.tokenInDelta > 0` and the price feed > 0, and avoids the lossy intermediate WAD conversion.

- Optionally enforcing a minimum input notional to prevent dust-sized swaps from entering this path at all:
- Require `amountUsdInDelta > 0` or `amountIn >= minNotional(tokenIn)` where `minNotional` is computed from current price and token decimals.

These changes ensure that slippage verification reverts deterministically with the contract's custom error (rather than an arithmetic division-by-zero) and close the transaction-level DoS vector.

Go back to Findings Summary

W1: Missing length equality check between fuse and inputsByFuse enables out-of-bounds panic in enter

Impact:	Warning	Confidence:	High
Target:	contracts/fuses/transient_storage/TransientStorageSetInputsFuse.sol	Type:	Data validation

Description

The `enter` function iterates over `data_.fuse.length` and dereferences `data_.inputsByFuse[i]` inside the loop without first validating that `data_.inputsByFuse.length` equals `data_.fuse.length`.

Affected surface:

- Function: `enter` in contract `TransientStorageSetInputsFuse`;
- Parameters: `data_.fuse` (address[]) and `data_.inputsByFuse` (bytes32[][]);
- No persistent state variables are modified; the issue is read-time validation and error signaling.

In Solidity 0.8.x, accessing `data_.inputsByFuse[i]` when `i >= data_.inputsByFuse.length` triggers a Panic(0x32) due to an array index out-of-bounds before the custom error `WrongInputsLength` can be evaluated. As a result, callers receive a generic panic revert code instead of the intended domain-specific revert, which breaks error handling and diagnosability in composed flows that expect `WrongInputsLength`.

This is a pure validation/robustness flaw. While it does not directly create a loss-of-funds condition, it degrades reliability and can cause denial-of-service of higher-level aggregations that rely on predictable error signaling.

Listing 97. Code snippet from contracts/fuses/transient_storage/TransientStorageSetInputsFuse.sol

```
29 uint256 len = data_.fuse.length;
30 for (uint256 i; i < len; ++i) {
31     if (data_.fuse[i] == address(0)) {
32         revert WrongFuseAddress();
33     }
34     if (data_.inputsByFuse[i].length == 0) {
35         revert WrongInputsLength();
36     }
37     TransientStorageLib.setInputs(data_.fuse[i], data_.inputsByFuse[i]);
38 }
```

Exploit Scenario

A caller can force a generic Panic(0x32) by supplying mismatched top-level array lengths so that the loop indexes `data_.inputsByFuse[i]` out of bounds.

Consider the following attack scenario:

1. Alice crafts a call to `enter` with `data_.fuse = [F1, F2]` and `data_.inputsByFuse =` (length 1), ensuring `data_.inputsByFuse[0]` is non-empty so the first iteration passes the zero-length check.
2. Alice submits the transaction; the function sets `len = data_.fuse.length = 2` and starts iterating.
3. On iteration `i = 1`, the code evaluates `data_.inputsByFuse[1].length` before any equality validation, which triggers Solidity Panic(0x32) (array index out-of-bounds).
4. The transaction reverts with a generic panic instead of `WrongInputsLength`, causing upstream aggregators or orchestrators to mis-handle the error and potentially fail entire composed operations.

Realistic impact in production:

- Composed “multi-fuse” or router-style calls become brittle: instead of receiving a stable custom error that integration code can handle and report, they observe a low-level panic revert;
- Operators and monitoring systems lose precise failure diagnostics; and
- Users pay gas for failed transactions that could have been rejected earlier with deterministic validation.

Recommendation

Validate top-level array lengths before indexing `data_.inputsByFuse[i]`, and only then perform per-item checks. This prevents out-of-bounds panics and guarantees consistent custom error signaling.

Fix the vulnerability by either:

- Adding an upfront equality check before the loop: `if (data_.inputsByFuse.length != data_.fuse.length) revert WrongInputsLength();`
- Reordering validations to avoid dereferencing `data_.inputsByFuse[i]` until the length relationship is known.

A corrected pattern inside `enter` could look like:

- Cache `len = data_.fuse.length`;
- Check `if (data_.inputsByFuse.length != len) revert WrongInputsLength();`;
- Iterate and validate each `data_.fuse[i] != address(0)` and `data_.inputsByFuse[i].length > 0`;
- Call `TransientStorageLib.setInputs(data_.fuse[i], data_.inputsByFuse[i])`.

This ensures any mismatch is rejected deterministically at the start of the function and avoids Panic(0x32) paths.

Go back to Findings Summary

W2: Comment-implementation mismatch in _convertValue (no early return for same type and decimals)

Impact:	Warning	Confidence:	High
Target:	contracts/fuses/transient_storage/TransientStorageMapperFuse.sol	Type:	Code quality

Description

The inline comment says the function returns the value unchanged when "types are UNKNOWN or the same with same decimals," but the code only checks the `UNKNOWN` case. This documentation drift can mislead maintainers and reviewers, hides a real behavioral discrepancy, and increases the probability of introducing or overlooking functional bugs.

Listing 98. Code snippet from contracts/fuses/transient_storage/TransientStorageMapperFuse.sol

```
92          // If types are UNKNOWN or the same with same decimals, return value
    unchanged
93          if (fromType_ == DataType.UNKNOWN || toType_ == DataType.UNKNOWN) {
94              return value_;
95          }
```

Exploit Scenario

While this is a code-quality issue, the mismatch can be leveraged by relying parties who assume identity behavior from the comment.

Consider the following attack scenario:

1. Alice integrates against the comment's promise and configures a same-type/same-decimals mapping, expecting the value to pass through

unchanged.

2. The function executes a conversion round-trip instead of returning early, altering the value's representation (for example, normalization or narrowing as per the type family).
3. Bob's downstream logic operates on the altered representation, allowing an operation that would have been blocked if the original value were preserved.

Recommendation

Align comments and implementation to remove ambiguity.

Fix the issue by either:

- Implement the missing early return for `(fromType_ == toType_ && fromDecimals_ == toDecimals_)` to match the comment; or
- Update the comment to accurately describe that a conversion will always occur except for `UNKNOWN` types, and document the implications.

As a safeguard, add tests that assert expectations for same-type/same-decimals mappings so documentation divergence is caught early.

Go back to Findings Summary

W3: Validator blocks `flashLoan`: documentation claims support but `_validateEulerVault` rejects it

Impact:	Warning	Confidence:	High
Target:	contracts/fuses/euler/EulerV2BatchFuse.sol	Type:	Logic error

Description

The documentation of `enter` claims support for "flash loan operations", but the validator `_validateEulerVault` does not allow `IBorrowing.flashLoan`. Only `borrow`, `repay`, `repayWithShares`, `deposit`, `withdraw`, and `disableController` are accepted; any unhandled selector reverts with `UnsupportedOperation`.

Affected elements:

- Function `_validateEulerVault` (selector gate for Euler vault calls);
- Function `_validatePlasmaVaultCallback` (permits only `CallbackHandlerEuler.onEulerFlashLoan` as a callback, but there is no allowance to initiate `flashLoan` itself);
- NatSpec of `enter` overstates supported operations.

Impact:

- Any batch that includes a `flashLoan` call reverts, contradicting the public contract documentation and causing unexpected integration failures.

Listing 99. Code snippet from contracts/fuses/euler/EulerV2BatchFuse.sol

```
217 /// @notice Validates Euler Vault batch items for various operations
218 /// @dev Supports borrow, repay, deposit, withdraw, and disableController
    operations with proper permission checks
219 /// @param item_ The batch item to validate
220 function _validateEulerVault(EulerV2BatchItem memory item_) internal view {
221     bytes4 selector = bytes4(_slice(item_.data, 0, 4));
```

```
222     bytes memory dataAfterSelector = _slice(item_.data, 4, item_.data.length
    - 4);
223     if (selector == IBorrowing.borrow.selector) {
224         (, address subaccount) = abi.decode(dataAfterSelector, (uint256,
    address));
225         bool canBorrow = EulerFuseLib.canBorrow(MARKET_ID,
    item_.targetContract, item_.onBehalfOfAccount);
226         if (
227             !canBorrow
228                 || subaccount !=
    EulerFuseLib.generateSubAccountAddress(address(this),
    item_.onBehalfOfAccount)
229         ) {
230             revert EulerVaultInvalidPermissions();
231         }
232     } else if (selector == IBorrowing.repay.selector || selector ==
    IBorrowing.repayWithShares.selector) {
233         (, address subaccount) = abi.decode(dataAfterSelector, (uint256,
    address));
234         bool canRepay = EulerFuseLib.canBorrow(MARKET_ID,
    item_.targetContract, item_.onBehalfOfAccount);
235         if (
236             !canRepay
237                 || subaccount !=
    EulerFuseLib.generateSubAccountAddress(address(this),
    item_.onBehalfOfAccount)
238         ) {
239             revert EulerVaultInvalidPermissions();
240         }
241     } else if (selector == ERC4626Upgradeable.deposit.selector) {
242         (, address subaccount) = abi.decode(dataAfterSelector, (uint256,
    address));
243         bool canSupply = EulerFuseLib.canSupply(MARKET_ID,
    item_.targetContract, item_.onBehalfOfAccount);
244         if (
245             !canSupply
246                 || subaccount !=
    EulerFuseLib.generateSubAccountAddress(address(this),
    item_.onBehalfOfAccount)
247         ) {
248             revert EulerVaultInvalidPermissions();
249         }
250     } else if (selector == ERC4626Upgradeable.withdraw.selector) {
251         (, address subaccount) = abi.decode(dataAfterSelector, (uint256,
    address));
252         // For withdraw operations, we should check if the vault can supply
    (has the asset)
```

```
253         // and validate the subaccount matches
254         bool canSupply = EulerFuseLib.canSupply(MARKET_ID,
    item_.targetContract, item_.onBehalfOfAccount);
255         if (
256             !canSupply
257                 || subaccount !=
    EulerFuseLib.generateSubAccountAddress(address(this),
    item_.onBehalfOfAccount)
258         ) {
259             revert EulerVaultInvalidPermissions();
260         }
261     } else if (selector == IVault.disableController.selector) {
262         bool canSupply = EulerFuseLib.canSupply(MARKET_ID,
    item_.targetContract, item_.onBehalfOfAccount);
263         if (!canSupply) {
264             revert EulerVaultInvalidPermissions();
265         }
266     } else {
267         revert UnsupportedOperation();
268     }
269 }
```

Exploit Scenario

A batch containing a flash loan will always fail because the selector is not allowed by the validator.

Consider the following sequence:

1. Alice integrates this fuse assuming flash loans are supported, as stated in the `enter` NatSpec.
2. Alice constructs a batch item for a vault with `IBorrowing.flashLoan.selector` and includes it in `EulerV2BatchFuseData.batchItems`.
3. The fuse calls `_validateEulerVault`, which does not recognize the `flashLoan` selector and falls into the final `else` branch.
4. The transaction reverts with `UnsupportedOperation`, wasting gas and preventing intended atomic execution.

5. Bob's protocol pipeline that depends on flash loans is effectively denied service by this fuse.

Result: integrations that rely on documented flash-loan support experience unexpected reverts and cannot execute planned strategies, and Bob's protocol absorbs avoidable gas costs and failures in production.

Recommendation

Bring documentation and logic into alignment:

- If flash loans should be supported, add a validation branch in `_validateEulerVault` for `IBorrowing.flashLoan.selector`, including permission checks analogous to `borrow`/`repay` (e.g., confirm the `subaccount` is `EulerFuseLib.generateSubAccountAddress(address(this), item_.onBehalfOfAccount)` and any policy constraints). Ensure the corresponding callback path is allowed and tested end-to-end; or
- If flash loans are intentionally not supported by this fuse, update the `enter` NatSpec to remove "flash loan operations" from the list of supported actions.

Additionally, add unit tests that exercise both the happy path and the rejection path for each selector to prevent regressions in supported-operation sets.

Go back to Findings Summary

W4: NatSpec claims reentrancy protection in `enter`, but no guard is implemented

Impact:	Warning	Confidence:	High
Target:	./contracts/fuses/euler/EulerV2BatchFuse.sol	Type:	Code quality

Description

The `enter` function's NatSpec states "This function includes reentrancy protection", but the function is not protected by any `nonReentrant`-style guard and the contract does not inherit `ReentrancyGuard`. The function performs multiple external calls (token `approve` via `SafeERC20.forceApprove` and `EVC.batch`) without synchronizing re-entrancy, which contradicts the documentation and can mislead future maintainers to rely on non-existent protections.

Affected elements:

- Function `enter`;
- External calls to `ERC20.forceApprove` and `EVC.batch`;
- No modifiers or guards are present on the contract (Wake: list_modifiers = none).

Realistic impact:

- Today, practical reentrancy impact is limited because the function does not mutate contract storage and approvals are reset immediately after `EVC.batch` completes. However, the inaccurate security annotation materially raises the risk that a later change adds stateful logic or relies on a presumed guard and accidentally introduces a reentrancy vulnerability.

Listing 100. Code snippet from contracts/fuses/euler/EulerV2BatchFuse.sol

```
53 /// @notice Executes a batch of Euler V2 operations including supply, borrow,
   repay, and flash loan operations
54 /// @dev This function validates all batch items, sets up approvals, executes
   the batch via EVC, and cleans up approvals
55 /// @param data_ The data structure containing batch items and approval
   configurations
56 /// @return batchSize The number of batch items executed
57 /// @return assets The array of assets used for approvals
58 /// @return vaults The array of Euler vaults used for approvals
59 /// @custom:security This function includes reentrancy protection and
   comprehensive validation
60 function enter(EulerV2BatchFuseData memory data_)
61     public
62     returns (uint256 batchSize, address[] memory assets, address[] memory
   vaults)
63 {
```

Exploit Scenario

A misleading security annotation can translate into a concrete exploit once stateful logic is added under the assumption that `enter` is reentrancy-protected.

Consider the following sequence:

1. Alice deploys a malicious ERC20 token that, when `approve` is called, performs a callback that re-enters `enter` on this fuse.
2. Alice transfers tokens to the fuse contract and submits a batch where `assetsForApprovals` includes her malicious token and a vault that can pull funds.
3. During the first `enter`, the loop calls `forceApprove` for Alice's token; the token re-enters `enter` before approvals are cleaned up.
4. The re-entrant `enter` executes an attacker-controlled `EVC.batch` while allowances are partially set from the first call, letting the vault pull funds

twice before approvals are reset.

Result: Bob's protocol can be double-charged or have funds drained from the fuse contract during the overlapping approval window. Even though today the function does not mutate storage, allowances and external calls can be interleaved in unexpected ways. The absence of an actual guard defeats the expectation set by the documentation and widens the attack surface for future changes.

Recommendation

Align documentation and implementation to remove ambiguity:

- If reentrancy protection is desired, add `ReentrancyGuard` from OpenZeppelin and mark `enter` with `nonReentrant`. Keep the current checks-effects-interactions order and keep approvals as short-lived as possible; or
- If the function is intentionally not guarded, delete the `@custom:security` claim and add a note that the function is safe today because it does not mutate storage and resets approvals at the end, and that any future state changes should re-evaluate the need for a guard.

Additional hardening (optional):

- Minimize approval windows by approving only the exact amounts required for the batch (rather than `type(uint256).max`) and approving per-asset right before the specific operation that needs it.

Go back to Findings Summary

W5: Arbitrary approves via multicall persist allowances that enable future drains of newly received tokens

Impact:	Warning	Confidence:	Medium
Target:	./contracts/zaps/ERC4626ZapInWithNativeToken.sol	Type:	Trust model

Description

Through the same unrestricted call loop, a caller can execute `ERC20.approve(spender, type(uint256).max)` from the zap contract for any token. These allowances persist after the transaction completes. If the zap contract later receives those tokens (through residuals, airdrops, or future user flows), the approved `spender` can unilaterally call `transferFrom` to drain the funds without any further interaction from the zap.

Affected elements:

- Function `zapIn` arbitrary calls with `Address.functionCall`;
- No allowance cleanup is performed anywhere after execution; and
- No whitelist/selector filtering prevents `approve` calls.

Listing 101. Code snippet from contracts/zaps/ERC4626ZapInWithNativeToken.sol

```
108         results = new bytes[](callsLength);
109         for (uint256 i; i < callsLength; i++) {
110             if (zapInData_.calls[i].nativeTokenAmount > 0) {
111                 results[i] = zapInData_.calls[i].target
112                     .functionCallWithValue(zapInData_.calls[i].data,
    zapInData_.calls[i].nativeTokenAmount);
113             } else {
114                 results[i] =
    zapInData_.calls[i].target.functionCall(zapInData_.calls[i].data);
115             }
```

```
116         }
```

This enables a persistent allowance poisoning vector against the contract's future holdings.

Exploit Scenario

A malicious caller can set unlimited allowances and later drain tokens when the zap contract receives them.

Consider the following attack scenario:

1. Alice calls `zapIn` with a first call that targets `TOKEN` and encodes `approve(attacker, type(uint256).max)`, executed by the zap contract.
2. The transaction completes; no tokens are immediately moved, but the allowance remains in place on `TOKEN` for `attacker`.
3. Sometime later, Bob uses `zapIn` and, after his operations, the zap contract temporarily holds a `TOKEN` balance.
4. The attacker calls `TOKEN.transferFrom(address(zap), attacker, TOKEN.balanceOf(address(zap)))` to drain the funds, without interacting with the zap.

Recommendation

Disallow or neutralize `approve`-like calls and revoke any allowances created during execution.

Fix the vulnerability by either:

- Rejecting calldata that encodes ERC20 `approve` (and `permit` where applicable) in the multicall loop; or
- Executing all third-party interactions via a dedicated allowance-holder helper that revokes allowances after use and never holds balances; or

- Recording allowances granted during the call and force-resetting them to zero before returning; or
- Maintaining an allowlist of tokens/spenders and maximum allowance values, forbidding arbitrary approvals.

These controls prevent persistent allowance poisoning and eliminate the path for future unauthorized drains.

Go back to Findings Summary

W6: EIP-7201 storage slots contradict comments; constants are contiguous sub-slots from one namespace

Impact:	Warning	Confidence:	Medium
Target:	contracts/fuses/whitelist/FuseWhitelistLib.sol	Type:	Code quality

Description

This library uses the EIP-7201 namespaced storage pattern but the declared storage slot constants contradict the inline documentation.

The comments for each constant state that the slot is computed as `keccak256(abi.encode(uint256(keccak256("<namespace>")) - 1)) & ~bytes32(uint256(0xff))` for a distinct `<namespace>` per structure. If this were true, the resulting 32-byte constants would differ in more than just the least significant byte (LSB).

In the implementation, however, all six constants share the exact same high 31 bytes and only differ by the LSB, i.e. `…edd00`, `…edd01`, `…edd02`, `…edd03`, `…edd04`, `…edd05`. This indicates they are contiguous sub-slots taken from a single masked namespace, not independently derived per-struct hashes as the comments suggest. This mismatch can mislead maintainers about how slots were produced and about the potential for collisions if multiple libraries are ever composed into a single storage context (e.g., a facet-based system).

Affected constants in this file

- `FUSES_TYPES`
- `FUSES_STATES`
- `FUSE_INFO`
- `FUSE_INFO_BY_MARKET_ID`

- FUSE_LISTS_BY_TYPE
- METADATA_TYPES

Listing 102. Code snippet from contracts/fuses/whitelist/FuseWhitelistLib.sol

```
140     /// @notice Storage slot for FusesTypes struct
141     /// @dev Storage slot calculation:
142     ///
    keccak256(abi.encode(uint256(keccak256("io.ipor.whitelists.fuseTypes")) -
    1)) & ~bytes32(uint256(0xff))
143     bytes32 private constant FUSES_TYPES =
    0xefe839ce0caa5648581e30daa19dcc84419e945902cc17f7f481f056193edd00;
144     /// @notice Storage slot for FusesStates struct
145     /// @dev Storage slot calculation:
146     ///
    keccak256(abi.encode(uint256(keccak256("io.ipor.whitelists.fuseStates")) -
    1)) & ~bytes32(uint256(0xff))
147     bytes32 private constant FUSES_STATES =
    0xefe839ce0caa5648581e30daa19dcc84419e945902cc17f7f481f056193edd01;
148     /// @notice Storage slot for FuseInfo struct
149     /// @dev Storage slot calculation:
150     ///
    keccak256(abi.encode(uint256(keccak256("io.ipor.whitelists.fuseInfo")) - 1))
    & ~bytes32(uint256(0xff))
151     bytes32 private constant FUSE_INFO =
    0xefe839ce0caa5648581e30daa19dcc84419e945902cc17f7f481f056193edd02;
152     /// @notice Storage slot for FuseListsByType struct
153     /// @dev Storage slot calculation:
154     ///
    keccak256(abi.encode(uint256(keccak256("io.ipor.whitelists.fuseListsByType")
    ) - 1)) & ~bytes32(uint256(0xff))
155     bytes32 private constant FUSE_LISTS_BY_TYPE =
    0xefe839ce0caa5648581e30daa19dcc84419e945902cc17f7f481f056193edd04;
156     /// @notice Storage slot for FuseInfoByMarketId struct
157     /// @dev Storage slot calculation:
158     ///
    keccak256(abi.encode(uint256(keccak256("io.ipor.whitelists.fuseInfoByMarketI
    d")) - 1)) & ~bytes32(uint256(0xff))
159     bytes32 private constant FUSE_INFO_BY_MARKET_ID =
160         0xefe839ce0caa5648581e30daa19dcc84419e945902cc17f7f481f056193edd03;
161     /// @notice Storage slot for MetadataTypes struct
162     /// @dev Storage slot calculation:
163     ///
    keccak256(abi.encode(uint256(keccak256("io.ipor.whitelists.metadataTypes"))
```

```
    - 1)) & ~bytes32(uint256(0xff))
164     bytes32 private constant METADATA_TYPES =
    0xefe839ce0caa5648581e30daa19dcc84419e945902cc17f7f481f056193edd05;
```

These constants are then used via inline assembly to assign `.slot` for the corresponding storage structs. This makes the constant values authoritative for where data is written in the calling contract's storage.

Listing 103. Code snippet from contracts/fuses/whitelist/FuseWhitelistLib.sol

```
565     function _getFusesTypesSlot() private pure returns (FusesTypes storage
    fusesTypes) {
566         assembly {
567             fusesTypes.slot := FUSES_TYPES
568         }
569     }
570
571     function _getFuseInfoByMarketIdSlot() private pure returns
    (FuseInfoByMarketId storage fuseInfoByMarketId) {
572         assembly {
573             fuseInfoByMarketId.slot := FUSE_INFO_BY_MARKET_ID
574         }
575     }
576
577     function _getFusesStatesSlot() private pure returns (FusesStates storage
    fusesStates) {
578         assembly {
579             fusesStates.slot := FUSES_STATES
580         }
581     }
582
583     function _getFuseListsByTypeSlot() private pure returns (FuseListsByType
    storage fuseListsByType) {
584         assembly {
585             fuseListsByType.slot := FUSE_LISTS_BY_TYPE
586         }
587     }
588
589     function _getMetadataTypesSlot() private pure returns (MetadataTypes
    storage metadataTypes) {
590         assembly {
591             metadataTypes.slot := METADATA_TYPES
592         }
```

```
593     }
594
595     function _getFuseListByAddressSlot() private pure returns
    (FuseListByAddress storage fuseInfo) {
596         assembly {
597             fuseInfo.slot := FUSE_INFO
598         }
599     }
```

Exploit Scenario

There is no direct exploit in the current library in isolation. The risk emerges if multiple libraries using the same base namespace are ever composed into a single storage context (for example, linked into the same logic contract or diamond facet).

Consider the following attack scenario:

1. Alice deploys or upgrades a composite logic contract that uses both this library and another library that also assigns a sub-slot at `…edd00` under the same 31-byte namespace;
2. Alice calls a function that writes to `FUSES_TYPES` (e.g., `addFuseType`) so that data are stored at the `…edd00` sub-slot;
3. The contract later reads or writes price-oracle data that also rely on the `…edd00` sub-slot through the other library; and
4. Bob's protocol experiences corrupted state (fuse type mappings overwriting price source mappings), which can lead to halted functionality or mispriced operations.

The underlying cause is the contradiction between the comments (which imply distinct namespaces per struct) and the actual constants (which allocate contiguous sub-slots within a single namespace). If developers rely on the comments and co-locate unrelated libraries, unintended storage overlap can occur.

Recommendation

Align the documentation and constants with the intended EIP-7201 pattern, and enforce namespace uniqueness across libraries.

Fix the vulnerability by either:

- Deriving each constant exactly as documented (distinct `keccak256("<namespace>")` seeds, then mask with `~0xff`) so that high 31 bytes differ across structs; or
- Explicitly adopt a single namespace for this module and document that the LSB is used for sub-slot indexing (e.g., `…edd00` through `…edd05`), then guarantee uniqueness across the entire codebase by:
  - Defining one top-level namespace per module and allocating sub-slots within that module only; and
  - Adding tests or static checks to fail builds on cross-module slot reuse.

Additionally:

- Consider computing and assigning constants at build time from their namespace strings (a small script) to prevent transcription errors;
- Keep the comments authoritative: if using offsets from one base, replace the current per-struct hash comments with a description of the chosen namespace and the specific offset for each struct.

Go back to Findings Summary

I1: ETH can be forcibly sent and permanently stuck; no recovery mechanism

Impact:	Info	Confidence:	High
Target:	contracts/zaps/ERC4626ZapInAllowance.sol	Type:	Code quality

Description

The contract explicitly rejects ETH via `receive` and `fallback` by reverting, but ETH can still be forcibly sent to this address using `selfdestruct` from another contract. Post-EIP-6780, `selfdestruct` still transfers the sending contract's balance to the target; it simply no longer deletes the target account in most cases. Because the contract exposes no payable sweep function, any forcibly sent ETH becomes permanently stuck.

Affected elements:

- Functions `receive` and `fallback` (both revert)
- No ETH recovery function is present

This results in a stuck-funds condition and can leave the contract with a non-zero ETH balance, which may surprise integrators or monitoring systems.

Listing 104. Code snippet from contracts/zaps/ERC4626ZapInAllowance.sol

```
64 receive() external payable {
65     revert EthTransfersNotAccepted();
66 }
67
68 fallback() external payable {
69     revert EthTransfersNotAccepted();
70 }
```

Exploit Scenario

The behavior can be used for griefing or to strand ETH forever at this address.

Consider the following attack scenario:

1. Alice deploys a throwaway helper contract, funds it with 1 wei, and calls `selfdestruct(target)` where `target` is this contract.
2. The EVM transfers the helper's entire ETH balance to the `target` address regardless of `receive`/`fallback` logic.
3. The contract has no `sweepETH` or similar function, so the ETH cannot be recovered by anyone.
4. Bob, operating the protocol, observes a non-zero ETH balance stuck at the contract address without a means to withdraw it.

Recommendation

Add an explicit recovery path while continuing to reject direct ETH:

- Implement a restricted `sweepETH(address to)` that transfers the full ETH balance to a trusted recipient (for example, the `ERC4626_ZAP_IN` contract or an owner address).
- Guard the function with appropriate access control and, if sending to an arbitrary address, consider using `nonReentrant` and the checks-effects-interactions pattern.
- Optionally, add a `recoverERC20(address token, address to)` to rescue mistakenly sent ERC20 tokens as well.

This preserves the current behavior for normal flows and prevents permanent loss of ETH forcibly sent via `selfdestruct`.

Go back to Findings Summary

I2: Unused on-chain deployment of ERC4626ZapInAllowance and exposed public address

Impact:	Info	Confidence:	High
Target:	./contracts/zaps/ERC4626ZapInWithNativeToken.sol	Type:	Unused code

Description

The constructor deploys a new `ERC4626ZapInAllowance` instance and stores its address in the public immutable `ZAP_IN_ALLOWANCE_CONTRACT`, but the variable is never read from or used by the contract. This adds unnecessary bytecode size and deployment gas cost, and the auto-generated public getter exposes an address that integrators may assume is functionally relevant.

Leaving an unused, public contract reference in immutable storage increases cognitive load and invites misuse. External users may transfer or approve tokens to that address expecting the zap to leverage it, but no logic utilizes it.

Listing 105. Code snippet from contracts/zaps/ERC4626ZapInWithNativeToken.sol

```
56     /// @notice Address of the ZapInAllowance contract that handles token
   transfers
57     /// @dev Immutable contract address created in constructor
58     address public immutable ZAP_IN_ALLOWANCE_CONTRACT;
59     address public referralContractAddress;
60
61     /// @notice Address of the current user performing a zap-in operation
62     /// @dev This is set during the zapIn function execution and cleared
   afterwards
63     address public currentZapSender;
64
65     constructor() Ownable(msg.sender) {
66         ZAP_IN_ALLOWANCE_CONTRACT = address(new ERC4626ZapInAllowance(
```

```
    address(this)));
67      }
```

Exploit Scenario

While not directly exploitable for theft, the presence of an unused, publicly exposed address can mislead integrators and cause operational errors.

Consider the following attack scenario:

1. Alice integrates the zap and discovers the public `ZAP_IN_ALLOWANCE_CONTRACT` address;
2. Alice assumes the zap uses this helper and sends approvals or transfers to that address to prepare deposits;
3. The zap never reads or calls that address during `zapIn`, so those approvals or transfers are never used; and
4. Bob's integration fails in production, or funds become stuck at an unrelated address, leading to user loss and support burden.

Recommendation

Remove dead code and the unused on-chain deployment, or make the address private and wire it into actual logic if it is required by design.

Recommended steps:

- If the helper is unnecessary, delete the `ZAP_IN_ALLOWANCE_CONTRACT` variable and the `new ERC4626ZapInAllowance` constructor call.
- If the helper is intended, refactor the zap to explicitly use it (e.g., for controlled token pulls), document the trust model, and add tests that cover its usage.
- Avoid exposing unused public getters that can confuse integrators; prefer private/internal references unless they are part of the contract's API

surface.

Go back to Findings Summary

I3: Unused public immutable VERSION adds dead code and exposes misleading getter

Impact:	Info	Confidence:	High
Target:	contracts/fuses/euler/EulerV2BalanceFuse.sol	Type:	Unused code

Description

The contract declares a public immutable `VERSION` and assigns it to `address(this)` in the constructor, but the value is never read anywhere else in the contract. Because it is `public`, the compiler generates an external getter `VERSION()` that returns the contract address, which is not a semantic version. This expands the bytecode and ABI surface without functional benefit and can mislead integrators that might treat `VERSION` as a conventional version identifier shared across fuses.

Affected elements:

- State variable `VERSION` declared at line 31.
- Single write in constructor at line 51.
- No reads from `VERSION` in any function, including `balanceOf`.

Technical evidence collected with Wake:

- State variables: `VERSION` (address, public, immutable) at line 31; `MARKET_ID` at line 35; `EVC` at line 39.
- Variable reads: only the assignment site; no operational reads.
- Variable writes: single assignment in the constructor at line 51.

Listing 106. Code snippet from contracts/fuses/euler/EulerV2BalanceFuse.sol

```
29      /// @notice Address of this fuse contract version
30      /// @dev Immutable value set in constructor, used for tracking and
    versioning
31      address public immutable VERSION;
32
33      /// @notice Market ID this fuse operates on
34      /// @dev Immutable value set in constructor, used to retrieve market
    substrates (Euler V2 vault addresses)
35      uint256 public immutable MARKET_ID;
36
37      /// @notice Ethereum Vault Connector (EVC) address for Euler V2 protocol
38      /// @dev Immutable value set in constructor, used for Euler V2 protocol
    interactions
39      IEVC public immutable EVC;
40
41      /**
42
43       * @notice Initializes the EulerV2BalanceFuse with a market ID and EVC
    address
44       * @param marketId_ The market ID used to identify the Euler V2 vault
    substrates
45       * @param eulerV2EVC_ The address of the Ethereum Vault Connector for
    Euler V2 protocol (must not be address(0))
46       * @dev Reverts if eulerV2EVC_ is zero address
47       */
48
49      constructor(uint256 marketId_, address eulerV2EVC_) {
50          if (eulerV2EVC_ == address(0)) {
51              revert Errors.WrongAddress();
52          }
53          VERSION = address(this);
54          MARKET_ID = marketId_;
55          EVC = IEVC(eulerV2EVC_);
56      }
```

Exploit Scenario

There is no direct on-chain exploit leading to fund loss; however, the exposed getter `VERSION()` can create operational risk when external systems rely on it as a version indicator.

Consider the following scenario:

- Alice deploys automation that whitelists fuses only if `VERSION()` matches an expected semantic version value.
- Alice includes this fuse, where `VERSION()` returns the contract address instead of a semantic version.
- The off-chain gate treats any non-zero value as a valid version and proceeds.
- Bob's integration enables this fuse under incorrect assumptions, potentially skipping intended guardrails or misclassifying the deployment in monitoring systems.

Recommendation

Remove the dead code or provide a meaningful versioning primitive with consistent semantics across fuses.

Fix options:

- Remove `VERSION` entirely from this fuse if unused.
- Replace `VERSION` with a semantic primitive (for example `uint64 public constant VERSION = 1;` or `bytes32 public constant VERSION_ID = keccak256("EulerV2BalanceFuse/1.0.0");`).
- Implement a dedicated `version()` getter (for example `function version() external pure returns (uint64)`), returning a stable value.

Update documentation to reflect the chosen scheme so integrators do not treat the getter as a generic semantic version when it is not.

Go back to Findings Summary

I4: Unused custom error FuseDoesNotHaveMarketId is dead code and misleading

Impact:	Info	Confidence:	High
Target:	./contracts/fuses/whitelist/FuseWhitelistLib.sol	Type:	Unused code

Description

The custom error `FuseDoesNotHaveMarketId(address)` is declared but never used. The library intentionally swallows failures when a fuse contract does not implement `MARKET_ID`, using a `try/catch` block in `addFuseToMarketId` and performing no action on failure. This renders the custom error dead code and increases cognitive overhead for maintainers.

Listing 107. Code snippet from contracts/fuses/whitelist/FuseWhitelistLib.sol

```
97     /// @notice Thrown when attempting to update a fuse type that does not
   have a MARKET_ID() method
98     error FuseDoesNotHaveMarketId(address fuseAddress);
```

The error is never referenced elsewhere. Instead, the library uses a silent `try/catch` pattern when adding a fuse to a market ID.

Listing 108. Code snippet from contracts/fuses/whitelist/FuseWhitelistLib.sol

```
507     function addFuseToMarketId(address fuseAddress_) internal {
508         try IFuseCommon(fuseAddress_).MARKET_ID() returns (uint256 marketId)
    {
509             FuseInfoByMarketId storage fuseInfoByMarketId =
    _getFuseInfoByMarketIdSlot();
510             fuseInfoByMarketId.fusesByMarketId[marketId].push(fuseAddress_);
511
512             emit FuseAddedToMarketId(fuseAddress_, marketId);
```

```
513         } catch {
514             // Fuse doesn't have MARKET_ID() method, skip adding to market
    ID list
515             // This is intentionally empty - we want to silently skip fuses
    without MARKET_ID()
516             // solhint-disable-next-line no-empty-blocks
517         }
518     }
```

Evidence gathered with Wake tools confirms no references to the error exist in the codebase, supporting the assessment that it is unused.

Exploit Scenario

There is no direct exploitation path for this issue; it is a code-quality problem. The risk is operational: future contributors may assume that the library reverts when a fuse lacks `MARKET_ID`, while it actually proceeds silently.

Consider the following attack scenario:

1. Alice submits a fuse that lacks `MARKET_ID` and triggers a flow that the integrator expects to revert on missing market ID;
2. The system calls `addFuseToMarketId`, which silently skips adding the fuse to any market, without reverting or signaling a problem;
3. The calling process continues under the assumption that all indexing steps succeeded; and
4. Bob's monitoring and operational processes become inconsistent (e.g., missing index entries), causing downstream components to rely on incomplete data.

Recommendation

Simplify the interface and make the behavior explicit.

Fix the vulnerability by either:

- Removing the unused `FuseDoesNotHaveMarketId` error to reduce cognitive overhead; or
- Emitting a dedicated event (e.g., `FuseSkippedAddingToMarketId(address)`) inside the `catch` to provide observability; or
- If a revert is desired by design, replace the silent `catch` with a revert using the existing error, and update any call sites accordingly.

Add a unit test that asserts the chosen behavior (silent skip or revert) and, if skipping, asserts that no events beyond the observability event are emitted.

Go back to Findings Summary

I5: Dead immutable ASSET_DECIMALS (never read): bytecode bloat and hidden normalization assumptions

Impact:	Info	Confidence:	High
Target:	./contracts/price_oracle/price_feed/PtPriceFeed.sol	Type:	Unused code

Description

The immutable `ASSET_DECIMALS` captured from `SY.assetInfo()` is written once in the constructor and never read anywhere else in the contract. This leaves a dead, immutable field that increases bytecode size and weakens the reader's confidence that decimal normalization was considered end-to-end.

Affected elements:

- State variable: `ASSET_DECIMALS` (immutable)
- Functions: `constructor` (writes it), `latestRoundData` (does not read it)

Evidence from static analysis confirms there are no reads of `ASSET_DECIMALS` beyond the constructor assignment. Wake variable-reads analysis returns only the assignment context and no true reads at call sites.

Listing 109. Code snippet from contracts/price_oracle/price_feed/PtPriceFeed.sol

```
41      /// @notice The decimals of the underlying asset from SY.assetInfo()
42      uint8 public immutable ASSET_DECIMALS;
```

The constructor assigns the field once and it is never used elsewhere:

Listing 110. Code snippet from contracts/price_oracle/price_feed/PtPriceFeed.sol

```
91          (, address assetAddress, uint8 assetDecimals) = sy.assetInfo();
92
93          PENDLE_MARKET = pendleMarket_;
94          TWAP_WINDOW = twapWindow_;
95          PRICE_MIDDLEWARE = priceMiddleware_;
96          ASSET_ADDRESS = assetAddress;
97          ASSET_DECIMALS = assetDecimals;
98          USE_PENDLE_ORACLE_METHOD = usePendleOracleMethod;
```

Operational impact today is informational (no direct exploitation). However, the presence of a dead field in a price feed is a maintainability hazard and can mask missing invariants. For example, if future edits make rate selection or scaling depend on token decimals, the assumption that `ASSET_DECIMALS` participates in normalization may be wrong: the variable exists, but the logic does not use it.

Exploit Scenario

There is no direct adversarial exploitation in the current version. The realistic failure mode is a maintenance/configuration footgun where the dead field conceals a missing normalization.

Consider the following attack scenario:

1. Alice deploys a new market where the underlying token uses 6 decimals and expects the feed to incorporate token decimals into scaling.
2. Alice reads the code and sees `ASSET_DECIMALS` recorded from `SY.assetInfo()` and assumes it is applied in price normalization.
3. The contract never reads `ASSET_DECIMALS`; prices remain scaled without considering token decimals, creating a silent 10^N mismatch if future rate semantics relied on token decimals.

4. Bob's integration consumes the feed output as authoritative and suffers valuation drift due to the incorrect assumption.

Recommendation

Choose one of the following and apply consistently:

- Remove the dead field: delete `ASSET_DECIMALS` and its constructor assignment to reduce bytecode size and avoid confusion.
- Or enforce the intended invariant: if token decimals must influence scaling, wire `ASSET_DECIMALS` into the computation and/or validate assumptions explicitly. For example, assert expected price-decimal conventions from the middleware and normalize using a single, auditable formula.

Actionable steps:

- If not needed, remove the declaration at lines 41-42 and the assignment at line 97.
- If needed, extend `latestRoundData` to use `ASSET_DECIMALS` in the scaling path and add a check that the middleware's `priceDecimals` matches the expected convention for `ASSET_ADDRESS`.

Go back to Findings Summary

I6: Documentation claims per-asset grant validation that the implementation does not perform

Impact:	Info	Confidence:	High
Target:	contracts/fuses/aave_v3/AaveV3WithPriceOracleMiddlewareBalanceFuse.sol	Type:	Code quality

Description

The `balanceOf` docstring states that each asset is validated via `PlasmaVaultConfigLib.isSubstrateAsAssetGranted()` before processing, but the implementation iterates the substrates returned by `PlasmaVaultConfigLib.getMarketSubstrates(MARKET_ID)` without performing the stated per-asset check.

Documented behavior:

Listing 111. Code snippet from contracts/fuses/aave_v3/AaveV3WithPriceOracleMiddlewareBalanceFuse.sol

```
62 /// @dev This function iterates through all substrates (assets) configured
   for the MARKET_ID and calculates:
63 ///      1. Validates each asset using
   PlasmaVaultConfigLib.isSubstrateAsAssetGranted() before processing
64 ///      2. For each validated asset, retrieves the balance of aTokens
   (supplied assets) and debt tokens (borrowed assets)
```

Actual implementation starts from the configured substrates and does not invoke the grant check per asset:

Listing 112. Code snippet from contracts/fuses/aave_v3/AaveV3WithPriceOracleMiddlewareBalanceFuse.sol

```
79 bytes32[] memory assetsRaw =
```

```
    PlasmaVaultConfigLib.getMarketSubstrates(MARKET_ID);

    uint256 len = assetsRaw.length;

    if (len == 0) {
        return 0;
    }

    for (uint256 i; i < len; ++i) {
        asset = PlasmaVaultConfigLib.bytes32ToAddress(assetsRaw[i]);
        // ... no per-asset call to PlasmaVaultConfigLib.isSubstrateAsAssetGranted()
    }
```

While `getMarketSubstrates` is expected to return only granted substrates, the docstring as written can mislead maintainers or integrators into assuming an additional validation occurs in code. This discrepancy is a documentation/clarity issue rather than a security flaw.

Exploit Scenario

There is no direct exploit; the risk is that operators or integrators rely on guarantees the code does not enforce.

Consider the following scenario:

1. Alice reads the function documentation and believes each asset will be validated again with `isSubstrateAsAssetGranted` during `balanceOf`;
2. Alice configures or integrates the fuse under the assumption that per-asset grant checks happen at runtime inside `balanceOf`;
3. The system executes `balanceOf` by iterating the substrates returned by configuration without performing that extra check; and
4. Bob's operational expectations are violated due to the mismatch between documentation and actual behavior, potentially leading to incorrect assumptions in integrations or reviews.

Recommendation

Align the documentation and implementation to avoid misleading guarantees.

Two clear options:

- Update the docstring to state that the function iterates the substrates from `getMarketSubstrates(MARKET_ID)` and relies on configuration to provide only granted assets; or
- If defense-in-depth is desired, add an explicit per-asset check before processing each substrate and revert on failure.

Either path maintains clarity for maintainers and auditors, preventing incorrect assumptions about runtime validations.

Go back to Findings Summary

I7: Redundant getPoolDataProvider call inside per-asset loop increases gas and failure surface

Impact:	Info	Confidence:	High
Target:	./contracts/fuses/aave_v3/AaveV3WithPriceOracleMiddlewareBalanceFuse.sol	Type:	Gas optimization

Description

Within the per-asset loop, the code fetches the Aave V3 pool data provider in every iteration even though it is constant for the fuse instance. This adds an unnecessary external call per asset, increasing gas costs and the chance an otherwise unrelated failure aborts the balance update.

The repeated call occurs here:

Listing 113. Code snippet from contracts/fuses/aave_v3/AaveV3WithPriceOracleMiddlewareBalanceFuse.sol

```
111 (aTokenAddress, stableDebtTokenAddress, variableDebtTokenAddress) =
    IAavePoolDataProvider(
112
    IPoolAddressesProvider(AAVE_V3_POOL_ADDRESSES_PROVIDER).getPoolDataProvider(
    )
113     ).getReserveTokensAddresses(asset);
```

`getPoolDataProvider()` returns the same address across loop iterations. Caching it once before the loop and reusing the cached address avoids a repeated external call and reduces the probability of spurious failure during on-chain balance refresh.

Exploit Scenario

There is no direct security exploit; the issue manifests as avoidable gas usage

and a larger failure surface.

Consider the following scenario:

1. Alice triggers a vault action that updates balances for many substrates in the same Aave V3 market;
2. For each substrate, the fuse performs an extra external call to `getPoolDataProvider()` inside the loop;
3. The accumulated overhead increases gas consumption and the chance of external-call-related failure; and
4. The balance update may cost more than necessary or, in worst cases, revert due to a transient failure unrelated to the actual reserve query.

Recommendation

Cache the pool data provider address once and reuse it inside the loop.

For example:

- Before the loop, read `address dataProvider = IPoolAddressesProvider(AAVE_V3_POOL_ADDRESSES_PROVIDER).getPoolDataProvider();`.
- Inside the loop, call `IAavePoolDataProvider(dataProvider).getReserveTokensAddresses(asset)`.

This removes one external call per iteration and reduces the balance update's failure surface.

Go back to Findings Summary

I8: NatSpec/documentation mismatch: references `dexData` instead of `dexsData`

Impact:	Info	Confidence:	High
Target:	./contracts/fuses/universal_token_swapper/SwapExecutorRestricted.sol	Type:	Code quality

Description

The function documentation for `execute` lists a parameter name `dexData`, while the actual struct field used by the code is `dexsData` (`bytes[] dexsData`). This inconsistency can confuse integrators preparing calldata or reading autogenerated documentation, leading to integration mistakes and reverted transactions.

Listing 114. Code snippet from contracts/fuses/universal_token_swapper/SwapExecutorRestricted.sol

```
49      * - `dexData`: An array of encoded function call data corresponding to
   each DEX.
50      * - `tokenIn`: The address of the input token.
51      * - `tokenOut`: The address of the output token.
```

Exploit Scenario

Documentation inconsistencies can cause avoidable integration errors.

Consider the following attack scenario:

1. Alice implements an off-chain integration following the published NatSpec and uses a field named `dexData` when building calldata.
2. Alice calls `execute` with the misnamed field; the on-chain code expects `dexsData` and the transaction reverts or decodes incorrectly.

3. Alice retries and continues to fail until spotting the mismatch.
4. Bob's protocol suffers operational friction and wasted gas due to repeated reverts.

Recommendation

Align documentation with the implemented interface.

Recommended actions:

- Update the NatSpec for `execute` to say `dexsData` instead of `dexData`.
- Regenerate any published documentation and examples to reflect the correct field name.
- Optionally add tests that validate externally documented examples compile and execute against the ABI to prevent future drift.

Go back to Findings Summary

I9: Documentation mismatch: updateMarketsBalances is restricted but NatSpec claims public/no restriction

Impact:	Info	Confidence:	High
Target:	contracts/vaults/PlasmaVault.sol	Type:	Code quality

Description

Function `updateMarketsBalances` is declared with the `restricted` modifier, but its NatSpec states "Public function, no role restrictions".

This discrepancy is misleading for integrators and operators. The function is access-controlled via `AccessManaged` and not publicly callable by anyone. The mismatch can cause unexpected reverts in production and breaks least-surprise for downstream tooling.

Listing 115. Code snippet from contracts/vaults/PlasmaVault.sol

```
361 /// @param marketIds_ Array of market IDs to update
362 /// @return uint256 Updated total assets after balance refresh
363 /// @custom:access Public function, no role restrictions
364 function updateMarketsBalances(uint256[] calldata marketIds_) external
    restricted returns (uint256) {
365     if (marketIds_.length == 0) {
366         return totalAssets();
367     }
368     uint256 totalAssetsBefore = totalAssets();
369     _updateMarketsBalances(marketIds_);
370     _addPerformanceFee(totalAssetsBefore);
371
372     return totalAssets();
373 }
```

Exploit Scenario

An integrator relies on the NatSpec to orchestrate periodic balance refreshes

and fee accounting.

Consider the following attack scenario:

1. Alice, an operator of a monitoring bot, reads the documentation that says the function has no role restrictions;
2. Alice deploys a bot from a fresh EOA to call `updateMarketsBalances` on a schedule;
3. Every transaction reverts due to the `restricted` modifier and AccessManager checks; and
4. Bob's operations dashboards and automations stop working unexpectedly, leading to stale accounting and delayed fee realization.

Recommendation

Align code and documentation to avoid misleading integrators.

Fix the vulnerability by either:

- Updating the NatSpec to accurately state the function is `restricted` (and document the role required); or
- Removing/loosening the `restricted` modifier if truly intended to be publicly callable, and adding any necessary internal safety checks.

Also add an access-control test that asserts the expected revert for unauthorized callers, and update runbooks/integrations that depend on this function.

Go back to Findings Summary

I10: `enterTransient` NatSpec output layout does not match implementation

Impact:	Info	Confidence:	High
Target:	./contracts/fuses/euler/EulerV2BatchFuse.sol	Type:	Code quality

Description

The NatSpec for `enterTransient` documents an output layout that omits the asset and vault counts, while the implementation writes both lengths into the output array. Integrators decoding outputs exactly as documented will misparse the layout.

Documented layout (NatSpec):

- `outputs[0] = batchSize`
- `outputs[1..n] = assets`
- `outputs[n+1..m] = vaults`

Actual layout (code):

- `[batchSize, assetsLength, assets…, vaultsLength, vaults…]`

Affected elements:

- Function `enterTransient` documentation (lines 111-116);
- Function `enterTransient` implementation (lines 136-151).

Listing 116. Code snippet from contracts/fuses/euler/EulerV2BatchFuse.sol

```
111 /// @notice Executes a batch of Euler V2 operations using transient storage
    for parameters
112 /// @dev Reads ABI-encoded EulerV2BatchFuseData from transient storage
    inputs as concatenated bytes32 chunks,
```

```
113 ///      calls enter(), and writes outputs to transient storage.
114 ///      Input format: inputs[0..n] = bytes32 chunks of ABI-encoded
    EulerV2BatchFuseData
115 ///      Output format: outputs[0] = batchSize, outputs[1..n] = assets,
    outputs[n+1..m] = vaults
116 function enterTransient() external {
```

Listing 117. Code snippet from contracts/fuses/euler/EulerV2BatchFuse.sol

```
136 // Write outputs to transient storage
137 // Format: [batchSize, assetsLength, assets..., vaultsLength, vaults...]
138 uint256 assetsLen = assets.length;
139 uint256 vaultsLen = vaults.length;
140 bytes32[] memory outputs = new bytes32[](1 + 1 + assetsLen + 1 + vaultsLen);
141
142 outputs[0] = TypeConversionLib.toBytes32(batchSize);
143 outputs[1] = TypeConversionLib.toBytes32(assetsLen);
144
145 for (uint256 i; i < assetsLen; ++i) {
146     outputs[2 + i] = TypeConversionLib.toBytes32(assets[i]);
147 }
148
149 outputs[2 + assetsLen] = TypeConversionLib.toBytes32(vaultsLen);
150
151 for (uint256 i; i < vaultsLen; ++i) {
152     outputs[3 + assetsLen + i] = TypeConversionLib.toBytes32(vaults[i]);
153 }
```

Exploit Scenario

A parsing mismatch can cause integrators to misinterpret outputs and act on wrong data.

Consider the following sequence:

1. Alice integrates this fuse and writes an off-chain decoder following the documented layout (no length fields).
2. Alice calls `enterTransient` expecting `outputs[1]` to be the first asset address.

3. In reality, `outputs[1]` encodes `assets.length`; Alice's decoder treats that 32-byte length as an address.
4. Subsequent logic that uses the mis-parsed "address" fails or routes funds incorrectly, causing transaction failures or operational errors.
5. Bob's aggregator aborts the workflow or sends operations to the wrong vault address, leading to failed transactions and wasted gas.

Result: while not a direct on-chain exploit, the documentation error reliably leads to failed integrations or misrouted operations, with Bob bearing the operational fallout.

Recommendation

Make the documentation and implementation consistent:

- Either update the NatSpec of `enterTransient` to explicitly document the actual layout `[batchSize, assetsLength, assets…, vaultsLength, vaults…]`; or
- Change the implementation to match the documented layout (omit the length fields) and keep the layout stable going forward.

Whichever approach is chosen, add tests that assert the exact contents of `TransientStorageLib.getOutputs(VERSION)` to prevent future drift.

Go back to Findings Summary

I11: Two sources of truth for output decimals (decimals constant vs _decimals helper) risk divergence

Impact:	Info	Confidence:	High
Target:	./contracts/price_oracle/price_feed/PtPriceFeed.sol	Type:	Code quality

Description

The contract maintains two sources of truth for feed decimals:

- External interface: `uint8 public constant override decimals = 8;`
- Internal helper: `_decimals()` returns `8` and is used in the scaling formula.

Duplicating the same constant increases the risk of divergence. If one value changes while the other does not, the on-chain interface will advertise one decimal precision while internal math uses another, causing systematic mis-scaling of reported prices.

Affected elements:

- State constant: `decimals`
- Function: `_decimals`
- Call site: `latestRoundData` uses `_decimals()` in the scaling factor

Listing 118. Code snippet from contracts/price_oracle/price_feed/PtPriceFeed.sol

```
46      // solhint-disable-next-line const-name-snakecase
47      uint8 public constant override decimals = 8;
```

Listing 119. Code snippet from contracts/price_oracle/price_feed/PtPriceFeed.sol

```
116         (uint256 assetPrice, uint256 priceDecimals) =
117
    IPriceOracleMiddleware(PRICE_MIDDLEWARE).getAssetPrice(ASSET_ADDRESS);
118
119         uint256 scalingFactor = FEED_DECIMALS + priceDecimals - _decimals();
120         price = SafeCast.toInt256((unitPrice * assetPrice) / 10 **
    scalingFactor);
```

Listing 120. Code snippet from contracts/price_oracle/price_feed/PtPriceFeed.sol

```
129     function _decimals() internal pure returns (uint8) {
130         return 8;
131     }
```

This is currently benign because both return 8, but it is fragile: a future edit might update one without the other. Consumers rely on `decimals` to scale answers, while the contract's own math relies on `_decimals()`; a mismatch would make every answer off by a power of ten.

Exploit Scenario

The issue is exploitable as an integrity failure if the two values ever diverge during maintenance.

Consider the following attack scenario:

1. Alice upgrades the contract and changes the public `decimals` constant to `6` to align with a consumer expectation but forgets to update `_decimals()`.
2. The feed now reports `decimals() == 6`, but still divides by `10^(18 + priceDecimals - 8)` internally.
3. Every published answer is off by 10^2 relative to what integrators expect when scaling by the advertised `decimals`.

4. Bob's protocol ingests mis-scaled prices and makes economically incorrect decisions (e.g., under-collateralization or improper liquidation thresholds).

Recommendation

Unify the source of truth for output decimals to eliminate drift risk.

Fix the vulnerability by either:

- Removing `_decimals()` and referencing the `decimals` constant directly in the scaling formula; or
- Keeping a single internal constant (for example `uint8 internal constant OUTPUT_DECIMALS = 8;`) and use it both for the interface and for internal math (`decimals` should reference the same value).

Actionable steps:

- Replace `_decimals()` at lines 129-131 with a reference to the single source of truth.
- Update `latestRoundData` to compute `scalingFactor = FEED_DECIMALS + priceDecimals - decimals;` (or the unified constant), and delete `_decimals()` entirely.
- Add a unit test asserting that the external `decimals` and the internal scaling digits are equal.

Go back to Findings Summary

I12: Unused local variable `coinBalance` in latestRoundData

Impact:	Info	Confidence:	High
Target:	./contracts/price_oracle/price_feed/CurveStableSwapNGPriceFeed.sol	Type:	Unused code

Description

An unused local variable is declared in function `latestRoundData`. The variable `coinBalance` is never read or assigned after declaration, which makes it dead code. While it does not change the returned price, it reduces clarity and can conceal incomplete refactors where a value was intended to be used.

The function computes `coinAmountForOneShareArray` from pool balances but does not use `coinBalance` at all. This is a code-quality issue with no direct runtime impact, yet it slightly increases bytecode size and makes future maintenance riskier.

Affected elements:

- Function `latestRoundData`
- Local variable `coinBalance`

Listing 121. Code snippet from contracts/price_oracle/price_feed/CurveStableSwapNGPriceFeed.sol

```
46 uint256 totalSupply = ICurveStableSwapNG(CURVE_STABLE_SWAP_NG).totalSupply();
47 if (totalSupply == 0) {
48     revert CurveStableSwapNG_InvalidTotalSupply();
49 }
50 uint256 coinBalance;
51 uint256[] memory coinAmountForOneShareArray = new uint256[](N_COINS);
52 for (uint256 i; i < N_COINS; i++) {
53     coinAmountForOneShareArray[i] = Math.mulDiv(
```

```
54         ICurveStableSwapNG(CURVE_STABLE_SWAP_NG).balances(i),
55         10 ** DECIMALS_LP, /// @dev DECIMALS_LP is the decimals of the LP
   token, which is 18 and this is value of one share
56         totalSupply
57     );
58 }
```

Exploit Scenario

There is no on-chain exploitability: because `coinBalance` is never used, it does not influence any control flow or observable state.

Consider the following attack scenario:

1. Alice calls `latestRoundData` to obtain a price;
2. The function executes and ignores `coinBalance` entirely;
3. The call returns the computed price as usual; and
4. Bob's protocol receives the same output regardless, with no security impact.

Recommendation

Remove the dead code or wire it to the intended logic.

Fix the issue by either:

- Deleting the declaration `uint256 coinBalance;` from `latestRoundData`; or
- If a per-coin intermediate value was intended, compute and use it explicitly, and add tests that fail if it is removed.

Additionally, enforce cleanliness in CI:

- Fail builds on compiler warnings about unused variables; and
- Run static analysis (for example, Slither) to catch unused code early.

Go back to Findings Summary

I13: Unused custom errors: `EVCInvalidSelector`, `DuplicateAsset`, `InvalidBatchItem`

Impact:	Info	Confidence:	High
Target:	contracts/fuses/euler/EulerV2BatchFuse.sol	Type:	Code quality

Description

The contract declares multiple custom errors which are never used anywhere in the file: `EVCInvalidSelector`, `DuplicateAsset`, and `InvalidBatchItem`. Their presence suggests incomplete or removed validation paths (e.g., duplicate-asset checks), and it increases cognitive overhead for maintainers and auditors because the code advertises checks that do not actually execute.

Affected elements:

- Declared errors: `EVCInvalidSelector`, `DuplicateAsset`, `InvalidBatchItem` (lines 33, 37, 38);
- No `revert` sites exist for these errors in this contract (confirmed by repository grep).

Listing 122. Code snippet from contracts/fuses/euler/EulerV2BatchFuse.sol

```
31 error EmptyBatchItems();
32 error ZeroAddress();
33 error EVCInvalidSelector();
34 error EVCInvalidCollateral();
35 error EulerVaultInvalidPermissions();
36 error ArrayLengthMismatch();
37 error DuplicateAsset();
38 error InvalidBatchItem();
39 error UnsupportedOperation();
```

Exploit Scenario

A misleading error surface can cause incorrect assumptions about validation coverage.

Consider the following sequence:

1. Alice prepares a batch with duplicated entries in `assetsForApprovals`, expecting the fuse to revert with `DuplicateAsset` as implied by the declared error.
2. The fuse proceeds without any duplicate-asset validation because no such check is implemented.
3. Downstream logic (off-chain monitoring, error-based retries, or integration guards) that relies on a specific `DuplicateAsset` revert never triggers, leading to operational confusion or unexpected behavior.
4. Bob, the protocol operator, spends time diagnosing the issue and may ship brittle workarounds, increasing maintenance risk.

Result: although not a direct exploit against funds, the unused errors create misleading assurances, increasing the likelihood of integration bugs and masking missing validations.

Recommendation

Either implement the intended validations and use the declared errors, or remove the dead declarations:

- Implement duplicate-asset validation in `_validate` for `assetsForApprovals` and revert `DuplicateAsset` on repetition (e.g., hash-set check over addresses);
- If EVC selector vetting is expected, add an explicit branch in `_validateEvc` for unsupported selectors and `revert EVCInvalidSelector()` instead of the generic `UnsupportedOperation`;

- If invalid batch structure needs to be signaled, add a structural check and `revert InvalidBatchItem()` accordingly.

If these checks are intentionally out-of-scope for this fuse, delete the unused errors to reduce cognitive load and to keep the public interface accurate.

Go back to Findings Summary

I14: Unused custom error `CurveStableSwapNG_InvalidBalance` is declared but never used

Impact:	Info	Confidence:	High
Target:	./contracts/price_oracle/price_feed/CurveStableSwapNGPriceFeed.sol	Type:	Unused code

Description

The custom error `CurveStableSwapNG_InvalidBalance` is declared on the contract but never referenced by any check or revert. Unused custom errors add noise, increase deployment bytecode size, and may falsely suggest that a specific invariant is enforced when it is not.

Affected elements:

- Contract `CurveStableSwapNGPriceFeed`
- Custom error `CurveStableSwapNG_InvalidBalance`

Listing 123. Code snippet from contracts/price_oracle/price_feed/CurveStableSwapNGPriceFeed.sol

```
15 error ZeroAddress();
16 error PriceOracleMiddleware_InvalidPrice();
17 error CurveStableSwapNG_InvalidTotalSupply();
18 error CurveStableSwapNG_InvalidBalance();
```

Evidence: repository-wide search shows only the declaration and no usages.

Exploit Scenario

There is no direct exploit path stemming from an unused error; however, the absence of the expected revert can hide anomalies and weaken operational

defenses.

Consider the following attack scenario:

1. Alice interacts with the system when the underlying pool is in an anomalous state that operators expected to trigger an "invalid balance" failure;
2. `latestRoundData` does not validate balances and never reverts with `CurveStableSwapNG_InvalidBalance` because the error is unused;
3. The call succeeds and returns a price, so monitoring that relies on the specific revert is not triggered; and
4. Bob's protocol continues execution, potentially masking an operational issue that should have caused a fail-fast path.

Recommendation

Either remove the unused custom error or implement a concrete validation that uses it.

Fix the issue by either:

- Deleting `error CurveStableSwapNG_InvalidBalance();` to reduce bytecode size and noise; or
- Adding a precise, documented check that reverts with `CurveStableSwapNG_InvalidBalance` when a well-defined invariant is violated, and covering it with tests.

Keep the error surface minimal and ensure every declared custom error is exercised by tests.

Go back to Findings Summary

I15: Always-true boolean return in transferApprovedAssets is redundant and misleading

Impact:	Info	Confidence:	High
Target:	contracts/zaps/ERC4626ZapInAllowance.sol	Type:	Gas optimization

Description

Function `transferApprovedAssets` declares a named boolean return `success` but never assigns to it and unconditionally returns `true` after calling `SafeERC20.safeTransferFrom`. Because `SafeERC20.safeTransferFrom` reverts on failure, the function cannot return `false`; the boolean adds no safety signal, expands the ABI surface, and slightly increases gas.

Affected elements:

- Function `transferApprovedAssets` (external, only `onlyERC4626ZapIn`)
- Event `AssetsTransferred`
- State variable `ERC4626_ZAP_IN` is read but not implicated in the issue

This pattern is misleading for integrators that might expect a meaningful success flag. Any failure reverts before reaching the `return` statement. The correct semantic is "returns only on success."

Listing 124. Code snippet from contracts/zaps/ERC4626ZapInAllowance.sol

```
43 function transferApprovedAssets(address asset_, uint256 amount_) external
   onlyERC4626ZapIn returns (bool success) {
44     if (amount_ == 0) {
45         revert AmountIsZero();
46     }
47
48     if (asset_ == address(0)) {
49         revert AssetIsZero();
```

```
50      }
51
52      address currentZapSender =
    IERC4626ZapIn(ERC4626_ZAP_IN).currentZapSender();
53
54      if (currentZapSender == address(0)) {
55          revert CurrentZapSenderIsZero();
56      }
57
58      IERC20(asset_).safeTransferFrom(currentZapSender, ERC4626_ZAP_IN,
    amount_);
59
60      emit AssetsTransferred(currentZapSender, asset_, amount_);
61      return true;
62  }
```

Exploit Scenario

There is no profitable attack vector because failures revert before any boolean is returned. However, the interface can mislead integrators into believing a `false` value is possible.

Consider the following operational scenario:

1. Alice integrates the contract and branches on the returned boolean from `transferApprovedAssets`.
2. Alice calls with insufficient allowance; `SafeERC20.safeTransferFrom` reverts; there is no `false` return to handle.
3. After granting allowance, the function returns, always yielding `true` on success.
4. Bob, maintaining the integrator, realizes the boolean provides no actionable signal and removes the branch; the extra ABI surface and gas usage were unnecessary.

Recommendation

Simplify the ABI and eliminate the misleading return value:

- Remove the named return and the `returns (bool)` clause.
- Omit the terminal `return true;` and rely on "return on success, revert on failure" semantics provided by `SafeERC20`.
- If backward compatibility is required, deprecate the return value in documentation and emit a semantic version bump when removing it in a subsequent release.

Suggested shape:

- `function transferApprovedAssets(address asset_, uint256 amount_) external onlyERC4626ZapIn { … }`

Go back to Findings Summary

I16: Gas waste from copying large external parameter into memory instead of using calldata

Impact:	Info	Confidence:	High
Target:	./contracts/fuses/universal_token_swapper/SwapExecutorRestricted.sol	Type:	Gas optimization

Description

The external function `execute` takes its `SwapExecutorData` argument as `memory`. Because the struct contains dynamic arrays (`address[] dexs`, `bytes[] dexsData`), this causes the entire payload to be copied from calldata into memory on entry. The function only reads from `data_` and does not persist it, so this copy is unnecessary and increases gas usage, especially for routes with many legs.

Listing 125. Code snippet from contracts/fuses/universal_token_swapper/SwapExecutorRestricted.sol

```
54      function execute(SwapExecutorData memory data_) external restricted {
55          uint256 len = data_.dexs.length;
56          for (uint256 i; i < len; ++i) {
57              data_.dexs[i].functionCall(data_.dexsData[i]);
58          }
```

Exploit Scenario

Excess gas expenditure can materially increase costs on complex routes.

Consider the following attack scenario:

1. Alice constructs a route with 15 DEX calls (large `dexs` and `dexsData`).
2. Alice calls `execute`; the entire struct and its dynamic arrays are copied into memory before use.

3. The call consumes significantly more gas than necessary, reducing the available gas budget and increasing execution costs.
4. Bob's protocol and end users pay higher fees or hit gas limits on mainnet under heavy load.

Recommendation

Adopt `calldata` for read-only external parameters to avoid unnecessary copying.

Change the signature to `function execute(SwapExecutorData calldata data_) external restricted`.

This makes `dexs` and `dexsData` be treated as `calldata` arrays under the hood, eliminating the top-level copy while preserving functionality. Ensure any internal helpers accept `calldata` where applicable.

Go back to Findings Summary

I17: Comment claims fixed 8-decimal prices while implementation uses dynamic oracle decimals (documentation mismatch)

Impact:	Info	Confidence:	High
Target:	./contracts/fuses/euler/EulerV2BalanceFuse.sol	Type:	Code quality

Description

In `balanceOf`, an inline comment states that price is represented in 8 decimals, but the implementation correctly reads the price together with its decimal precision from the price oracle middleware and uses it in normalization. The mismatch is misleading and may cause maintainers or integrators to reason about the function with an incorrect 8-decimal assumption.

The code obtains the price and its `priceDecimals` from the oracle and uses `eulerVaultAssetDecimals + priceDecimals` when converting to WAD:

Listing 126. Code snippet from contracts/fuses/euler/EulerV2BalanceFuse.sol

```
84          uint256 price; // @dev price represented in 8 decimals (USD is quote
    asset)
85          uint256 priceDecimals;
86          uint256 balanceOfSubAccountInEulerVault;
87          uint256 balanceOfSubAccountInUnderlyingAsset;
88          EulerSubstrate memory substrate;
89          address subAccount;
90
91          for (uint256 i; i < len; ++i) {
92              substrate = EulerFuseLib.bytes32ToSubstrate(substrates[i]);
93
94              eulerVaultAsset = ERC4626(substrate.eulerVault).asset();
95
96              (price, priceDecimals) =
    IPriceOracleMiddleware(priceOracleMiddleware).getAssetPrice(eulerVaultAsset);
```

The default middleware in this repository returns USD prices with `QUOTE_CURRENCY_DECIMALS = 18` and reports that `decimals` to callers.

Listing 127. Code snippet from contracts/price_oracle/PriceOracleMiddleware.sol

```
25     /// @dev Number of decimals used for USD price representation
26     /// @notice Every price returned by a specific price feed is converted to
   this decimals
27     uint256 public constant QUOTE_CURRENCY_DECIMALS = 18;
```

Listing 128. Code snippet from contracts/price_oracle/PriceOracleMiddleware.sol

```
50     function getAssetPrice(address asset_) external view returns (uint256
   assetPrice, uint256 decimals) {
51         (assetPrice, decimals) = _getAssetPrice(asset_);
52     }
```

Therefore, the implementation handles dynamic oracle decimals correctly; the comment is inaccurate and should be aligned with actual behavior to avoid confusion in future changes and reviews.

Exploit Scenario

There is no direct exploitable impact. However, misleading documentation can lead to fragile integrations or future code changes that incorrectly assume 8-decimal prices.

Consider the following attack scenario:

1. Alice maintains an off-chain adapter that parses `balanceOf` outputs assuming an 8-decimal USD price and applies an extra scaling factor;
2. Alice deploys an on-chain upgrade or configuration that reuses this mistaken assumption in a new component interacting with vault balances;

3. The component miscalculates balance thresholds and proceeds with operations under incorrect risk assumptions; and
4. Bob, a regular user, experiences unexpected behavior (for example, tighter or looser limits) due to the faulty integration.

Recommendation

Align comments and documentation with actual implementation to prevent incorrect assumptions.

Recommended actions:

- Replace the misleading inline comment `price represented in 8 decimals` with wording that reflects dynamic oracle decimals (for example, "price and `priceDecimals` are returned by the oracle; values are normalized to WAD").
- Ensure repository-level documentation specifies that the price oracle middleware typically returns 18-decimal USD prices, but the code reads decimals dynamically and must not assume a fixed precision.
- Optionally, add an assertion in tests that `priceDecimals` equals `QUOTE_CURRENCY_DECIMALS` for the configured middleware to prevent regressions.

Go back to Findings Summary

Appendix A: How to cite

Please cite this document as:

Ackee Blockchain Security, Wake Arena Report | IPOR-Labs: ipor-fusion, Wake Arena AI Report.

ackee
blockchain security

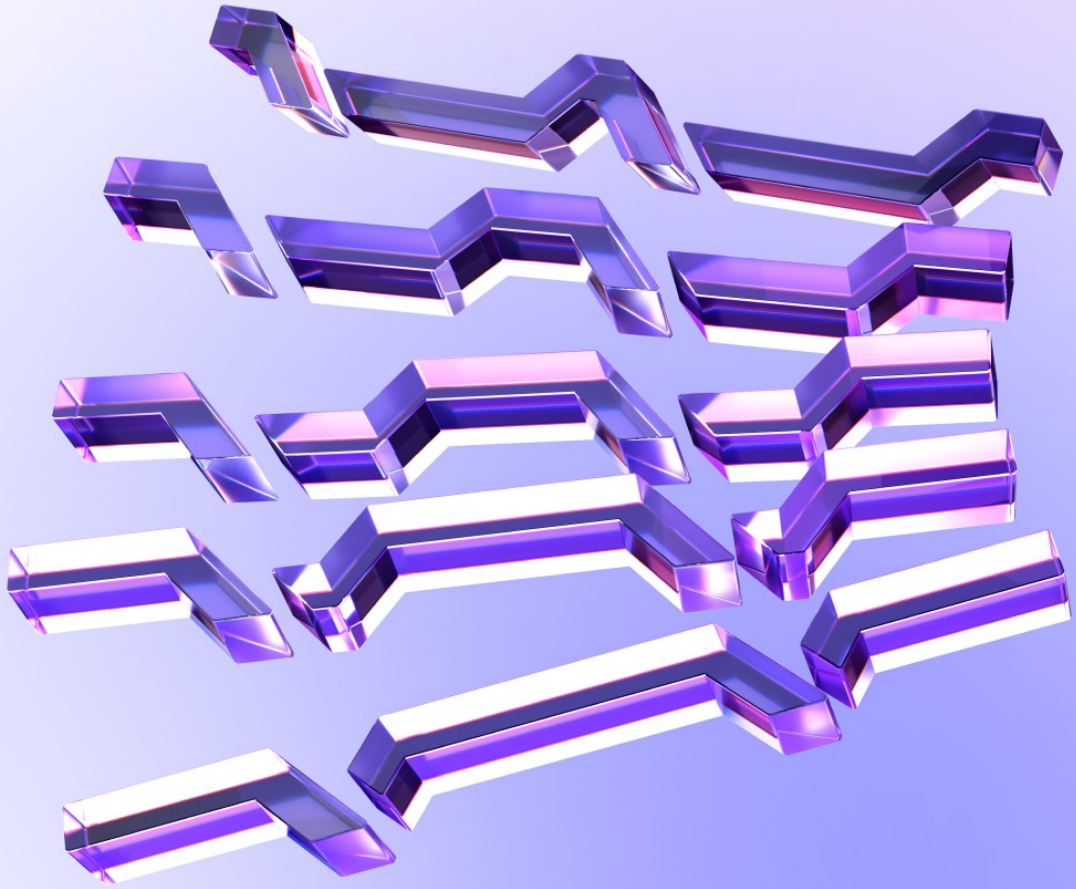
Wake

Thank You

Ackee Blockchain a.s.

Rohanske nabrezi 717/4
186 00 Prague
Czech Republic

hello@ackee.xyz



Wake Arena Report